

UNIVERSIDAD AUSTRAL | INGENIERÍA

Desarrollo y evaluación de un prototipo de sistema inteligente para consulta de normativa nacional argentina mediante recuperación agéntica

Tesis de Maestría

Leonardo Ulises Spairani

Tutores
Dra. Laura Lucía Bélfer y Dr. Rafael Crescenzi

Junio, 2026

Versión: 2026.06.24.20.04.50

Resumen

Presentamos el diseño, implementación y evaluación de un sistema Agentic-RAG de código abierto para la consulta del Código de Minería Argentino. El acceso a la normativa legal argentina está limitado por sistemas de búsqueda basados en coincidencias léxicas que no capturan la semántica del lenguaje jurídico, mientras que las herramientas avanzadas existentes son predominantemente comerciales y de acceso restringido. El sistema integra una base de datos vectorial con embeddings semánticos, una arquitectura multi-agente con agentes especializados en recuperación y generación de respuestas, y un pipeline de evaluación LLM-as-a-Judge con tres jueces independientes. La evaluación emplea un diseño factorial 3×3 —sin RAG, RAG básico y Agentic-RAG combinados con tres modelos de lenguaje— sobre un benchmark de 25 preguntas legales (79 unidades de evaluación) validadas por profesionales del derecho argentino. Los resultados revelan una interacción significativa entre estrategia de recuperación y modelo de lenguaje ($H = 56.62$, $p < 10^{-11}$). El análisis de efectos simples con corrección de Holm-Bonferroni muestra que el RAG básico elimina las diferencias de desempeño entre modelos de distinta capacidad, mientras que Agentic-RAG beneficia significativamente a gpt-5-mini (mediana de 0.92 frente a 0.33 sin RAG, $p_{\text{corr}} < 0.0001$) pero degrada a gpt-5-nano frente a RAG básico (0.08 vs. 0.58, $p_{\text{corr}} = 0.017$). Para gpt-5, Agentic-RAG no produce diferencias significativas respecto a las otras estrategias tras la corrección por comparaciones múltiples.

Palabras claves

Agentic-RAG, Generación Aumentada por Recuperación, Código de Minería Argentino, PLN jurídico, LLM-as-a-Judge, benchmark legal, base de datos vectorial, agentes inteligentes

Dedicatoria

A mi familia ...

Declaraciones

Se han empleado productos basados en inteligencia artificial para asistir en la elaboración de este trabajo.

Índice

1 Introducción 1
1.1 Motivación e importancia del tema de estudio 1
1.2 Requerimientos y desafíos 2
1.3 Problemas y soluciones 3
1.3.1 Problemas no resueltos 3
1.3.2 Solución propuesta 4
1.3.3 Limitaciones de la solución propuesta 5
1.4 Resumen de problemas y soluciones 6
1.5 Objetivo del trabajo 6
1.5.1 Objetivo general 6
1.5.2 Objetivos específicos 6
1.6 Transferencia de los resultados obtenidos 6
1.7 Contribuciones principales 7
1.8 Trabajos presentados vinculados con el trabajo final 7
1.9 Estructura general del trabajo final 7
2 Fundamentos e Historia del procesamiento del lenguaje natural (PLN) 9
2.1 Introducción al Procesamiento de Lenguaje Natural 9
2.2 Evolución de los modelos del lenguaje 11
2.2.1 Modelos estadísticos 12
2.2.2 Sistemas basados en reglas y en recuperación de información 15
2.2.3 Modelos basados en espacio de vectores y espacio de estados 16
2.2.4 Modelos de lenguaje basados en redes neuronales artificiales 18
2.2.4.1 El perceptrón y las primeras redes neuronales artificiales 18
2.2.4.2 Redes neuronales recurrentes (RNN) 20
2.2.4.3 Memorias de corto y largo plazo (LSTM) 21
2.2.4.4 Unidades recurrentes con compuertas (GRU) 22
2.2.5 La arquitectura Transformer 23
2.2.6 Modelos fundacionales basados en Transformer 24
2.2.7 Innovaciones recientes en modelos de lenguaje 25
2.2.7.1 2022 25
2.2.7.2 2023 26
2.2.7.3 2024 y 2025 26
2.2.7.4 Desarrollos posteriores al alcance de este trabajo 26
2.3 Síntesis del capítulo 27
3 Aplicación del PLN al Dominio Legal 28
3.1 Características del dominio legal como problema de PLN 28
3.2 Breve historia del PLN aplicado al derecho 29
3.3 Tareas típicas de PLN en el dominio legal 31
3.4 Adaptación de grandes modelos de lenguaje al dominio legal 32
3.5 Prompt engineering y razonamiento encadenado para tareas legales 33
3.6 Alineamiento, sesgos y equidad en modelos del lenguaje jurídicos 35
3.7 Riesgos centrales: alucinaciones y citaciones falsas 36

3.8 Síntesis del capítulo 38
4 Agentic-RAG: Generación Aumentada por Recuperación Agéntica 39
4.1 Concepto de Agentic-RAG 39
4.2 Arquitecturas de Agentic-RAG 39
4.2.1 De RAG clásico a Agentic-RAG 39
4.2.2 Descomposición modular típica 40
4.2.3 Patrones de flujo de trabajo agéntico aplicados a RAG 40
4.2.4 Arquitecturas jerárquicas y multi-herramienta 41
4.2.5 Arquitecturas multi-agente 41
4.2.6 Comparativa de arquitecturas 42
4.3 Agentes especializados en Agentic-RAG 43
4.3.1 Tipos de agentes por rol en el pipeline 43
4.3.2 Ejemplos concretos de agentes especializados 43
4.3.3 Agentes de recuperación avanzados 44
4.3.4 Evaluadores, jueces y supervisores 44
4.4 Flujo de trabajo en Agentic-RAG 44
4.4.1 Comparación de flujos: RAG estático vs. Agentic-RAG 44
4.4.2 Flujo de trabajo genérico de Agentic-RAG 45
4.4.3 Patrones de flujos de trabajo específicos 46
4.4.4 Entrenamiento y optimización del flujo 46
4.5 Casos de uso de Agentic-RAG 46
4.5.1 Dominios donde Agentic-RAG aporta ventajas claras 46
4.5.2 Salud y diagnóstico clínico 47
4.5.3 Recomendadores y personalización 47
4.5.4 Pruebas de software y entornos empresariales 47
4.5.5 Dominios regulados y de alto riesgo 48
4.5.6 Investigación académica y análisis político 48
4.5.7 Benchmarks y plataformas de evaluación 48
4.6 Consideraciones prácticas para ingeniería 49
4.6.1 Diseño de arquitecturas en producción 49
4.6.2 Compromisos clave 49
4.6.3 Direcciones abiertas de investigación 49
4.7 LLM-as-a-judge 50
4.7.1 Definición y fundamentos 50
4.7.2 Taxonomía de atributos evaluables 50
4.7.3 Metodologías de prompting 51
4.7.4 Métodos de ajuste fino 51
4.7.5 Sesgos y vulnerabilidades 52
4.7.6 LLM-as-a-judge en sistemas RAG 52
4.7.7 Meta-evaluación de LLM-as-a-judge 53
4.7.8 Implicaciones para esta tesis 53
4.8 Síntesis del capítulo 53
5 Metodología e Implementación 55
5.1 Introducción 55

5.2 Enfoque Metodológico 55
5.2.1 Diseño de la Investigación 55
5.2.2 Fases del Proyecto 55
5.3 Selección y Extracción del Corpus 56
5.3.1 Características del Código de Minería Argentino 56
5.3.2 Obtención del Texto Legal 56
5.3.3 Extracción de Texto mediante Modelos de Visión 56
5.3.4 Validación de Calidad de Extracción 57
5.3.5 Segmentación y Estructuración del Corpus 57
5.4 Base de Datos Vectorial 58
5.4.1 Selección de la Base de Datos Vectorial 58
5.4.2 Esquema de Datos 58
5.4.3 Creación de Índices 58
5.4.4 Estrategia de Chunking 58
5.5 Modelo de Embeddings 59
5.5.1 Criterios de Selección 59
5.5.2 Modelo Seleccionado 59
5.5.3 Generación e Inserción de Embeddings 60
5.6 Sistema RAG Básico 60
5.6.1 Módulo de Recuperación 60
5.6.2 Módulo de Generación 60
5.6.3 Pipeline Completo 60
5.7 Sistema Agentic-RAG 61
5.7.1 Arquitectura General 61
5.7.2 Framework de Orquestación (Pydantic AI) 61
5.7.3 Agente Especialista Legal 61
5.7.4 Agente de Recuperación 62
5.7.5 Agente de Calidad de Recuperación (Diseñado pero no Activado) . . 63
5.7.6 Orquestación y Flujo de Comunicación 64
5.7.7 Logging y Trazabilidad 64
5.7.8 Definición de Agentes y Herramientas 64
5.7.8.1 Agente Especialista Legal 65
5.7.8.2 Agente de Recuperación 65
5.7.9 Ejemplo de Interacción entre Agentes 66
5.7.10 Hiperparámetros de Configuración 67
5.8 Selección de Modelos LLM 67
5.8.1 Criterios de Selección 67
5.8.2 Modelos Evaluados 68
5.8.3 Costos por Token 68
5.9 Diseño del Benchmark 68
5.9.1 Generación de Preguntas 68
5.9.2 Estructura del Benchmark 68
5.9.3 Colaboración con Profesionales del Derecho 69
5.10 Métricas y Sistema de Evaluación 69

5.10.1 Métricas de Similitud Textual 69
5.10.2 Métricas Semánticas 70
5.10.3 Evaluación LLM-as-a-Judge 70
5.11 Gestión de Costos y Tokens 71
5.11.1 Tracking de Uso 71
5.11.2 Optimización de Llamadas 72
5.12 Herramientas y Tecnologías 72
5.12.1 Lenguajes y Frameworks 72
5.12.2 Infraestructura 72
5.12.3 Análisis Estadístico 72
5.12.3.1 Verificación de Normalidad (Shapiro-Wilk) 72
5.12.3.2 Comparaciones No Paramétricas (Kruskal-Wallis y Dunn) . 73
5.12.3.3 Análisis de Interacción Factorial (Scheirer-Ray-Hare) 73
5.12.3.4 Análisis de Acuerdo Inter-Juez (Krippendorff's Alpha) 73
5.13 Desafíos Técnicos y Soluciones 73
5.13.1 Manejo de Errores y Reintentos 73
5.13.2 Optimización de Rendimiento 74
5.14 Repositorio y Documentación 74
5.14.1 Estructura del Repositorio 74
5.14.2 Documentación Técnica 74
5.15 Conclusiones del Capítulo 74
6 Evaluación y Resultados 76
6.1 Introducción 76
6.2 Diseño Experimental 76
6.3 Exploración Descriptiva y Selección de Variables de Respuesta 76
6.3.1 Análisis de Correlación entre Métricas 76
6.3.2 Selección de Variables de Respuesta 77
6.3.3 Imputación de Valores Fuera de Rúbrica 79
6.4 Verificación de Normalidad 80
6.4.1 Test de Shapiro-Wilk 80
6.4.2 Resultados 80
6.4.3 Implicación Metodológica 80
6.5 Comparaciones entre Estrategias y Modelos 80
6.5.1 Test de Kruskal-Wallis 80
6.5.1.1 Efecto de la Estrategia de Recuperación 80
6.5.1.2 Efecto del Modelo 80
6.5.2 Prueba Post-Hoc de Dunn 80
6.5.3 Análisis Estratificado: Efecto del Modelo dentro de cada Sistema . . 81
6.5.4 Efectos Simples: Estrategia dentro de cada Modelo 81
6.6 Análisis Factorial: Interacción Estrategia × Modelo 83
6.6.1 Test de Scheirer-Ray-Hare 83
6.6.2 Resultados de la Interacción 83
6.6.3 Patrón de Interacción 84
6.6.4 Implicaciones para Resultados Previos 85

6.7 Análisis de Costos ... 85
6.7.1 Costos Descriptivos por Modelo y Sistema ... 85
6.7.2 Relación Costo-Desempeño ... 86
6.7.3 Índice de Eficiencia ... 87
6.7.4 Resumen de Tokens y Costos del Experimento ... 87
6.8 Evaluación LLM-as-a-Judge ... 88
6.8.1 Modelos de Juez ... 88
6.8.2 Resultados de «respuesta_correcta» ... 88
6.8.3 Resultados de Precisión Legal ... 88
6.9 Acuerdo Inter-Juez ... 88
6.9.1 Correlación de Spearman entre Jueces ... 88
6.9.2 Alpha de Krippendorff ... 89
6.9.2.1 Alpha por Sistema de Recuperación ... 89
6.9.3 Test Chi-Cuadrado de Homogeneidad ... 90
6.9.4 Decisión sobre `score_precision_legal` ... 90
6.9.5 Análisis de Sesgo Sistemático ... 90
6.10 Discusión de Resultados ... 90
6.10.1 Fortalezas del Sistema ... 90
6.10.2 Debilidades y Limitaciones ... 90
6.10.3 Implicaciones Prácticas ... 91
6.11 Conclusiones del Capítulo ... 91
7 Trabajos Relacionados ... 93
7.1 Introducción ... 93
7.2 Sistemas de Búsqueda Legal ... 93
7.2.1 Sistemas Tradicionales Basados en Palabras Clave ... 93
7.2.2 Sistemas Comerciales con IA ... 94
7.3 Sistemas RAG en Dominios Especializados ... 95
7.3.1 RAG para el Dominio Legal ... 95
7.3.2 RAG para Documentación Técnica y Otros Dominios ... 96
7.4 Sistemas Agénticos ... 96
7.4.1 Agentes para Tareas Legales ... 96
7.4.2 Agentic-RAG en la Literatura ... 97
7.5 Benchmarks para Evaluación Legal ... 97
7.5.1 LegalBench, LexGLUE y Otros Benchmarks en Inglés ... 97
7.5.2 Benchmarks en Español y Modelos para el Dominio Legal Hispanohablante ... 98
7.6 Comparación con el Presente Trabajo ... 99
7.6.1 Diferencias Metodológicas y Contribuciones Originales ... 99
7.7 Desarrollos Posteriores al Período Experimental ... 100
7.7.1 Modelos Propietarios de Nueva Generación ... 101
7.7.2 Modelos de Código Abierto y Razonamiento ... 101
7.7.3 *Frameworks* Agénticos y Agentic-RAG ... 102
7.7.4 Implicaciones para el Presente Trabajo ... 102
7.8 Conclusiones del Capítulo ... 103

8 Conclusiones y Trabajos Futuros 104
8.1 Introducción 104
8.2 Conclusiones 104
8.2.1 Cumplimiento de Objetivos 104
8.2.2 Contribuciones del Trabajo 105
8.2.3 Impacto de la Propuesta 106
8.3 Discusión de Resultados 106
8.3.1 Fortalezas del Sistema 106
8.3.2 Debilidades y Limitaciones 107
8.4 Trabajos Futuros 109
8.4.1 Extensión del Corpus Legal 109
8.4.2 Mejoras en la Arquitectura 109
8.4.3 Ampliación del Benchmark 109
8.4.4 Integración con Sistemas Existentes 110
8.4.5 Evaluación con Usuarios Reales 110
8.4.6 Optimización del Entrenamiento con Aprendizaje por Refuerzo 110
8.4.7 Re-evaluación sobre Modelos de Nueva Generación 110
8.5 Reflexiones Finales 110
Referencias 112
A Estructura del Repositorio de Código 128
A.1 Organización de Carpetas 128
A.2 Descripción de Componentes Principales 129
A.2.1 Módulos de Exploración y Extracción (`archivo/`) 129
A.2.2 Pipeline de Experimentación (`examples/notebooks/`) 130
A.2.3 Principales Dependencias del Proyecto 131
B Ejemplos de Consultas y Respuestas 132
B.1 Consulta de Ejemplo 132
B.2 Consultas sin RAG 132
B.3 Consultas con RAG Básico 133
B.4 Consultas con Agentic-RAG 133
C Benchmark Completo 135
C.1 Metadatos del Benchmark 135
C.2 Estructura del Benchmark 135
C.3 Ejemplos de Preguntas 135
C.3.1 Pregunta 1: Dominio de Yacimiento de Litio en Propiedad Privada 136
C.3.2 Pregunta 2: Conflictos de Interés de Funcionario Judicial con Actividad Minera 136
C.4 Respuestas Esperadas (Ejemplos) 136
C.4.1 Respuestas a la Pregunta 1 136
C.4.2 Respuestas a la Pregunta 2 137
Acrónimos y Siglas 138

Índice de Figuras

Figura 1 Cronología de los hitos principales en la evolución de los modelos del lenguaje (1913–1990s). Primera parte. 10
Figura 2 Cronología de los hitos principales en la evolución de los modelos del lenguaje (2003–2021). Segunda parte. 11
Figura 3 Diagrama de flujo de generación de texto basado en n-gramas, siguiendo el método de Shannon (1948). 14
Figura 4 Flujo de procesamiento de ELIZA. Basado en el mecanismo de descomposición y reensamblaje descrito por (Weizenbaum, 1966). 16
Figura 5 Visualización t-SNE de *word embeddings* generados a partir de literatura del siglo XIX. Los puntos próximos en el plano corresponden a palabras con significado o contexto de uso similar, ilustrando cómo los VSM codifican relaciones semánticas como relaciones geométricas entre vectores (Grayson, 2017). 17
Figura 6 Diagrama de bloques de un bloque Mamba, una arquitectura de espacio de estados selectivo. Se observa la aplicación de proyecciones lineales, convoluciones y el mecanismo de selección que vuelve a los parámetros del SSM función de la entrada. La recurrencia lineal $h_t = Ah_{\{t-1\}} + Bx_t$ permite procesar secuencias largas en tiempo lineal, a diferencia del costo cuadrático de la atención. Fuente: Emsi (2024) 18
Figura 7 Ejemplo esquemático de una red neuronal artificial multicapa (MLP). Se observan la capa de entrada (a la izquierda), una capas ocultas de neuronas interconectadas y la capa de salida (a la derecha). Cada conexión entre neuronas tiene un peso sináptico asociado, y la información fluye estrictamente hacia adelante (*feed-forward*). Fuente: Antimundo (2026) 20
Figura 8 Red neuronal recurrente (RNN) desplegada en el tiempo. La misma red (con pesos compartidos) se aplica en cada paso de la secuencia: el estado oculto h_t depende tanto de la entrada actual x_t como del estado anterior $h_{\{t-1\}}$, lo que le da a la red memoria de los pasos previos. Fuente: Ffets (2026) ... 21
Figura 9 Estructura interna de una celda LSTM. Se observan las tres compuertas: de olvido (f_t), de entrada (i_t) y de salida (o_t), que controlan respectivamente qué información se descarta, qué nueva información se incorpora y qué parte del estado de la celda C_t se propaga como salida. Fuente: fdeloche (2017a) 22
Figura 10 Estructura interna de una celda GRU. A diferencia de la LSTM, la GRU combina las compuertas de entrada y olvido en una sola *compuerta de actualización* z_t, y agrega una *compuerta de reinicio* r_t que controla cuánta información del pasado se descarta. No hay una celda de memoria separada: el estado oculto h_t integra la información directamente. Fuente: fdeloche (2017b). 23
Figura 11 Cronología de los hitos principales en la evolución de los modelos del lenguaje (2022–2025). Tercera parte. 25
Figura 12 Arquitectura básica de RAG clásico. 40
Figura 13 Arquitectura de Agentic-RAG con componentes agénticos. 41

Figura 14 Diagrama del flujo de generación de embeddings e inserción en la base de datos vectorial Milvus. 59
Figura 15 Configuración *Sin RAG*: el modelo genera la respuesta directamente a partir de su conocimiento paramétrico, sin acceso a documentos externos. 61
Figura 16 Configuración *RAG Básico*: la consulta del usuario se emplea para recuperar los 10 fragmentos más similares de la base vectorial, que se inyectan como contexto en el prompt del modelo. 62
Figura 17 Configuración *Agentic-RAG*: el Agente Especialista Legal delega la recuperación al Agente de Recuperación, que descompone la consulta en subconsultas y ejecuta múltiples búsquedas vectoriales (bucle). Los fragmentos recuperados se retornan directamente al Agente Especialista Legal para la generación de la respuesta. 63
Figura 18 Matriz de correlación de Spearman entre las 13 métricas textuales y las 2 métricas de evaluación agéntica. Los bloques de correlación alta (rojo intenso) delimitan los cuatro clústeres: léxico/n-gramas (recuadro superior-izquierdo), embeddings neuronales (recuadro central), volumen (recuadro inferior-derecho) y agéntico (últimas dos filas/columnas). Las líneas punteadas separan los clústeres identificados. 77
Figura 19 Biplot del análisis de componentes principales (PCA) sobre las 13 métricas textuales estandarizadas. Las flechas azules representan las métricas activas; las flechas rojas representan las variables agénticas proyectadas como suplementarias. PC1 (61,0%) captura principalmente el contraste entre métricas de similitud (derecha) y métricas de error/volumen (izquierda). PC2 (25,8%) separa las métricas de volumen del resto. `respuesta_correcta_avg` se ubica en una posición intermedia entre los clústeres semántico y léxico, mientras que `precision_legal_avg` se alinea más estrechamente con el clúster léxico. El círculo punteado indica el umbral de correlación $r = 0.6$ con las componentes principales. 78
Figura 20 Distribución de `respuesta_correcta_avg` por estrategia de recuperación. Las cajas representan el rango intercuartílico (IQR); la línea central indica la mediana; los diamantes rojos señalan la media. Se observa una tendencia general de mejora desde la configuración sin RAG hacia Agentic-RAG, con mayor variabilidad en esta última debido a la interacción con el modelo. 79
Figura 21 Gráficos de interacción modelo × estrategia de recuperación. Las líneas representan la media de cada modelo con intervalos de confianza del 95%. El cruce de las líneas confirma la interacción disordinal: el modelo óptimo depende de la estrategia empleada. 84
Figura 22 Costo medio por consulta (USD) para cada combinación modelo × estrategia de recuperación. El costo del sistema Agentic-RAG incluye las llamadas a los agentes de Recuperación y Calidad, que emplean gpt-5 independientemente del modelo del Agente Especialista. 86
Figura 23 Relación costo-desempeño para `respuesta_correcta_avg` y las cinco métricas complementarias. Cada punto representa una combinación modelo × estra-

tegia. El cuadrante superior-izquierdo (alto desempeño, bajo costo) contiene las configuraciones más eficientes. 86
Figura 24 Índice de eficiencia (desempeño / costo normalizado) para la métrica `respuesta_correcta_avg`. Las barras se colorean según el modelo y se texturizan según la estrategia. Las configuraciones más eficientes combinan alto desempeño con bajo costo relativo. 87
Figura 25 Distribución proporcional de puntajes por juez y rúbrica. Cada celda indica la proporción de evaluaciones que recibieron cada puntaje. Se observan diferencias en los patrones de distribución: algunos jueces tienden a concentrar sus evaluaciones en valores centrales (0,50–0,75), mientras que otros exhiben distribuciones más polarizadas. 88
Figura 26 Alpha de Krippendorff ordinal por sistema de recuperación y rúbrica. Las líneas punteadas indican los umbrales de acuerdo aceptable ($\alpha = 0.667$) y bueno ($\alpha = 0.8$). Se observa que `score_respuesta_correcta` supera el umbral aceptable en todos los sistemas, mientras que `score_precision_legal` cae por debajo en *base_rag* y *no_rag*. 89
Figura 27 Línea de tiempo de los principales desarrollos en LLMs y Agentic-RAG entre noviembre de 2025 y abril de 2026, período posterior a la finalización de los experimentos de este trabajo. 100
Figura 28 Respuesta sin RAG (gpt-5-nano) para la subpregunta 1.1. 132
Figura 29 Respuesta con RAG básico (gpt-5-nano) para la subpregunta 1.1. 133
Figura 30 Respuesta con Agentic-RAG (gpt-5-nano) para la subpregunta 1.1. 134

Índice de Tablas

Tabla 1 Resumen de problemas identificados y soluciones propuestas 6
Tabla 2 Comparativa de arquitecturas RAG. 42
Tabla 3 Diseño factorial 3 × 3 del experimento. Cada celda marcada representa una configuración experimental evaluada. 55
Tabla 4 Estadísticas del corpus del Código de Minería Argentino. Desglose por tipo de parte estructural. 57
Tabla 5 Esquema de la colección en Milvus. Cada artículo se almacena con su texto, metadatos estructurales y vector de embeddings de 768 dimensiones. 58
Tabla 6 Hiperparámetros de configuración de los agentes activos del sistema Agentic-RAG. †El modelo del Agente Especialista Legal varía según la configuración experimental (gpt-5-nano, gpt-5-mini o gpt-5). 67
Tabla 7 Costos por millón de tokens (Mtok) de los modelos GPT-5 empleados. El ratio indica el factor de costo relativo al modelo más económico (gpt-5-nano). . 68
Tabla 8 Rúbrica de evaluación `respuesta_correcta`: «La respuesta provista está alineada con la respuesta esperada.» Evaluada en español. 71
Tabla 9 Rúbrica de evaluación `precision_legal`: «La respuesta cita correctamente los artículos relevantes del Código de Minería Argentino.» Evaluada en español. 71
Tabla 10 Efectos simples de estrategia dentro de cada modelo (Mann-Whitney U). `Med. 1` y `Med. 2` son las medianas de `respuesta_correcta_avg` para cada estrategia. `p corr. Holm` es el p-value corregido por Holm-Bonferroni; ✓ indica significativo con corrección de Holm ($p_{\text{corr}} < 0.05$). $m = 9$ comparaciones. 82
Tabla 11 Medianas por configuración (estrategia × modelo). `Resp. correcta` es la variable de respuesta principal utilizada en las pruebas de hipótesis; las columnas restantes (`Token F1`, `ROUGE-L`, `SBERT (es)`, `BERTScore`, `Prec. legal`) se incluyen con fines descriptivos. Los valores más altos indican mejor desempeño. Destaca la mediana de 0,958 de Agentic-RAG + gpt-5 en respuesta correcta, y el valor anómalamente bajo de 0,083 para Agentic-RAG + gpt-5-nano. 83
Tabla 12 Resultados del test de Scheirer-Ray-Hare para `respuesta_correcta_avg`. . 84
Tabla 13 Medias de `respuesta_correcta_avg` por combinación modelo × sistema. . 85
Tabla 14 Alpha de Krippendorff global por rúbrica. 89
Tabla 15 Alpha de Krippendorff ordinal por sistema de recuperación. 89
Tabla 16 Comparación del presente trabajo con sistemas y trabajos relacionados .. 99
Tabla 17 Descripción de los directorios del pipeline de experimentación. 130
Tabla 18 Principales dependencias del proyecto (Python ≥ 3.13). 131
Tabla 19 Métricas de la respuesta sin RAG (gpt-5-nano) para la subpregunta 1.1. 133
Tabla 20 Metadatos del benchmark de evaluación. 135

1 Introducción

1.1 Motivación e importancia del tema de estudio

El acceso a la normativa legal es esencial para el funcionamiento del Estado de derecho, la transparencia institucional y la participación ciudadana. En Argentina, la legislación nacional se encuentra dispersa en diversas fuentes, como Infoleg (Sistema de Información Legislativa y Documental)[1], el Sistema Argentino de Información Jurídica (SAIJ)[2] y boletines oficiales[3], siendo las dos primeras fuentes secundarias, que en parte se nutren de los boletines oficiales nacionales. Esta distribución de la información dificulta su consulta y comprensión, especialmente para usuarios no especializados.

Esta dificultad de consulta se ve agravada por los sistemas tradicionales de búsqueda legal, basados en coincidencias de palabras clave, que fallan en capturar la semántica del lenguaje jurídico y las relaciones contextuales entre conceptos legales, lo que conlleva a resultados irrelevantes o incompletos. Además, la estructura compleja y jerárquica de los documentos legales requiere herramientas que comprendan y puedan navegar estas relaciones.

La Generación Aumentada por Recuperación (RAG) ha emergido como una solución prometedora para mejorar la consulta de información en dominios especializados, información confidencial, documentación interna, etc. Esta estrategia consiste en crear representaciones del texto de interés en forma de vectores (secuencias de números), almacenarlos en una base de datos para luego emplear algoritmos matemáticos para calcular similitudes entre cadenas de texto y enviar los resultados de la búsqueda al usuario (Gao et al., 2023).

Al combinar grandes modelos de lenguaje (LLM) con mecanismos de recuperación de información. La incorporación de agentes inteligentes en este proceso, conocida como Agentic-RAG, añade capacidades de planificación, razonamiento y adaptación (Singh et al., 2025).

Implementar un sistema Agentic-RAG para la consulta de normativa argentina tiene el potencial de transformar la manera en que se accede y utiliza la legislación. Facilitaría la búsqueda de información legal, reduciría las barreras de acceso para usuarios no especializados y mejoraría la eficiencia de profesionales del derecho y funcionarios públicos. Además, contribuiría a la transparencia institucional y al fortalecimiento del Estado de derecho, al hacer la normativa más accesible y comprensible para todos los ciudadanos. Como prueba de concepto, el presente trabajo se enfoca en el Código de Minería Argentino, un cuerpo normativo estructurado que permite validar la arquitectura propuesta antes de escalar a la totalidad de la legislación nacional.

[1] https://www.infoleg.gob.ar/: mantenido por el Ministerio de Justicia de la Nación.

[2] https://www.saij.gob.ar/: mantenido por la Dirección Nacional del Sistema Argentino de Información Jurídica (SAIJ) dependiente del Ministerio de Justicia de la Nación.

[3] https://www.boletinoficial.gob.ar: mantenido por la Secretaría Legal y Técnica de la Presidencia de la Nación, a través de la Dirección Nacional del Registro Oficial.

Este proyecto se alinea con las tendencias actuales en inteligencia artificial y procesamiento de (PLN), aplicándolas a un dominio crítico como el legal.

1.2 Requerimientos y desafíos

El desarrollo de un sistema Agentic-RAG para la consulta de normativa argentina implica abordar diversos requerimientos técnicos y desafíos específicos del dominio legal. El primer requisito es contar con una fuente de datos; en este caso particular, se empleará el Código de Minería Argentino extraído del archivo pdf descargado de la página del Boletín Oficial de la República Argentina y actualizado con los cambios publicados en el sitio Infoleg. El siguiente requisito es un sistema para extraer el texto de los artículos del código y almacenarlo en archivos de texto plano o en una base de datos para luego ser transformado en «embeddings» por un modelo de vectorización semántica y almacenarlo en una base de datos vectorial. Otros requisitos incluyen la integración de modelos de lenguaje preentrenados capaces de generar respuestas coherentes y contextualizadas a partir de la información recuperada, y el desarrollo de agentes inteligentes que coordinen las tareas de recuperación y generación, adaptándose a las necesidades y contextos específicos de las consultas.

Entre los desafíos del proyecto está en primer lugar, la complejidad del lenguaje jurídico, lo que presenta un obstáculo significativo, ya que el lenguaje legal es altamente técnico y especializado lo que necesita métodos más avanzados y complejos de evaluación de la calidad de recuperación. Una vez extraído el texto, se presenta la necesidad de seleccionar una estructura adecuada de representación de los datos, ya que depende de ello la calidad de los «embeddings» a ser almacenados en forma de vectores (Tsitsulin et al., 2023). Asimismo, la calidad de los «embeddings» también depende la correcta selección del tipo de «embedding» a emplear, por ejemplo, de documentos o tópicos (Isbarov & Huseynova, 2024) y del tamaño de fragmentación de texto, también conocido como «chunking» (Zhong et al., 2024).

Además, la estructura jerárquica de los documentos legales, que suelen tener múltiples niveles de jerarquía como títulos, capítulos, artículos e incisos, complica su representación y recuperación efectiva. Por otro lado, la legislación está en constante cambio, lo que exige que el sistema sea capaz de actualizarse regularmente para reflejar las modificaciones y nuevas normativas. Asimismo, es fundamental emplear métricas como la precisión, relevancia y coherencia y desarrollar un sistema punto de referencia (benchmark) para evaluar las respuestas generadas por el sistema. Finalmente, el sistema debe ser accesible y fácil de usar para usuarios con distintos niveles de conocimiento legal, incluyendo interfaces intuitivas y explicaciones claras de las respuestas proporcionadas. Abordar estos requerimientos y desafíos es esencial para el éxito del proyecto, garantizando que el sistema desarrollado sea efectivo, confiable y útil para una amplia gama de usuarios interesados en la normativa legal argentina.

1.3 Problemas y soluciones

1.3.1 Problemas no resueltos

A pesar de los avances en tecnologías de recuperación de información y procesamiento de lenguaje natural, persisten desafíos significativos en la consulta de normativa legal. En primer lugar, los sistemas tradicionales de búsqueda legal en Argentina, como Infoleg y SAIJ, se basan principalmente en coincidencias de palabras clave limitando su capacidad para comprender la semántica y contexto de las consultas, llevando a resultados que pueden ser irrelevantes o incompletos. Al día de la fecha, se ha encontrado un sistema de búsqueda asistido por IA, el sistema de «Búsqueda de Jurisprudencia Asistida por IA» del Tribunal Fiscal de la Nación[4], limitado a documentos emitidos por dicho tribunal. Fuera de esa excepción, no se han identificado bases de datos o sistemas de búsqueda más avanzados de uso libre en Argentina, aunque sí se han encontrado plataformas pagas como Lexius[5] o Juztina[6] que permiten realizar búsquedas y análisis avanzados con asistencia de inteligencia artificial empleando sus bases de datos privadas.

Uno de los principales problemas en los sistemas basados en grandes modelos de lenguaje o Large Language Models (LLM) son las alucinaciones. Este término se puede definir como «generación de información que no está basada en datos reales o verificables», es decir, el sistema «inventa» información. Esto es una ocurrencia normal en los grandes modelos de lenguaje (Islam et al., 2025) que puede atenuarse en gran medida empleando RAG (Shuster et al., 2021) y potencialmente mediante el diseño de un sistema de agentes que empleen un método estandarizado de comunicación (Gosmar & Dahl, 2025).

En esta línea, surge el concepto de generación aumentada por recuperación agéntica o Agentic-RAG, el cual combina la versatilidad de los Agentes y la RAG para crear un sistema capaz de tener memoria a corto y largo plazo, capacidad de planificar, así como también seleccionar herramientas a emplear dependiendo del rol y tarea a realizar (Singh et al., 2025).

Otro desafío es la evaluación del desempeño de los modelos de lenguaje natural en un área específica. Para realizar esta tarea se suelen emplear «benchmarks» o puntos de referencia, que contienen preguntas y respuestas esperadas para varios tópicos legales, principalmente en idioma inglés. Dependiendo de la cantidad de preguntas que el sistema responde de manera correcta, se le asigna un puntaje lo que permite comparar varios sistemas entre sí. Algunos de los benchmarks empleados para evaluar el desempeño de los sistemas en el ámbito legal son: LexGLUE (Chalkidis et al., 2021), LegalBench (Guha et al., 2023), LEXTREME (Niklaus et al., 2024) y LegalBench-RAG que consiste en la adaptación de LegalBench a sistemas de Generación Aumentada por Recuperación (Pipitone & Alami, 2024). Actualmente, la creación de benchmarks es una tarea que requiere de la intervención de humanos competentes en el área a ser evaluada, lo que conlleva a una gran inversión de tiempo y dinero, aunque este paradigma está

[4] https://jurisprudenciatfn.mecon.gob.ar/
[5] https://lexius.io/
[6] https://www.juztina.ai/ar/es

cambiando hacia una mayor automatización (Cao et al., 2025). Al día de la fecha no se han encontrado benchmarks para normativa argentina.

Por último, la mayor barrera al día de la fecha para el desarrollo de este tipo de sistemas y evaluar de manera fiable su desempeño, es el costo económico del uso de los modelos LLM ya que la estimación no es trivial y puede rápidamente escalar a grandes sumas. Asimismo, existe un número elevado de modelos y proveedores por lo que la selección puede llegar a requerir una aproximación sistemática (Shekhar et al., 2024).

1.3.2 Solución propuesta

La solución propuesta consiste en el desarrollo de un sistema de Generación Aumentada por Recuperación con agentes (Agentic-RAG) que será estructurado en tres módulos principales:

1. **Extracción y estandarización de datos normativos**: Este módulo se encarga de la recopilación de los artículos del Código de Minería Argentino desde Infoleg, aplicando técnicas de web scraping y procesamiento de lenguaje natural para extraer y estructurar la información legal. La normalización de los datos incluye la estandarización de formatos, corrección de inconsistencias y eliminación de duplicados, asegurando la calidad y coherencia de la información recopilada.

2. **Base de datos vectorial**: Los artículos del Código de Minería normalizados se transforman en vectores semánticos utilizando modelos de generación de embeddings. Estos vectores se almacenan en una base de datos vectorial que permite realizar búsquedas basadas en la similitud semántica, facilitando la recuperación de información relevante incluso cuando las consultas no coincidan exactamente con los términos utilizados en los artículos (Jing et al., 2024).

3. **Sistema Agentic-RAG**: Este componente integra modelos de lenguaje preentrenados con capacidades de recuperación de información, coordinados por agentes inteligentes (LLMs o Small Language Models, SLM) que gestionan el flujo de trabajo. Los agentes son responsables de tareas como la interpretación de consultas, planificación de acciones, recuperación de artículos relevantes y generación de respuestas (Ehtesham et al., 2025).

El sistema tiene dos funcionalidades principales. La primera consiste en la **extracción y estandarización de datos** donde se recopilan los artículos del Código de Minería y se colocan en la base de datos vectorial.

La segunda funcionalidad será atender las consultas de los usuarios, lo que desencadena un conjunto de subtareas:

1. **Recepción de la consulta:** El usuario introduce su consulta en lenguaje natural.

2. **Interpretación de la consulta:** Un agente especializado analiza la consulta para identificar la intención del usuario y los elementos clave de la búsqueda.

3. **Planificación de la recuperación:** Basándose en la interpretación de la consulta, el agente planifica las acciones necesarias para recuperar la información relevante, determinando qué fuentes consultar y qué estrategias de búsqueda emplear.

4. **Recuperación de documentos:** El sistema realiza búsquedas en la base de datos vectorial para identificar los documentos legales más relevantes según la similitud semántica con la consulta.

5. **Generación de la respuesta:** Utilizando los documentos recuperados de la base de datos vectorial, un modelo de lenguaje preentrenado genera una respuesta coherente y contextualizada que responde a la consulta del usuario.

6. **Presentación de la respuesta:** La respuesta final se presenta al usuario junto con enlaces a los documentos legales relevantes y opciones para profundizar en la información.

Para evaluar la efectividad del sistema propuesto, se llevarán a cabo pruebas utilizando un conjunto de consultas representativas del dominio legal argentino. Se medirán métricas como la precisión, exhaustividad, relevancia y coherencia de las respuestas generadas, comparando el rendimiento del sistema Agentic-RAG con soluciones existentes.

Además, se realizarán pruebas utilizando un **benchmark a pequeña escala, generado con asistencia de LLM y validado por profesionales del derecho argentino**. Este benchmark, al ser revisado por expertos en el campo, garantizará que las evaluaciones se basen en casos de uso reales y desafiantes, proporcionando una medida fidedigna de la capacidad del sistema para asistir en la práctica jurídica diaria.

1.3.3 Limitaciones de la solución propuesta

La solución propuesta presenta algunas limitaciones que deben considerarse:

- El sistema se limita al Código de Minería Argentino como prueba de concepto, sin incluir otras normativas nacionales, provinciales o municipales.
- La calidad de las respuestas depende de la calidad de la extracción de texto y de los modelos de embeddings seleccionados.
- El costo computacional y económico del uso de modelos LLM puede limitar la escalabilidad del sistema.
- El benchmark desarrollado es de pequeña escala, enfocado específicamente en el Código de Minería.
- El sistema se implementa como un pipeline experimental en notebooks de Jupyter, sin una interfaz de usuario final desplegada. La interacción con el sistema requiere conocimientos técnicos para ejecutar los scripts y notebooks correspondientes.

Como trabajo futuro, se podría ampliar el sistema para incluir la totalidad de la normativa nacional disponible en Infoleg, así como normativa provincial y municipal, expandir el benchmark con la colaboración de más profesionales del derecho, y desarrollar una interfaz de usuario accesible para usuarios no técnicos.

1.4 Resumen de problemas y soluciones

A continuación, se detallan los principales problemas identificados, con un breve resumen de cada uno de ellos y las soluciones propuestas:

Problema	Solución
Sistemas de búsqueda legal basados en palabras clave con limitada comprensión semántica	Implementación de búsqueda semántica mediante base de datos vectorial con embeddings
Alucinaciones en modelos de lenguaje que generan información no verificable	Uso de RAG para anclar respuestas en documentos reales y verificables
Ausencia de benchmarks para normativa argentina	Desarrollo de benchmark específico generado con asistencia de LLM y validado por profesionales del derecho argentino

Tabla 1: Resumen de problemas identificados y soluciones propuestas

1.5 Objetivo del trabajo

1.5.1 Objetivo general

Desarrollar un sistema Agentic-RAG de código abierto como prueba de concepto, utilizando el Código de Minería Argentino, para responder consultas de usuarios profesionales y no profesionales en el derecho.

1.5.2 Objetivos específicos

1. Desarrollar una base de datos vectorial del Código de Minería Argentino diseñada específicamente para trabajar con sistemas RAG.
2. Desarrollar un sistema de agentes de inteligencia artificial para que trabajen de manera conjunta con el sistema RAG.
3. Desarrollar un benchmark específico para el Código de Minería Argentino para evaluar el desempeño del sistema.
4. Cargar el código y una descripción de la metodología a un repositorio abierto para que cualquier organización, empresa o individuo pueda replicar y adaptar el sistema a otras normativas.

1.6 Transferencia de los resultados obtenidos

El código y la documentación generada durante el proyecto se cargarán a un repositorio del sistema de control de versiones «Git»[7] y luego se subirá la información a la plataforma «GitHub»[8] en un repositorio público para que cualquier individuo pueda descargarlo.

[7] https://git-scm.com/
[8] https://github.com/

La solución propuesta tiene aplicaciones prácticas que trascienden el ámbito específico de la normativa argentina:

- **Adaptación a otras jurisdicciones**: La metodología y arquitectura del sistema pueden ser replicadas para crear sistemas similares en otros países de habla hispana o con sistemas legales comparables.
- **Extensión a otros dominios**: Los principios de Agentic-RAG pueden aplicarse a otros dominios especializados como medicina, ingeniería o finanzas, donde se requiere consulta de documentación técnica extensa.
- **Contribución al software libre**: Al ser de código abierto, el proyecto contribuye al ecosistema de herramientas legales accesibles, permitiendo que organizaciones sin recursos para adquirir soluciones comerciales puedan implementar sistemas de consulta avanzados.
- **Modelo para instituciones públicas**: El sistema puede servir como referencia para que organismos gubernamentales implementen herramientas similares para mejorar el acceso ciudadano a la información legal.

1.7 Contribuciones principales

Las principales contribuciones del presente trabajo final son:

1. **Sistema Agentic-RAG de código abierto**: Desarrollo de un sistema completo que integra recuperación semántica, base de datos vectorial y agentes inteligentes para la consulta del Código de Minería Argentino como prueba de concepto.
2. **Base de datos vectorial del Código de Minería**: Creación de una base de datos especializada con embeddings semánticos de los artículos del Código de Minería Argentino, optimizada para búsquedas por similitud.
3. **Benchmark para el Código de Minería Argentino**: Desarrollo de un benchmark específico para evaluar sistemas de consulta legal sobre el Código de Minería, generado con asistencia de LLM y validado por profesionales del derecho.
4. **Metodología replicable**: Documentación detallada de la metodología empleada, permitiendo la replicación del sistema para otras normativas y jurisdicciones.
5. **Contribución a la transparencia institucional**: Demostración del potencial para mejorar el acceso ciudadano a la información legal, fortaleciendo el Estado de derecho y la participación ciudadana.

1.8 Trabajos presentados vinculados con el trabajo final

Al momento de la escritura de este documento, no se han presentado trabajos de investigación en congresos o revistas científicas vinculados directamente con el presente trabajo final.

1.9 Estructura general del trabajo final

El presente trabajo final se estructura de la siguiente manera:

- **Capítulo 2 - Fundamentos e Historia del Procesamiento de Lenguaje Natural**: Se presentan los conceptos fundamentales sobre la evolución de los modelos de lenguaje, desde modelos estadísticos hasta los grandes modelos de lenguaje (LLM), incluyendo embeddings semánticos, tokenización, mecanismos de atención y las limitaciones como las alucinaciones.
- **Capítulo 3 - Aplicación de PLN al Dominio Legal**: Se analizan las particularidades del lenguaje jurídico, las tareas típicas del procesamiento de lenguaje natural legal, las técnicas de adaptación de LLMs al dominio legal y los riesgos asociados (alucinaciones, sesgos), fundamentando la complementariedad entre LLMs y RAG.
- **Capítulo 4 - Agentic-RAG: Generación Aumentada por Recuperación Agéntica**: Se presenta el marco teórico unificado de RAG y sistemas agénticos, describiendo arquitecturas, patrones de flujo de trabajo, agentes especializados, la técnica de evaluación LLM-as-a-Judge y los desafíos y limitaciones asociados.
- **Capítulo 5 - Metodología e Implementación**: Se describe la metodología empleada y la implementación técnica del sistema, incluyendo la selección del corpus legal, extracción y preprocesamiento de datos, diseño de la base de datos vectorial, arquitectura del sistema Agentic-RAG, diseño del benchmark y el sistema de evaluación.
- **Capítulo 6 - Evaluación y Resultados**: Se presenta el diseño experimental factorial 3×3, las métricas utilizadas (textuales, semánticas y LLM-as-a-Judge), los resultados obtenidos para las nueve configuraciones, el análisis estadístico (Kruskal-Wallis, Dunn, Scheirer-Ray-Hare, Krippendorff's alpha) y el análisis de costos.
- **Capítulo 7 - Trabajos Relacionados**: Se presenta una revisión de los trabajos previos y plataformas existentes, organizada en cuatro ejes: sistemas de búsqueda legal, RAG en dominios especializados, sistemas agénticos y benchmarks para evaluación legal, posicionando el presente trabajo en el contexto de la literatura.
- **Capítulo 8 - Conclusiones y Trabajos Futuros**: Se sintetizan las conclusiones del trabajo, se revisa el cumplimiento de los objetivos, se discuten las fortalezas y limitaciones identificadas, y se proponen líneas de investigación futuras.

Adicionalmente, el trabajo incluye anexos con la estructura del repositorio de código, ejemplos de consultas y respuestas, y el benchmark completo utilizado para la evaluación, así como un listado de acrónimos y siglas empleados en el documento.

2 Fundamentos e Historia del procesamiento del lenguaje natural (PLN)

Nota: El campo de los grandes modelos de lenguaje evoluciona a un ritmo sin precedentes. Este capítulo presenta los fundamentos teóricos vigentes al momento del diseño del sistema (segundo semestre de 2025). Los desarrollos posteriores al período experimental se discuten en el capítulo de trabajos relacionados.

2.1 Introducción al Procesamiento de Lenguaje Natural

El Procesamiento de Lenguaje Natural (PLN o NLP, por sus siglas en inglés) es un área de la inteligencia artificial centrada en desarrollar modelos computacionales que permitan a las máquinas comprender, generar y razonar sobre lenguaje humano. En el corazón de muchas aplicaciones de PLN se encuentran los modelos de lenguaje, que asignan probabilidades a secuencias de palabras y permiten estimar la probabilidad de la siguiente palabra dado un contexto (Jurafsky & Martin, 2026).

La historia del desarrollo de los modelos del lenguaje se remota hace más de 100 años a los inicios del siglo XX, comenzando desde modelos estadísticos relativamente simples, basados en distribuciones de frecuencias de palabras, hasta redes neuronales, basados en la arquitectura del Transformer. En este capítulo se realiza una breve reseña de las diferentes estrategias para el desarrollo de los modelos del lenguaje.

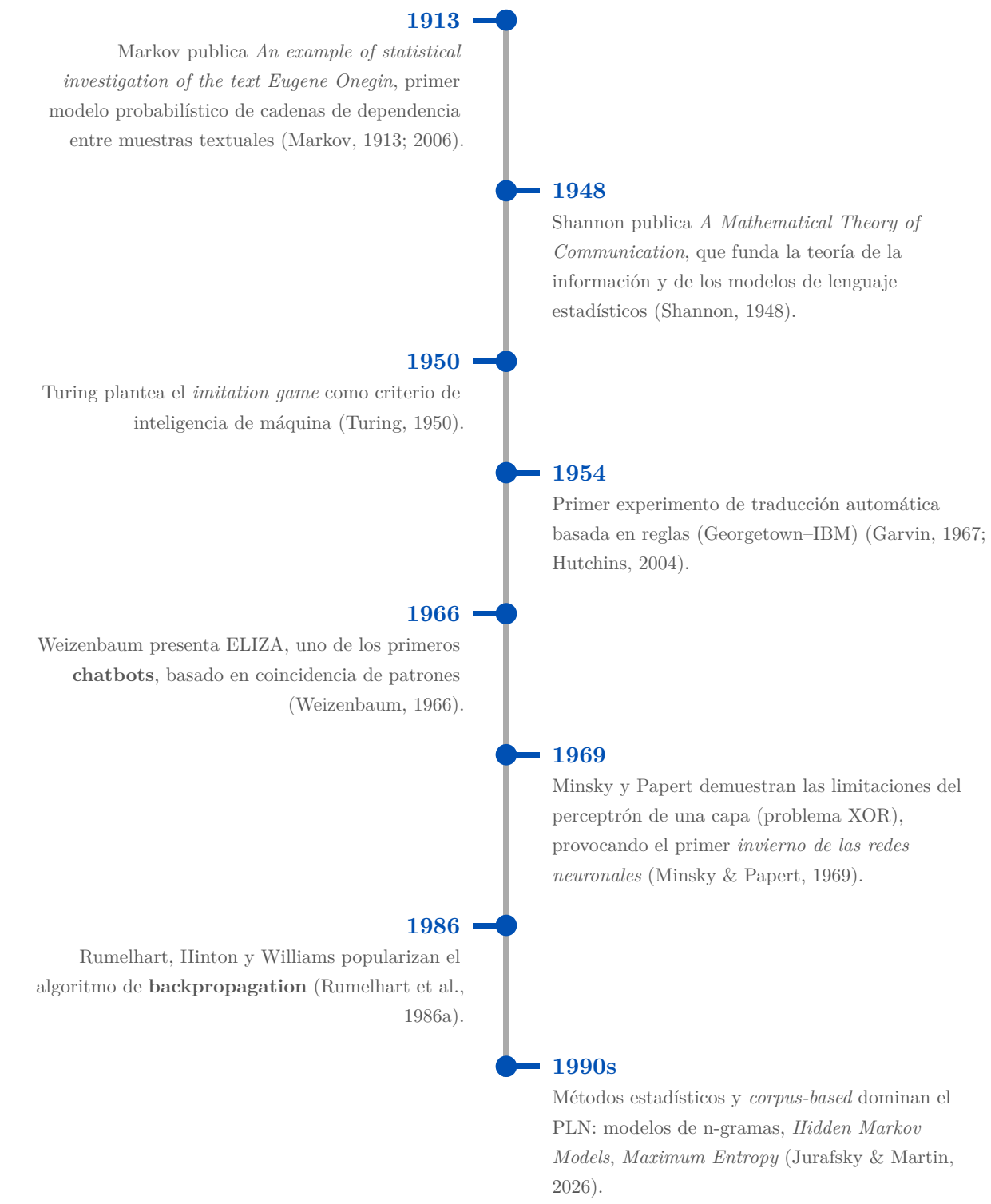


Figura 1: Cronología de los hitos principales en la evolución de los modelos del lenguaje (1913–1990s). Primera parte.

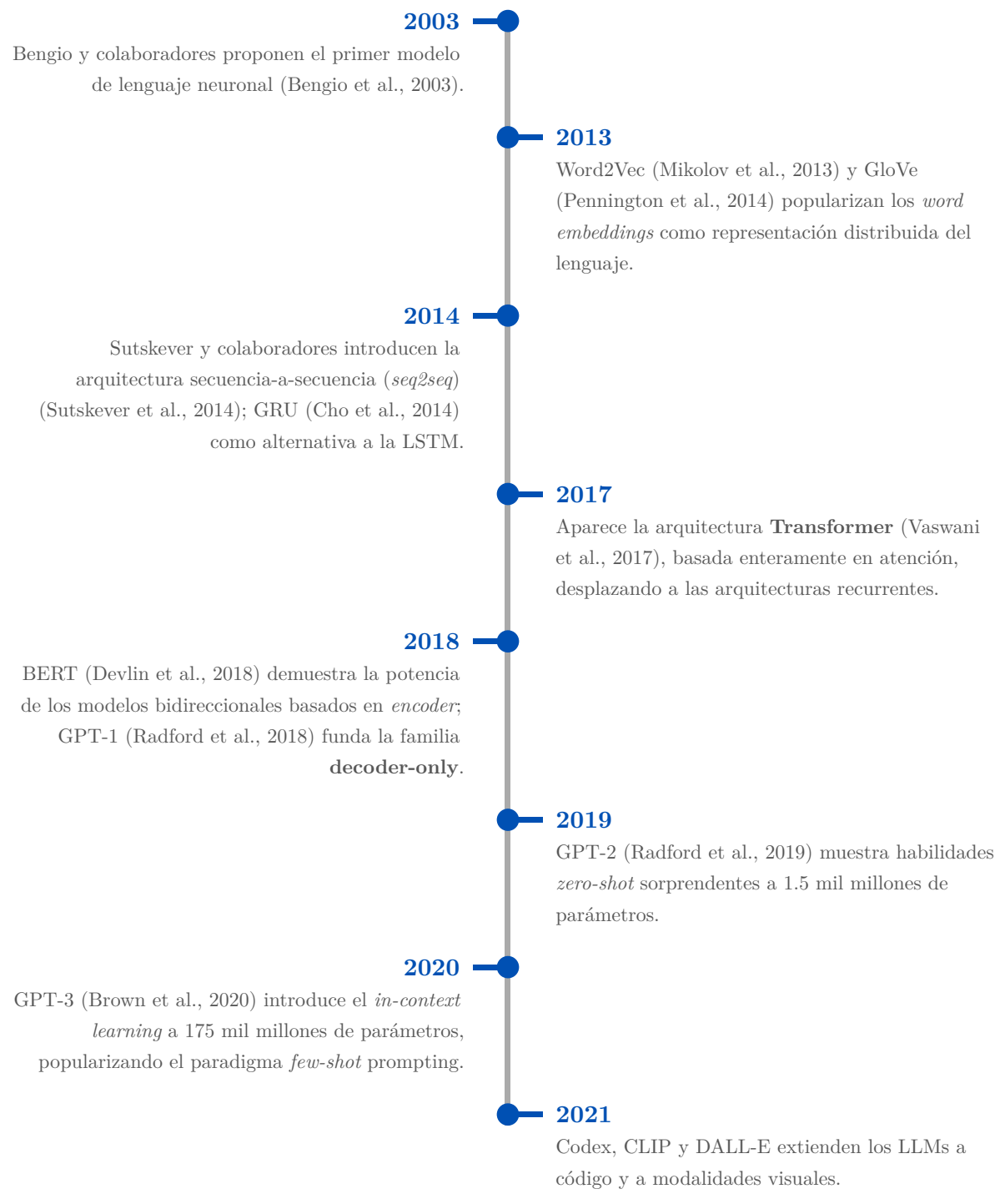


Figura 2: Cronología de los hitos principales en la evolución de los modelos del lenguaje (2003–2021). Segunda parte.

2.2 Evolución de los modelos del lenguaje

De acuerdo a la taxonomía mencionada por Raeini (2025), los modelos del lenguaje pueden clasificarse de la siguiente manera:

1. Estadísticos
2. Basados en reglas
3. Basados en recuperación de información

4. Basados en espacio de vectores
5. Redes Neuronales
6. Redes Neuronales basadas en la arquitectura Transformer
7. Estado de espacios

2.2.1 Modelos estadísticos

La primera aproximación para tratar de predecir secuencias de texto fueron los trabajos de Markov (1913); Markov (2006) y Shannon (1948) que derivaron en la creación de modelos estadísticos permitiendo caracterizar la probabilidad de secuencias de letras (palabras y frases). Esta estrategia se basa en el uso de n-gramas, los cuales consisten en secuencias de caracteres de longitud n. Por ejemplo, para la frase «Pablito clavó un clavito qué clavito clavó Pablito» se pueden obtener los siguientes n-gramas:

Unigramas — n = 1:

N.	Palabra
1	Pablito
2	clavó
3	un
4	clavito
5	qué
6	clavito
7	clavó
8	Pablito

Bigramas — n = 2:

N.	Palabra 1	Palabra 2
1	Pablito	clavó
2	clavó	un
3	un	clavito
4	clavito	qué
5	qué	clavito
6	clavito	clavó
7	clavó	Pablito

Trigramas — n = 3:

N.	Palabra 1	Palabra 2	Palabra 3
1	Pablito	clavó	un
2	clavó	un	clavito
3	un	clavito	qué
4	clavito	qué	clavito

N.	Palabra 1	Palabra 2	Palabra 3
5	qué	clavito	clavó
6	clavito	clavó	Pablito

A partir de los cuales es posible construir un modelo estadístico siguiendo la ecuación:

$$P(w \mid h) = \frac{\text{count}(h, w)}{\text{count}(h)}$$

Donde $\text{count}(h, w)$ es el número de veces que la secuencia h seguida de w aparece en el texto, y $\text{count}(h)$ es el número total de veces que aparece el contexto h. Para el caso de la palabra «clavó» empleando bigramas se obtiene:

$$P(\text{clavó} \mid \text{Pablito}) = \frac{\text{count}(\text{Pablito}, \text{clavó})}{\text{count}(\text{Pablito})}$$

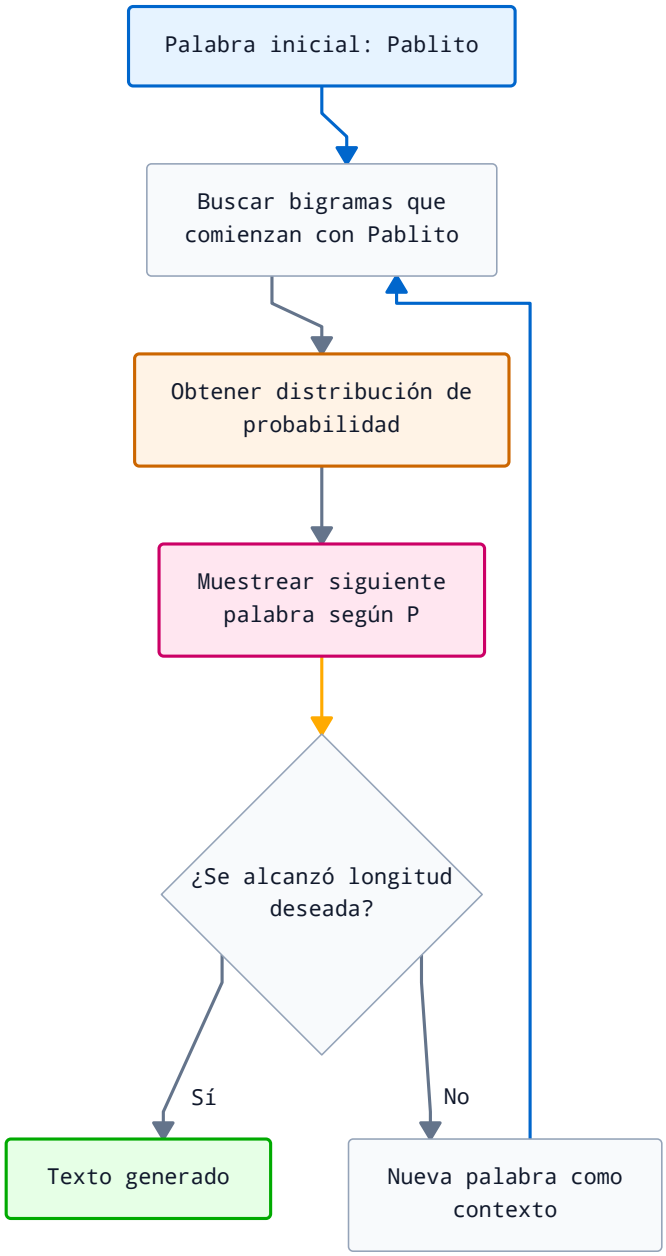


Figura 3: Diagrama de flujo de generación de texto basado en n-gramas, siguiendo el método de Shannon (1948).

A pesar de su utilidad, los modelos de n-gramas presentan limitaciones sustanciales que restringen severamente su aplicabilidad a vocabularios y corpus de gran escala (Chen & Goodman, 1999; Jurafsky & Martin, 2026):

1. **Explosión combinatoria y dispersión.** El número de n-gramas únicos crece de manera aproximadamente exponencial con el tamaño del vocabulario y con el orden n. Esto vuelve a los modelos de orden superior ($n \geq 4$) rápidamente intratables, tanto en memoria como en tiempo de cómputo, aún para corpus de tamaño modesto.
2. **Gestión de palabras fuera de vocabulario (OOV).** Toda palabra que no haya sido observada en el corpus de entrenamiento recibe probabilidad cero. Análogamente, los n-gramas que contienen palabras raras o no vistas obtienen estimaciones de probabilidad poco fiables, fenómeno conocido como *data sparsity*. En la práctica se

aplican técnicas de suavizado (*smoothing*) como Laplace, Good–Turing, Kneser–Ney o interpolated Kneser–Ney para mitigar este problema (Chen & Goodman, 1999).

3. **Falta de generalización semántica.** Los modelos de n-gramas tratan a cada palabra como un índice discreto y no capturan ningún tipo de relación semántica, sintáctica o morfológica entre ellas. Palabras semánticamente próximas como *perro* y *can* son igualmente «distantes» en el espacio del modelo, lo que impide transferir conocimiento entre términos relacionados.
4. **Dependencia rígida del orden de las palabras.** La probabilidad se calcula sobre secuencias literales de tokens; variantes como *El perro ladra* y *Ladra el perro* producen distribuciones completamente diferentes. Esta rigidez torna al modelo muy sensible a la longitud del contexto y al estilo de los textos observados.
5. **Ventana de contexto limitada.** La historia que condiciona la predicción está restringida a los $n-1$ tokens previos. Para modelar dependencias de mayor alcance (por ejemplo, la concordancia entre sujeto y verbo en oraciones largas, o las correferencias entre párrafos distantes) hacen falta arquitecturas que operen sobre representaciones más ricas, como las que veremos a partir de las redes neuronales recurrentes y, más adelante, los mecanismos de atención y la arquitectura Transformer.

Estas limitaciones motivaron la búsqueda de representaciones distribuidas (como los *word embeddings*) y, posteriormente, de arquitecturas neuronales capaces de capturar dependencias de largo plazo.

2.2.2 Sistemas basados en reglas y en recuperación de información

Ya por los años 60, el desarrollo inicial de los asistentes virtuales como los **chatbots** se centró en sistemas estructurados que operaban bajo directrices preprogramadas, es decir, sistemas de reglas escritas a mano por humanos. Weizenbaum (1966) presentó a ELIZA, un **chatbot** que emulaba a un psicoterapeuta rogeriano[9] mediante técnicas de coincidencia de patrones (Mnasri, 2019; Weizenbaum, 1966). El programa analiza la consulta del usuario, identifica las palabras clave con mayor jerarquía dentro de su base de conocimientos y ejecuta reglas de transformación contenidas en scripts específicos. El éxito de ELIZA impulsó el desarrollo de arquitecturas similares como PARRY, ALICE, CLEVER y SHRDLU (Mnasri, 2019; Shum et al., 2018; Winograd, 1972). Su principal desventaja radica en que eran sistemas muy básicos, limitados estrictamente a las reglas manuales y bases de datos integradas en sus scripts.

[9] La terapia rogeriana, también llamada terapia centrada en el cliente, fue desarrollada por el psicólogo estadounidense Carl Rogers en la década de 1940 (Rogers, 1951). Se caracteriza por la escucha activa, la empatía incondicional y la reformulación no directiva de los planteos del paciente, condiciones que la vuelven especialmente fácil de simular mediante reglas de coincidencia de patrones.

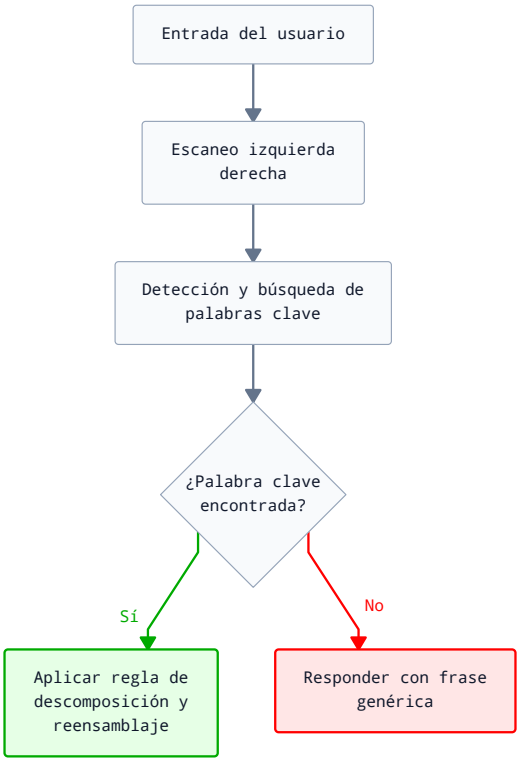


Figura 4: Flujo de procesamiento de ELIZA. Basado en el mecanismo de descomposición y reensamblaje descrito por (Weizenbaum, 1966).

Otro tipo de **chatbots** son aquellos basados en recuperación de información (Information Retrieval, IR), donde los sistemas formulan respuestas al contrastar los patrones de entrada con pares de solicitud-respuesta ya estructurados en una base de datos de diálogos (Peng & Ma, 2019). Para seleccionar la respuesta óptima, se apoyan en metodologías de recuperación de datos a nivel de palabra tales como TF-IDF, similitud del coseno y modelos de espacio de estados. CleverBot es un referente de esta categoría; fue diseñado a finales de la década de 1980 y difundido formalmente a finales de los años 1990 (Mnasri, 2019).

2.2.3 Modelos basados en espacio de vectores y espacio de estados

Los espacios vectoriales constituyen otro pilar fundamental en el desarrollo de los modelos de lenguaje.

Los modelos de espacio de vectores (VSM, por sus siglas en inglés) son enfoques algebraicos que transforman los elementos del lenguaje en vectores dentro de un espacio determinado. La incrustación de palabras (**word embedding**) es el nombre que recibe la técnica de representar palabras mediante vectores. Esta es una estrategia que surgió en la década de 1980 (Rumelhart et al., 1985) aunque su verdadero auge llegó con investigaciones clave como la de Bengio et al. (2003), seguidas por Word2Vec (Mikolov et al., 2013) y GloVe (Pennington et al., 2014) en los años 2010.

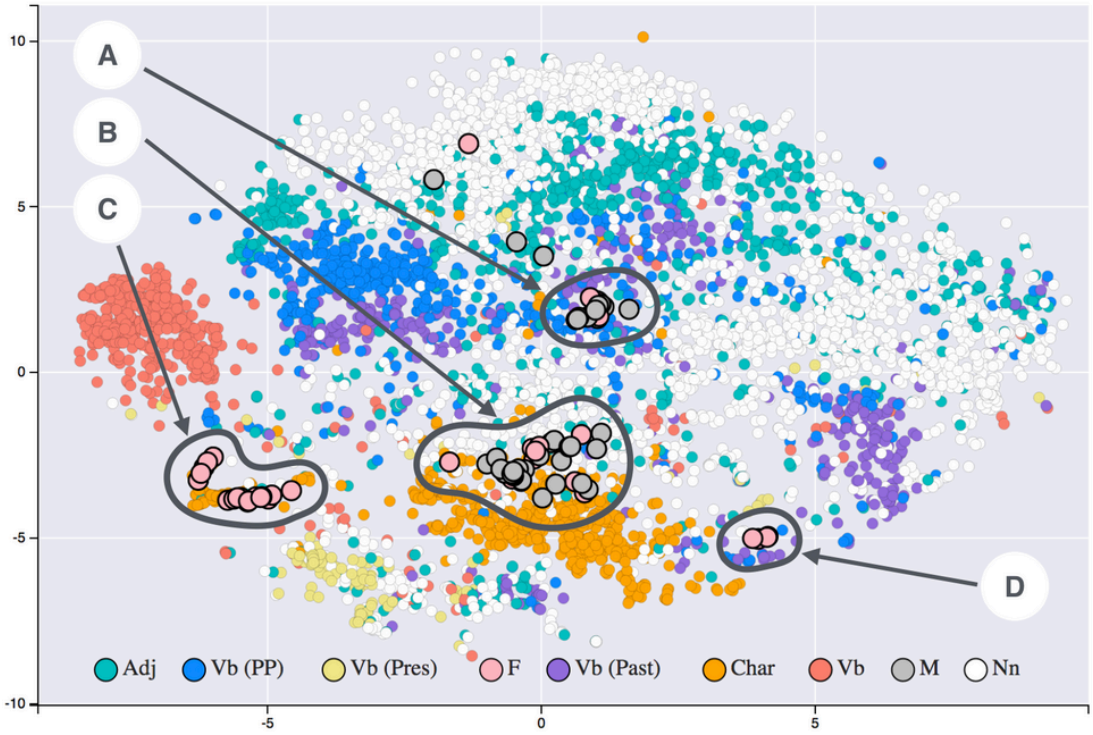


Figura 5: Visualización t-SNE de *word embeddings* generados a partir de literatura del siglo XIX. Los puntos próximos en el plano corresponden a palabras con significado o contexto de uso similar, ilustrando cómo los VSM codifican relaciones semánticas como relaciones geométricas entre vectores (Grayson, 2017).

Mientras que los VSM se han aplicado masivamente en el modelado del lenguaje, los modelos de espacio de estados (SSM, por sus siglas en inglés) han comenzado a captar el interés en el campo del procesamiento del lenguaje natural recientemente (Dao et al., 2022). Se utilizaban tradicionalmente en la teoría de control y el procesamiento de señales para modelar secuencias de datos, por lo que se destacan por su gran capacidad para gestionar y procesar secuencias de texto muy largas. Por este motivo se perfilan como candidatos ideales para diseñar nuevas arquitecturas y optimizar los modelos de lenguaje actuales. Para más información puede consultar las publicaciones de Mamba de Gu & Dao (2023).

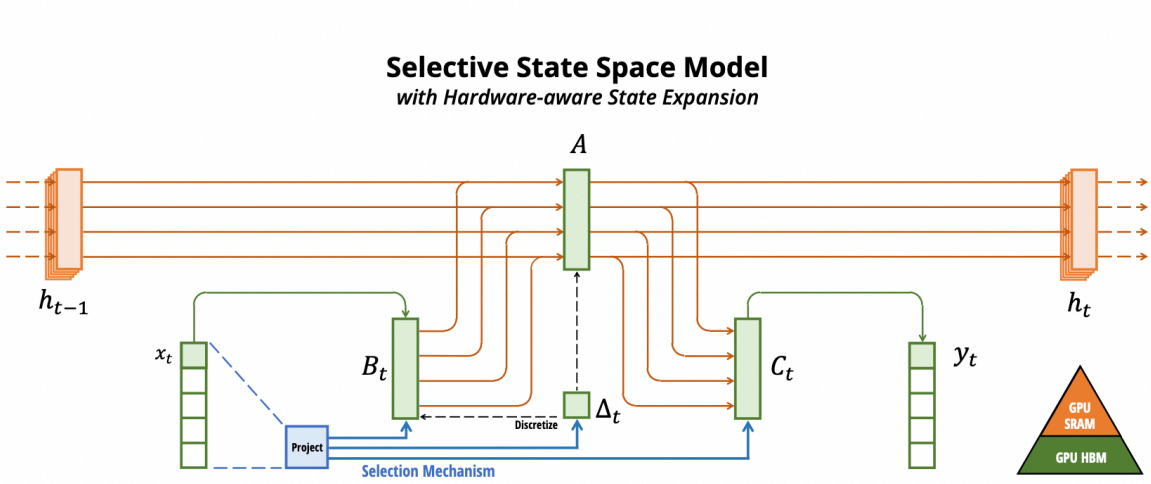


Figura 6: Diagrama de bloques de un bloque Mamba, una arquitectura de espacio de estados selectivo. Se observa la aplicación de proyecciones lineales, convoluciones y el mecanismo de selección que vuelve a los parámetros del SSM función de la entrada. La recurrencia lineal $h_t = Ah_{\{t-1\}} + Bx_t$ permite procesar secuencias largas en tiempo lineal, a diferencia del costo cuadrático de la atención. Fuente: Emsi (2024)

2.2.4 Modelos de lenguaje basados en redes neuronales artificiales

En esta sección repasamos la historia de las redes neuronales artificiales (RNA) y las familias de RNA que han servido como base de modelos de lenguaje más avanzados, en particular las redes neuronales recurrentes (RNN), las de memoria de corto-largo plazo (LSTM), las unidades recurrentes con compuertas (GRU), el mecanismo de atención y la arquitectura Transformer.

2.2.4.1 El perceptrón y las primeras redes neuronales artificiales

Las redes neuronales artificiales (RNA) son estructuras conformadas por elementos de procesamiento simples, densamente interconectados y adaptativos (denominados neuronas artificiales o nodos), capaces de realizar cálculos masivamente paralelos sobre los datos, y que son utilizadas como herramientas de modelado computacional en un amplio abanico de disciplinas para resolver problemas reales complejos (Basheer & Hajmeer, 2000).

El perceptrón de Rosenblatt. El modelo fundacional de neurona artificial fue propuesto por Rosenblatt (1958), en el marco de una teoría conexionista del cerebro y de los sistemas de aprendizaje. Rosenblatt partió de los trabajos de Hebb sobre la plasticidad sináptica[10] y de los modelos previos de McCulloch y Pitts (McCulloch & Pitts, 1943), y propuso un sistema de unidades de procesamiento con pesos sinápticos ajustables, organizados en una capa de *proyección* (sensorial) conectada a una capa de *asociación* donde se efectúa la suma ponderada de las entradas, cuya salida alimenta una capa final de *respuesta* (Rosenblatt, 1958). La regla de aprendizaje del perceptrón modifica los pesos en proporción al error entre la salida obtenida y la deseada, siempre que los datos sean linealmente separables. Esta restricción resultó crucial: Minsky y

[10]La regla de Hebb (Hebb, 1949) postula que la conexión entre dos neuronas se fortalece cuando ambas se activan simultáneamente, ofreciendo la primera base biológica plausible para el aprendizaje en redes neuronales.

Papert demostraron que un perceptrón de una sola capa no podía resolver problemas no linealmente separables como la función lógica XOR (Minsky & Papert, 1969), lo que provocó lo que se conoce como el *invierno de las redes neuronales* y frenó la financiación del campo durante más de una década (Russell & Norvig, 2020).

Organización en capas y perceptrones multicapa. La limitación anterior se superó intercalando una o más *capas ocultas* entre la entrada y la salida, dando lugar a los perceptrones multicapa (MLP) (Rumelhart et al., 1986b). Cada capa oculta aplica una transformación no lineal a la representación de la capa anterior, de modo que la composición de varias no linealidades permite aproximar funciones arbitrariamente complejas (esto es formalizado más tarde por el *teorema de aproximación universal* de Cybenko (1989) y Hornik et al. (1989)). Sin embargo, durante muchos años no se contó con un algoritmo eficiente para ajustar los pesos de todas las capas de manera conjunta (Rumelhart et al., 1986a).

Backpropagation. La solución llegó de la mano del algoritmo de *retropropagación* (*backpropagation*), popularizado por Rumelhart, Hinton y Williams en 1986 (Rumelhart et al., 1986a). La idea central es propagar el error cometido en la salida hacia atrás, capa por capa, calculando para cada peso la derivada parcial de la pérdida con respecto a ese peso mediante la regla de la cadena. Combinado con un método de optimización basado en gradiente descendente, backpropagation permite entrenar MLP con varias capas ocultas de manera eficiente, y constituye hasta hoy la columna vertebral del aprendizaje profundo. La popularización de backpropagation se considera el hito que dio inicio, ya entrada la década de 2010, al *deep learning* moderno (LeCun et al., 2015).

A continuación se describen los mecanismos fundamentales que rigen el funcionamiento de las RNA (Basheer & Hajmeer, 2000), integrando los conceptos introducidos por Rosenblatt y Rumelhart, Hinton y Williams (Rosenblatt, 1958; Rumelhart et al., 1986a):

1. **Neurona artificial (modelo tipo perceptrón).** Recibe n señales de entrada x_i con sus respectivos pesos sinápticos w_i. Calcula la entrada neta como el producto interno:

$$j = \sum_{i=1}^{n} w_i x_i$$

   La salida y se obtiene aplicando una función de activación (en el perceptrón clásico, un umbral lineal con sesgo b): la neurona se «dispara» (por ejemplo, $y = 1$) solo cuando $j > b$, y permanece inactiva ($y = 0$) en caso contrario.

2. **Pesos excitatorios e inhibitorios.** Pesos positivos ($w_i > 0$) potencian la entrada neta y excitan a la neurona; pesos negativos la reducen e inhiben su actividad.

3. **Arquitectura multicapa (MLP).** Para resolver problemas no linealmente separables se intercalan una o más capas ocultas entre la capa de entrada y la de salida. Las neuronas ocultas procesan la información proveniente de la entrada y la transmiten a la capa de salida.

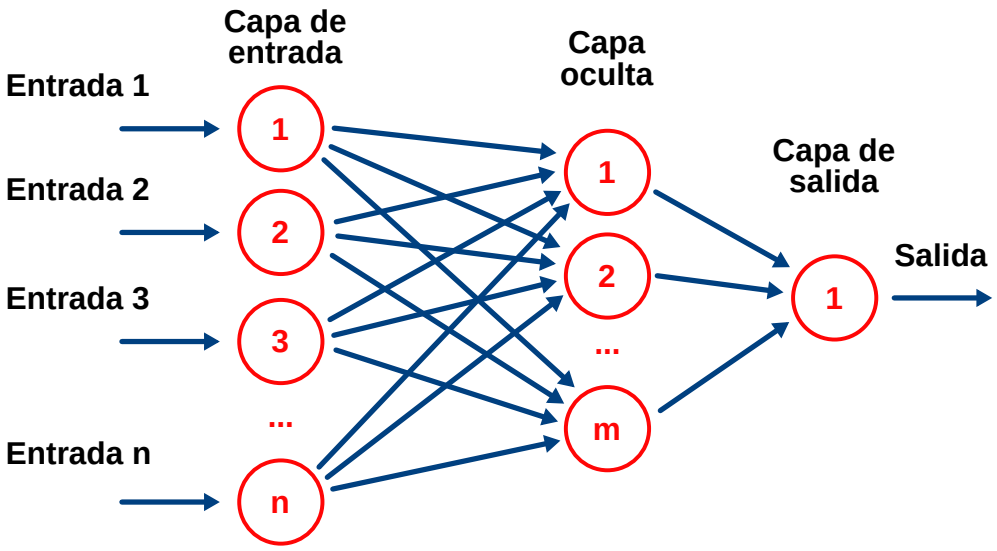


Figura 7: Ejemplo esquemático de una red neuronal artificial multicapa (MLP). Se observan la capa de entrada (a la izquierda), una capas ocultas de neuronas interconectadas y la capa de salida (a la derecha). Cada conexión entre neuronas tiene un peso sináptico asociado, y la información fluye estrictamente hacia adelante (*feed-forward*). Fuente: Antimundo (2026)

4. **Aprendizaje.** Tanto la red biológica como la artificial ajustan incrementalmente la magnitud de los pesos. En el perceptrón simple, los pesos se modifican en proporción al error entre la salida deseada Y y la obtenida y. En los MLP, este ajuste se realiza típicamente mediante el algoritmo de **retropropagación** (**backpropagation**) con la regla delta, que busca (mediante una técnica de búsqueda especializada) el conjunto de pesos que minimiza el error.
    1. **Objetivo.** Desarrollar algoritmos matemáticos que permitan a las RNA aprender imitando el procesamiento de información y la adquisición de conocimiento del cerebro humano, aunque sin pretender replicar la operación del sistema biológico, sino aprovechar lo que se conoce de su funcionalidad para resolver problemas complejos.

2.2.4.2 Redes neuronales recurrentes (RNN)

Tras los modelos estadísticos y los basados en reglas, cobraron protagonismo los modelos de lenguaje sustentados en redes neuronales artificiales. Dentro de este grupo, las redes neuronales recurrentes (RNN) han tenido un rol central. Surgieron en la década de 1980 (Elman, 1990; Hopfield, 1982; Jordan, 1986; Rumelhart et al., 1985) y se concibieron como una clase de redes orientadas al procesamiento de datos secuenciales y series temporales. Su aporte al desarrollo de modelos de lenguaje fue considerable, aunque muestran un desempeño limitado cuando las secuencias presentan dependencias de largo alcance (Bengio et al., 1994). Una de sus restricciones más conocidas es la aparición del gradiente evanescente[11] y del gradiente explosivo[12]. Estas limitaciones impulsaron la

[11] El *gradiente evanescente* (*vanishing gradient*) describe el fenómeno por el cual, durante el entrenamiento de redes profundas mediante retropropagación, los gradientes de la pérdida con respecto a los pesos de las capas iniciales se vuelven extremadamente pequeños a medida que la señal se propaga

búsqueda de arquitecturas alternativas capaces de tratar secuencias extensas con mayor eficacia.

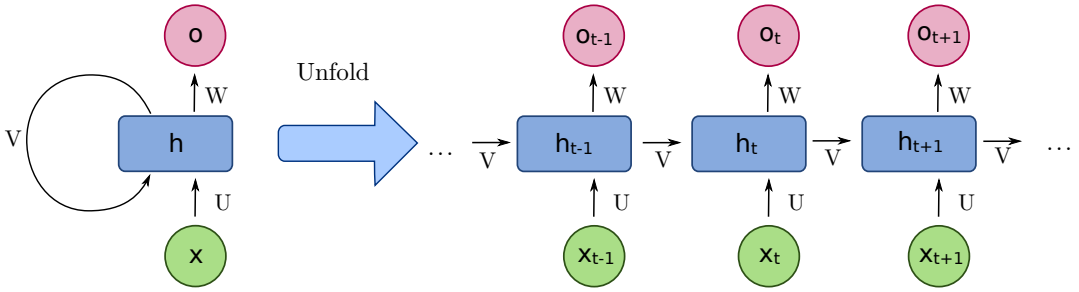


Figura 8: Red neuronal recurrente (RNN) desplegada en el tiempo. La misma red (con pesos compartidos) se aplica en cada paso de la secuencia: el estado oculto h_t depende tanto de la entrada actual x_t como del estado anterior $h_{\{t-1\}}$, lo que le da a la red memoria de los pasos previos. Fuente: Ffets (2026)

2.2.4.3 Memorias de corto y largo plazo (LSTM)

Las redes de memoria a corto y largo plazo (LSTM, por sus siglas en inglés) son un tipo particular de red neuronal recurrente que aborda de manera más satisfactoria el problema de las dependencias de largo plazo. Introducidas por Hochreiter & Schmidhuber (1997), su objetivo principal fue mitigar el problema del gradiente evanescente que afecta a las RNN. Una LSTM se compone de una celda de memoria y tres compuertas: de entrada, de salida y de olvido. La compuerta de olvido es la que determina qué información se conserva a lo largo del tiempo, regulando así la memoria de corto o largo plazo. Aunque las LSTM resuelven limitaciones de las RNN básicas, esta mejora tiene un costo: las LSTM son más complejas, requieren más parámetros, su entrenamiento resulta computacionalmente más caro, demanda más tiempo y necesita una mayor cantidad de datos de entrenamiento.

hacia atrás. Esto hace que esos pesos apenas se actualicen y la red no logre aprender dependencias de largo plazo en secuencias largas. El problema fue analizado formalmente por Hochreiter (1998).

[12]El *gradiente explosivo* (*exploding gradient*) es el fenómeno opuesto: durante la retropropagación los gradientes crecen de manera descontrolada a través de las capas, provocando actualizaciones de peso enormes que vuelven el entrenamiento inestable. Suele aparecer cuando la matriz de pesos recurrentes tiene valores propios mayores que uno en valor absoluto. Pascanu et al. (2013) propusieron una solución práctica, el *gradient clipping*, que limita la norma del gradiente a un umbral máximo cuando se supera.

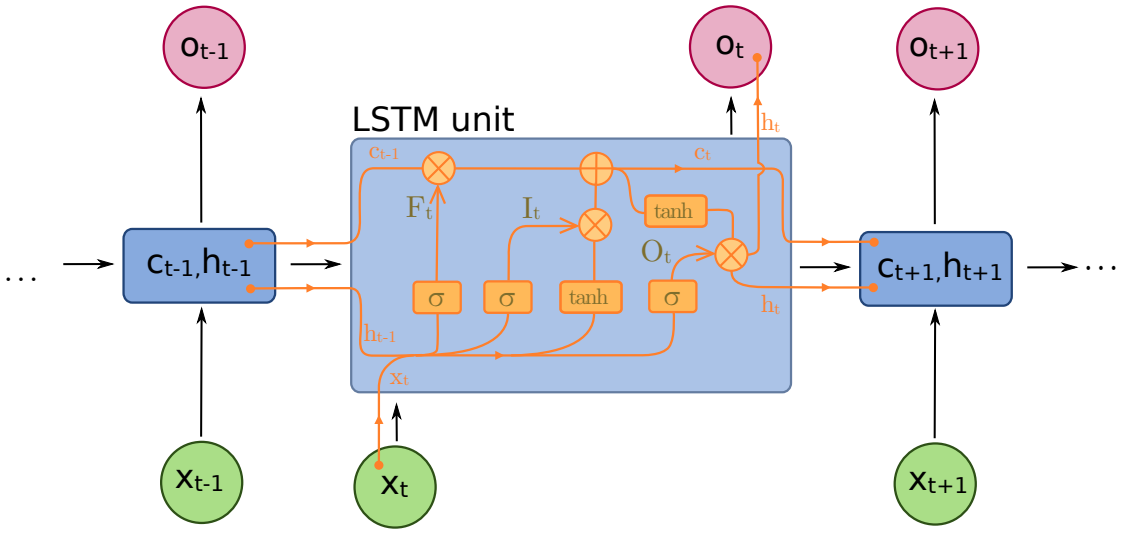


Figura 9: Estructura interna de una celda LSTM. Se observan las tres compuertas: de olvido (f_t), de entrada (i_t) y de salida (o_t), que controlan respectivamente qué información se descarta, qué nueva información se incorpora y qué parte del estado de la celda C_t se propaga como salida. Fuente: fdeloche (2017a)

2.2.4.4 Unidades recurrentes con compuertas (GRU)

Las unidades recurrentes con compuertas (GRU, por sus siglas en inglés) fueron presentadas por Cho et al. (2014) como otro intento por resolver el problema del gradiente evanescente en las RNN. Las GRU comparten con las LSTM la idea de incorporar compuertas para lidiar con esta dificultad; sin embargo, las GRU solo cuentan con dos: una compuerta de actualización y una de reinicio. Además, a diferencia de las LSTM, no disponen de una celda de memoria adicional. Esta arquitectura más simple se traduce en un menor número de parámetros y, en consecuencia, en un entrenamiento más rápido. No obstante, tanto las LSTM como las GRU presentan ventajas y desventajas, por lo que la elección entre una y otra depende de la aplicación y del contexto. En particular, las LSTM tienden a desempeñarse mejor en tareas con dependencias de largo plazo marcadas.

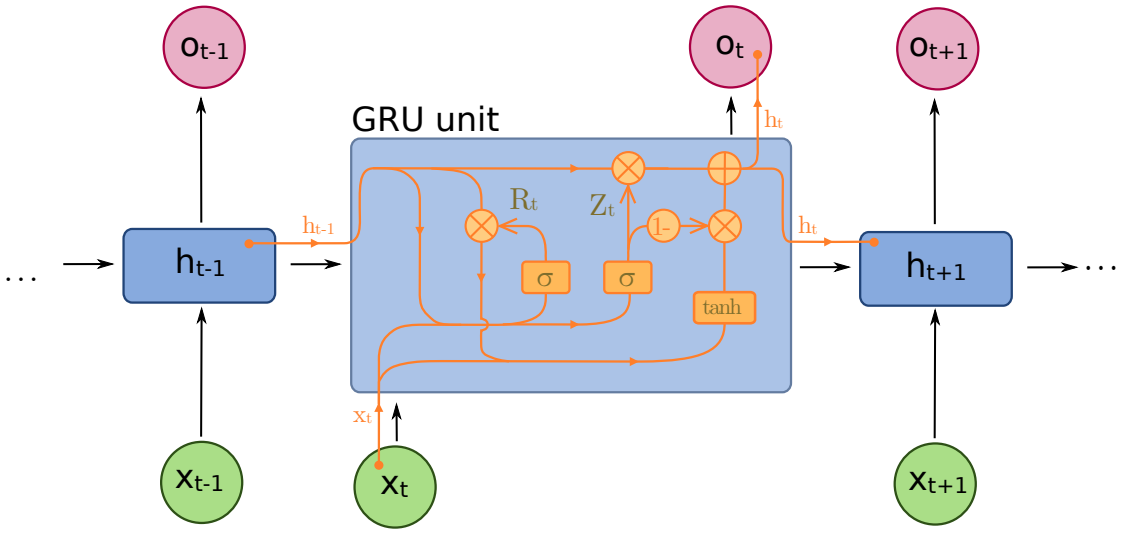


Figura 10: Estructura interna de una celda GRU. A diferencia de la LSTM, la GRU combina las compuertas de entrada y olvido en una sola *compuerta de actualización* z_t, y agrega una *compuerta de reinicio* r_t que controla cuánta información del pasado se descarta. No hay una celda de memoria separada: el estado oculto h_t integra la información directamente. Fuente: fdeloche (2017b).

2.2.5 La arquitectura Transformer

El Transformer, introducido por Vaswani et al. (2017) , representa un cambio de paradigma frente a las arquitecturas recurrentes. A diferencia de las RNN y LSTM, que procesan secuencias de manera secuencial, el Transformer se basa enteramente en mecanismos de atención, lo que permite el procesamiento paralelo de todos los tokens de una secuencia simultáneamente. Esta propiedad, sumada a la posibilidad de entrenar a gran escala sobre hardware masivamente paralelo (GPUs y TPUs), es la principal razón por la que el Transformer se convirtió en la arquitectura dominante del PLN desde 2017 (Tunstall et al., 2022).

Arquitectura encoder–decoder. El Transformer original se organiza en dos mitades simétricas. El *encoder* recibe la secuencia de entrada y produce, para cada token, una representación contextual que incorpora información de todos los demás tokens. El *decoder* genera la secuencia de salida token a token, consumiendo tanto la salida del encoder como los tokens previamente generados. Cada mitad es una pila de bloques que se compone, básicamente, de dos sub-capas: una de atención multi-cabezal seguida de una red *feed-forward* posición por posición, con conexiones residuales y normalización de capa.

Mecanismo de atención escalada (scaled dot-product attention). Para cada posición de la secuencia se proyectan tres vectores a partir de su representación de entrada: una *query* (q), una *key* (k) y un *value* (v). La atención entre una query y todas las keys se calcula como la similitud coseno escalada:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

donde d_k es la dimensión de las keys. Intuitivamente, la query representa la pregunta *¿a qué tokens de la entrada debería prestarle atención esta posición?*, las keys representan *identificadores* contra los que se compara esa pregunta, y los values son la *información* que efectivamente se agrega a la representación de la posición actual, ponderada por la similitud query-key (Vaswani et al., 2017).

Multi-head attention. En lugar de aplicar una única atención, el Transformer aprende h proyecciones independientes de Q, K y V y aplica la atención escalada en paralelo sobre cada una. Las salidas se concatenan y se proyectan de nuevo. Esta *multi-cabezal* permite que cada cabeza aprenda a atender a distintos tipos de relaciones: una cabeza puede especializarse en la dependencia sintáctica sujeto–verbo, otra en correferencias, otra en la posición local, etc. Formalmente:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

$$\text{donde head}_i = \text{Attention}\Big(QW_i^Q, KW_i^K, VW_i^V\Big)$$

Positional encoding. Como la atención opera sobre conjuntos y no sobre secuencias, el Transformer no tiene noción intrínseca del orden de los tokens. Para reintroducir esa información se suma a cada embedding de token un *positional encoding*, una señal posicional de la misma dimensión. En el paper original se propuso una combinación de funciones sinusoidales de distintas frecuencias, lo que permite al modelo extrapolar a longitudes de secuencia no vistas durante el entrenamiento. Variantes más modernas como ALiBi (Press et al., 2022) y RoPE (*Rotary Position Embedding*) (Su et al., 2024) codifican la posición relativa directamente dentro del cómputo de la atención y son habituales en LLMs contemporáneos.

Máscara causal en el decoder. Para que el decoder pueda entrenarse de forma paralela sobre toda la secuencia de salida (en lugar de estrictamente token a token), se aplica una *máscara causal* sobre la matriz de atención que impide a cada posición atender a tokens futuros. El resultado es la *masked multi-head self-attention*, que es la operación central de los modelos generativos tipo GPT (Radford et al., 2018).

2.2.6 Modelos fundacionales basados en Transformer

A partir de la arquitectura Transformer surgieron los grandes modelos de lenguaje (LLM, por sus siglas en inglés), entrenados a partir de corpus de texto de escala masiva. Dos familias principales dominaron el panorama inicial:

- **BERT** (Devlin et al., 2018): modelo bidireccional basado exclusivamente en el **encoder** del Transformer, optimizado para tareas de extracción y comprensión de información a partir de texto.
- **GPT** (Radford et al., 2018): modelo unidireccional basado en el **decoder** del Transformer, especializado en la predicción y generación de secuencias de texto.

Estas arquitecturas dieron origen a diversas familias de LLMs como GPT, PaLM, LLaMA, entre otras, que se han convertido en la columna vertebral del PLN moderno. Para una revisión exhaustiva, consultar Zhao & others (2026).

2.2.7 Innovaciones recientes en modelos de lenguaje

A partir del transformer la investigación en los modelos se aceleró cada vez más, en esta sección se presenta una cronología de hitos relevantes en la evolución de los LLMs, desde 2022 hasta el momento de cierre del marco teórico de este trabajo (segundo semestre de 2025).

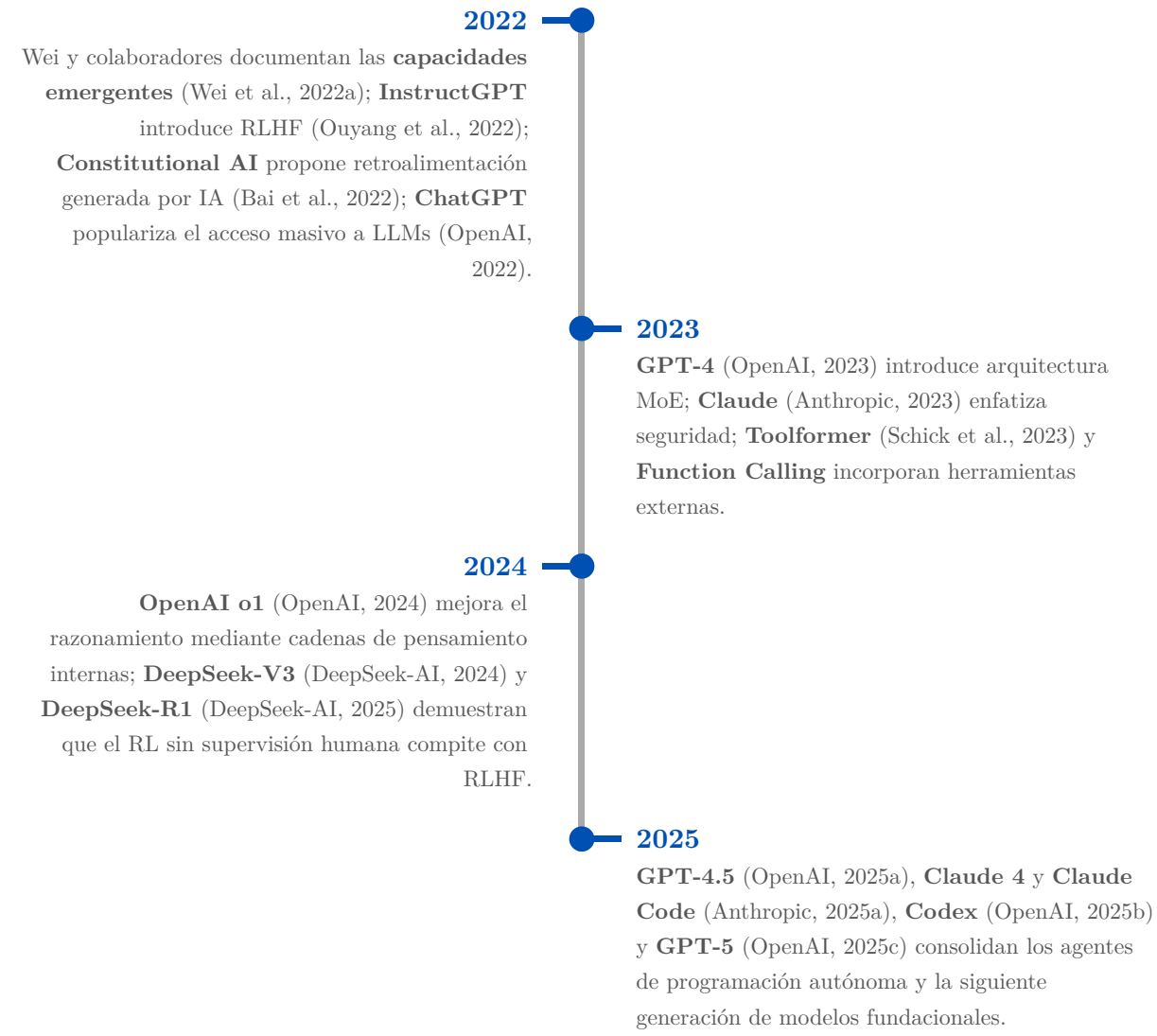


Figura 11: Cronología de los hitos principales en la evolución de los modelos del lenguaje (2022–2025). Tercera parte.

2022

- **Emergent Abilities of Large Language Models** (Wei et al., 2022a): Wei y colaboradores documentaron que ciertas capacidades de los modelos de lenguaje — resolución de problemas aritméticos simples, comprensión lectora de varios pasos, decodificación de instrucciones compuestas, razonamiento por analogías, entre otras — aparecen de manera discontinua y no predecible a partir de cierto umbral de escala (número de parámetros y de datos de entrenamiento). Estas *capacidades emergentes* no se observan en modelos pequeños ni pueden extrapolarse linealmente desde su comportamiento, lo que sugiere que los LLMs no son simplemente una versión *a*

mayor escala de sus contrapartes pequeñas, sino que en cierto régimen de tamaño exhiben habilidades cualitativamente nuevas. La caracterización rigurosa de este fenómeno abrió una línea de investigación muy activa sobre sus condiciones, límites e implicaciones para la seguridad y la interpretabilidad de los modelos.

- **InstructGPT** (Ouyang et al., 2022): entrenamiento de modelos de lenguaje para seguir instrucciones con retroalimentación humana (RLHF, por sus siglas en inglés).
- **Constitutional AI** (Bai et al., 2022): método para entrenar asistentes inofensivos mediante retroalimentación generada por la propia IA.
- **ChatGPT** (noviembre 2022) (OpenAI, 2022): lanzamiento público que popularizó el acceso masivo a los LLMs. Este hito puede entenderse como una manifestación a gran escala de las capacidades emergentes descritas por (Wei et al., 2022a): aunque los modelos GPT-3 de acceso programático ya exhibían buena parte de esas habilidades, fue la interfaz conversacional de ChatGPT la que las volvió evidentes para el público general. En la práctica, ChatGPT demostró que modelos entrenados primariamente para predecir el siguiente token eran capaces de seguir instrucciones matizadas, mantener el contexto a lo largo de un diálogo, rechazar pedidos inapropiados y producir texto de apariencia humana en una amplia variedad de dominios —capacidades para las que no habían sido entrenados de forma explícita. Este comportamiento, interpretado a la luz del trabajo de Wei y colaboradores, reforzó la idea de que las capacidades emergentes no son un artefacto de evaluación, sino una propiedad intrínseca de los modelos suficientemente grandes.

2023

- **GPT-4** (OpenAI, 2023): modelo multimodal con arquitectura **Mixture of Experts** (MoE), cuyos orígenes se remontan a (Jacobs et al., 1991) y (Shazeer et al., 2017).
- **Claude** (marzo 2023) (Anthropic, 2023): asistente desarrollado por Anthropic con énfasis en seguridad y alineamiento.
- **Toolformer** (Schick et al., 2023) y **OpenAI Function Calling**: incorporación de uso de herramientas externas por parte de los LLMs.

2024 y 2025

- **OpenAI o1** (OpenAI, 2024): modelo con capacidades de razonamiento mejoradas mediante cadenas de pensamiento internas.
- **DeepSeek-V3** (DeepSeek-AI, 2024) y **DeepSeek-R1** (DeepSeek-AI, 2025): modelos que demostraron que el aprendizaje por refuerzo sin supervisión humana —con técnicas como GRPO (**Group Relative Policy Optimization**)— puede alcanzar calidad comparable al estado del arte a un costo significativamente menor.
- **GPT-4.5** (febrero 2025) (OpenAI, 2025a): iteración avanzada de la familia GPT.
- **Claude 4** y **Claude Code** (mayo 2025) (Anthropic, 2025a): agentes de larga ejecución con capacidades de programación autónoma.
- **Codex** (mayo 2025) (OpenAI, 2025b): agente de programación de OpenAI.
- **GPT-5** (agosto 2025) (OpenAI, 2025c): siguiente generación de modelos fundacionales.

Desarrollos posteriores al alcance de este trabajo

Desarrollos posteriores a septiembre de 2025 enfatizan el harness engineering para co-diseñar entornos de control y andamiajes de agentes, superando la ingeniería de prompts tradicional (Anthropic, 2025b; Hashimoto, 2026).

Paralelamente, el local-first AI se posiciona como estándar para la soberanía de datos, ejecutando modelos de frontera directamente en el hardware del cliente o infraestructura periférica (Cruzes, 2026; IBM Institute for Business Value, 2026).

2.3 Síntesis del capítulo

A lo largo de este capítulo se ha trazado la evolución del procesamiento del lenguaje natural desde sus orígenes estadísticos hasta los grandes modelos de lenguaje contemporáneos. Se partió de los modelos de n-gramas fundados en los trabajos de Markov y Shannon, se revisaron los sistemas basados en reglas y recuperación de información como ELIZA, y se introdujeron los espacios vectoriales y de estados como representaciones distribuidas del lenguaje. Posteriormente, se describió el desarrollo de las redes neuronales artificiales —desde el perceptrón de Rosenblatt y el algoritmo de *backpropagation* hasta las redes recurrentes, LSTM y GRU— que culminó en la arquitectura Transformer, basada enteramente en mecanismos de atención. Finalmente, se presentó una cronología de las innovaciones recientes (2022–2025), incluyendo las capacidades emergentes, el alineamiento mediante RLHF, el razonamiento encadenado y los agentes de programación autónoma.

Esta progresión sienta las bases técnicas necesarias para abordar la aplicación del PLN a dominios especializados. En particular, el dominio legal presenta desafíos propios —vocabulario técnico, razonamiento normativo y estructuras documentales extensas— que motivan el desarrollo de sistemas de recuperación aumentada y agentes capaces de operar sobre corpus jurídicos. Estos temas se abordan en el siguiente capítulo, dedicado a la aplicación del PLN al dominio legal.

3 Aplicación del PLN al Dominio Legal

Este capítulo aplica los conceptos de PLN y grandes modelos de lenguaje del capítulo Sección 2 al dominio específico del derecho. Se analizan las particularidades del lenguaje jurídico que lo vuelven un dominio desafiante, las tareas típicas que aborda el campo del *legal NLP*, las técnicas de adaptación de LLMs al dominio, los riesgos asociados (alucinaciones, sesgos) y, finalmente, la complementariedad entre LLMs y RAG que fundamenta la decisión metodológica del sistema desarrollado en este trabajo.

3.1 Características del dominio legal como problema de PLN

El derecho no es simplemente «otro dominio» del PLN: presenta una combinación de propiedades formales, estructurales y pragmáticas que lo vuelven especialmente desafiante para los modelos de lenguaje contemporáneos. Comprender estas particularidades es necesario para justificar las decisiones de diseño del sistema que se desarrollan en los capítulos posteriores.

Un lenguaje técnico, formulista y altamente referido. El lenguaje jurídico tiene tres rasgos distintivos que lo diferencian del lenguaje general. Primero, una densidad terminológica muy alta: palabras como *usufructo*, *prescripción adquisitiva*, *litisconsorcio* o *cosa juzgada* designan conceptos con definición legal precisa y un uso que rara vez coincide con su sentido coloquial. Segundo, un formulismo sintáctico estable: las normas se redactan siguiendo plantillas rituales («*Visto*, *Considerando*, *Resuelve*»; «*El Senado y la Cámara de Diputados… sancionan con fuerza de Ley*») que un LLM debe reconocer y respetar cuando genera texto del mismo tipo. Tercero, una red densa de remisiones internas: un artículo típicamente invoca a otros artículos, a decretos reglamentarios, a jurisprudencia concordante y a doctrina. Esta red es tan densa que la comprensión cabal de un solo artículo puede requerir la lectura de varias docenas de documentos vinculados. Ariai et al. (2025) caracterizan estas propiedades como los desafíos distintivos del PLN legal, destacando la complejidad del lenguaje y la limitada disponibilidad de corpus abiertos.

Documentos extensos con estructura jerárquica profunda. A diferencia de las tareas de PLN sobre textos cortos (reseñas, tweets, preguntas), los documentos legales son extensos: una ley promedio tiene decenas de artículos; un código civil puede tener varios miles; un fallo puede tener cientos de páginas. Además, presentan una estructura jerárquica rigurosa (Código, Título, Capítulo, Sección, Artículo, Inciso, Párrafo) que debe preservarse al fragmentarlos para indexación. Chalkidis et al. (2021) documentan esta extensión como uno de los principales obstáculos para los modelos de PLN estándar, y Niklaus et al. (2024) confirman la observación en un contexto multilingüe.

Asimetría en el costo del error. Un sistema de PLN mal calibrado en un dominio como recomendación de películas produce una recomendación subóptima; en el dominio legal, un error puede tener consecuencias graves. Una respuesta alucinada que invoca un artículo inexistente en un dictamen, o que aplica erróneamente una figura jurídica, puede afectar derechos de personas, la validez de un acto administrativo o el resultado de un juicio. Esta asimetría entre costo de error y costo de desarrollo del sistema es la

razón por la que el campo del PLN legal es más conservador en la adopción de modelos generativos: cualquier técnica que aumente la opacidad de la decisión (como los modelos *black-box*) debe compensarse con mecanismos robustos de trazabilidad y verificación. Ariai et al. (2025) identifican la detección y mitigación de sesgos y la necesidad de modelos interpretables entre los principales desafíos abiertos del área.

Dependencia jurisdiccional y multilingüismo. El derecho es un dominio profundamente localizado: una ley argentina solo es válida en Argentina, y aun dentro de un país la legislación nacional, provincial y municipal coexiste con sistemas federales o autonómicos. Esto hace que los modelos entrenados predominantemente en inglés (que es el caso de la mayoría de los LLMs generalistas y de los corpus legales como CaseLaw-BERT) tengan una utilidad limitada en jurisdicciones de habla hispana. Niklaus et al. (2024) ofrecen una visión sistemática de este problema, mostrando que el rendimiento de los modelos legales preentrenados en inglés cae significativamente cuando se aplican a jurisdicciones de Alemania, España, Francia, Italia y, especialmente, América Latina.

Sistema continental vs. common law. Una distinción fundamental que condiciona el diseño de cualquier sistema legal es la diferencia entre el sistema de *common law* (anglosajón), donde las decisiones judiciales son fuente de derecho y existe el principio de *stare decisis* (los tribunales deben seguir los precedentes), y el sistema continental o *civil law* (al que pertenece Argentina), donde la ley escrita es la fuente principal y la jurisprudencia es fuente complementaria. Esta diferencia tiene consecuencias directas en el diseño del sistema: en un sistema de *common law*, la base de conocimientos debe incluir fallos previos con énfasis en su valor argumentativo; en un sistema continental, la base debe priorizar la legislación vigente y sus modificaciones, y la jurisprudencia cumple un rol interpretativo. El sistema desarrollado en este trabajo se enfoca en el sistema continental argentino, específicamente en el Código de Minería, por lo que su diseño de recuperación privilegia la legislación escrita sobre la jurisprudencia.

3.2 Breve historia del PLN aplicado al derecho

La intersección entre el procesamiento de lenguaje natural y el derecho tiene más de seis décadas, aunque el grueso del trabajo se concentró en las últimas dos. A continuación se presenta una síntesis cronológica de los hitos principales.

Años 1960–1970: información legal y sistemas expertos tempranos. El primer sistema de recuperación de información legal fue construido por John Horty en la Universidad de Pittsburgh en 1960, enfocado en textos de *statutory law* (Horty, 1960). En 1973 se lanzó LEXIS, el primer sistema comercial de búsqueda en textos legales completos de EE.UU., seguido en 1975 por Westlaw. Estos sistemas se basaban en búsqueda booleana de palabras clave con operadores lógicos (AND, OR, NOT) y no utilizaban técnicas de PLN propiamente dichas. En paralelo, durante los años 80, se desarrollaron los primeros *sistemas expertos legales* (LDS, HYPO, CABARET), que aplicaban razonamiento basado en casos (*case-based reasoning*) para emular el razonamiento de un abogado[13].

[13] https://en.wikipedia.org/wiki/Legal_expert_system

Años 2000: recuperación de información legal con técnicas estadísticas. La maduración de técnicas como Latent Semantic Analysis (LSA) y clasificadores SVM permitió los primeros trabajos de PLN propiamente dicho aplicados a textos legales: categorización de documentos por área del derecho, *e-discovery* (búsqueda de evidencia en grandes volúmenes de documentos para litigios) y extracción de información estructurada (nombres de partes, fechas, montos). El *benchmark* de búsqueda de jurisprudence desarrollado para las primeras competencias de *Legal Track* de TREC (Text REtrieval Conference) consolidó la evaluación cuantitativa del área (Baron et al., 2006).

Año 2010–2017: la era de los embeddings y las redes neuronales aplicadas a textos legales. La popularización de *word embeddings* (Word2Vec (Mikolov et al., 2013), GloVe (Pennington et al., 2014)) y, poco después, de las redes neuronales convolucionales y recurrentes aplicadas a texto, habilitó los primeros trabajos de representación distribuida del lenguaje jurídico. Surgieron también los primeros recursos en español orientados al dominio legal, aunque la disponibilidad siguió siendo limitada en comparación con los corpus en inglés. Sin embargo, los modelos seguían entrenándose con cantidades modestas de datos, y el rendimiento en tareas legales quedaba muy por debajo del de los profesionales humanos.

Año 2018–2020: el salto a los modelos de lenguaje de dominio. La aparición de BERT (Devlin et al., 2018) y la disponibilidad de grandes corpus de texto legal digitalizado (CourtListener, EUR-Lex, Westlaw, Harvard Case Law) habilitaron el preentrenamiento de modelos *desde cero* sobre corpus jurídicos. El hito más importante de esta etapa es *Legal-BERT* (Chalkidis et al., 2020), una familia de modelos BERT preentrenados sobre el corpus *Case Law Access Project* (Harvard, 3.4 millones de casos federales y estatales de EE.UU.) que superó a BERT general en todas las tareas de LegalBench. Le siguieron variantes regionales: *CaseLaw-BERT* (Australia), *EUBERT* (legislación de la UE), *Legal-BERTimbau* (Silveira et al., 2023) (Brasil), entre otros. Estos modelos demostraron empíricamente que la adaptación de dominio produce ganancias significativas en tareas especializadas, validando la línea de investigación de LLMs jurídicos verticales.

Año 2020–2022: la irrupción de los LLMs generalistas. La aparición de GPT-3 (Brown et al., 2020) y de los modelos *instruct* (InstructGPT (Ouyang et al., 2022), ChatGPT) cambió el panorama: LLMs entrenados con corpus generalistas comenzaron a desempeñarse sorprendentemente bien en tareas legales, sin necesidad de fine-tuning específico de dominio. Esta observación abrió un debate metodológico en curso: ¿conviene adaptar un LLM generalista al derecho vía fine-tuning, o es más eficiente usar un LLM generalista combinado con técnicas de *prompt engineering* y RAG? (Guha et al., 2023) y (Pipitone & Alami, 2024) ofrecen evidencia experimental sobre esta cuestión para el ecosistema en inglés.

Año 2023–presente: agentes y RAG en el dominio legal. A partir de 2023, la convergencia de LLMs generalistas competentes, *frameworks* agénticos maduros (LangChain, LlamaIndex, AutoGen) y técnicas de RAG llevó al surgimiento de los primeros sistemas comerciales y académicos de consulta legal basados en agentes: Casetext

CoCounsel (ahora Thomson Reuters), Harvey, Spellbook (contratos), y los productos regionales como Lexius y Juztina en Argentina. Ariai et al. (2025) ofrecen una revisión sistemática de este ecosistema, identificando la integración de RAG y agentes como una de las principales direcciones de desarrollo. Esta es la línea en la que se inscribe el presente trabajo.

3.3 Tareas típicas de PLN en el dominio legal

El campo del *legal NLP* ha consolidado un conjunto de tareas canónicas que sirven tanto para evaluar modelos como para diseñar sistemas. La taxonomía más completa y reciente es la de *LegalBench* (Guha et al., 2023), que define 162 tareas agrupadas en seis categorías. Ariai et al. (2025) ofrecen una revisión sistemática más amplia que cubre tareas adicionales como resumen de documentos legales, minería de argumentos y predicción de fallos, confirmando que la taxonomía de LegalBench sigue siendo la referencia más estructurada para el ecosistema en inglés.

1. **Issue Spotting (identificación de cuestiones jurídicas).** Determinar si un texto plantea una cuestión jurídica específica, o clasificar el área del derecho (civil, penal, laboral, administrativo, etc.) a la que pertenece un documento. Ejemplo: dado un fallo, identificar si trata sobre responsabilidad extracontractual o sobre derecho de propiedad.

2. **Rule Application (aplicación de reglas).** Dado un conjunto de hechos y una norma, determinar si la norma se aplica a los hechos. Esta tarea modela el razonamiento silogístico básico del derecho: subsumir los hechos en la hipótesis de la norma.

3. **Rule Conclusion (extracción de conclusiones normativas).** Extraer o predecir la conclusión jurídica a la que se arriba tras aplicar una o más normas a un conjunto de hechos. Incluye la *predicción de fallos* (*legal judgment prediction*), que es una tarea académicamente interesante pero éticamente controversial.

4. **Interpretation (interpretación de normas).** Resolver ambigüedades, identificar el alcance de un término jurídico, o distinguir entre interpretaciones alternativas de una norma.

5. **Rhetorical (análisis retórico-jurídico).** Identificar la estructura argumentativa de un documento legal: clasificar párrafos según su función (hechos, derecho, petición), identificar la tesis de un recurso, o detectar argumentos de las partes.

6. **Fact Determination (determinación de hechos).** Extraer hechos relevantes de un caso, evaluar su credibilidad o relevancia probatoria, o determinar la relación entre hechos y cuestiones jurídicas.

De manera complementaria, (Pipitone & Alami, 2024) destacan otra familia de tareas que gana relevancia con la aparición de los sistemas RAG:

1. **Recuperación de información legal (LIR, *Legal Information Retrieval*).** Dado un corpus legal y una consulta en lenguaje natural, recuperar los documentos o artículos más relevantes. Esta tarea es el componente central de cualquier sistema

RAG legal: la calidad de la respuesta final está acotada por la calidad de la recuperación.

2. **Respuesta a preguntas legales (LegalQA).** Dado un contexto legal (uno o varios documentos recuperados) y una pregunta, generar una respuesta fundamentada en el contexto. Esta es la tarea objetivo del sistema desarrollado en este trabajo: el usuario realiza una consulta sobre el Código de Minería Argentino y el sistema produce una respuesta basada en los artículos relevantes.

3. **Verificación de afirmaciones (*fact-checking*).** Determinar si una afirmación sobre derecho es correcta, está bien atribuida o es consistente con la normativa vigente. Esta tarea es crítica en el contexto de las alucinaciones: un sistema legal debe poder verificar sus propias salidas.

El sistema de este trabajo se enfoca específicamente en dos de estas tareas: **recuperación de información legal** y **respuesta a preguntas legales** sobre el Código de Minería Argentino. La elección se debe a que estas son las tareas directamente necesarias para asistir a un usuario no especializado (o incluso a un profesional) en la consulta puntual de un artículo, y son las que mejor se adaptan a la arquitectura RAG agéntica que se propone. La **predicción de fallos** y la **clasificación de cláusulas**, aunque valiosas, quedan fuera del alcance de este trabajo y se mencionan como líneas de trabajo futuro.

3.4 Adaptación de grandes modelos de lenguaje al dominio legal

Una vez disponibles los LLMs y los corpus legales digitalizados, la pregunta operativa es: ¿cómo especializamos un LLM en derecho? Existen cuatro familias principales de técnicas, ordenadas de mayor a menor costo computacional y, en general, de mayor a menor grado de modificación del modelo base.

Pre-entrenamiento de dominio (domain-adaptive pretraining). Consiste en continuar el entrenamiento auto-supervisado (predicción del siguiente token o *masked language modeling*) de un LLM base sobre un corpus jurídico, sin necesidad de datos etiquetados. Es la técnica más costosa en cómputo y datos: requiere millones de documentos y semanas de entrenamiento sobre clusters de GPUs. Su justificación es que el lenguaje jurídico tiene distribuciones léxicas, sintácticas y semánticas que difieren del lenguaje general, y que exponer al modelo a grandes cantidades de texto legal durante el preentrenamiento produce representaciones mejor adaptadas. *Legal-BERT* (Chalkidis et al., 2020) es el ejemplo paradigmático: un BERT preentrenado desde cero sobre el *Case Law Access Project* supera a BERT base en todas las tareas de LegalBench.

Fine-tuning completo (full fine-tuning). Una vez preentrenado el modelo base (con o sin adaptación de dominio), se ajustan **todos** sus parámetros con datos etiquetados específicos de la tarea objetivo. Por ejemplo, dado un dataset de preguntas y respuestas sobre el Código Civil Argentino, se ajusta el modelo para que responda correctamente. Es más barato que el preentrenamiento porque los datasets son más pequeños, pero para LLMs grandes (más de 7 mil millones de parámetros) requiere hardware especializado y corre el riesgo de *catastrophic forgetting* (el modelo «olvida» conocimiento general

al especializarse). Ariai et al. (2025) revisan las estrategias de adaptación de modelos generalistas al dominio legal, incluyendo el fine-tuning completo, y señalan que el olvido catastrófico es una limitación significativa de este enfoque.

Parameter-Efficient Fine-Tuning (PEFT). Conjunto de técnicas que ajustan solo una pequeña fracción de los parámetros del modelo, manteniendo el modelo base congelado. La técnica más popular es *LoRA* (Hu et al., 2021), que inyecta matrices de bajo rango en las capas de atención y solo entrena estas matrices (1–5% de los parámetros totales). *QLoRA* (Dettmers et al., 2023) combina LoRA con cuantización a 4 bits, permitiendo fine-tunear modelos de 65 mil millones de parámetros en una sola GPU de consumo. Variantes como *prefix-tuning*, *prompt-tuning* y *adapters* siguen la misma filosofía. En el dominio legal, (Bommasani et al., 2023) aplican técnicas de mitigation de sesgos sobre *Legal-BERT* usando variantes de adapters, mostrando que se puede especializar preservando propiedades de fairness.

In-Context Learning (ICL). No se ajustan pesos en absoluto: el modelo recibe en su *prompt* algunos ejemplos representativos de la tarea (*few-shot*) o instrucciones detalladas (*zero-shot*), y generaliza a partir de ellos. La introducción de los LLMs de gran escala demostró que el ICL permite resolver tareas nuevas sin necesidad de entrenamiento adicional, lo cual es especialmente útil en derecho donde la jurisprudencia evoluciona y donde etiquetar datos es costoso. Sahoo et al. (2024) ofrecen una revisión sistemática de las técnicas de prompt engineering, incluyendo ICL, y Schulhoff et al. (2024) presentan una taxonomía exhaustiva de 58 técnicas de prompting basadas en texto. Los prompts del sistema desarrollado en este trabajo se apoyan fuertemente en ICL combinado con RAG, en línea con la decisión de no entrenar un modelo propio.

La elección entre estas técnicas depende de restricciones concretas: presupuesto computacional, disponibilidad de datos etiquetados, necesidad de transparencia y auditabilidad. En el dominio legal, la auditabilidad es especialmente importante: un sistema cuyo razonamiento pueda ser inspeccionado y reproducido es preferible a una *caja negra* aunque tenga mayor precisión numérica. Esto tiende a favorecer técnicas que preserven el modelo base y solo modifiquen componentes identificables (PEFT, ICL, RAG) por sobre el fine-tuning completo.

3.5 Prompt engineering y razonamiento encadenado para tareas legales

El *prompt engineering* es la disciplina de diseñar los textos de entrada al LLM para obtener la salida deseada (Sahoo et al., 2024). En el dominio legal, donde las salidas deben ser precisas, fundamentadas y citables, las técnicas de prompt engineering tienen un papel central. A continuación se describen las principales, con énfasis en las que son relevantes para el sistema de este trabajo.

Zero-shot, one-shot y few-shot prompting. En el *zero-shot prompting*, el LLM recibe únicamente la instrucción de la tarea; en *one-shot* recibe un ejemplo de referencia; en *few-shot* recibe varios ejemplos. En el dominio legal, los ejemplos suelen ser fragmentos de jurisprudencia o de dictámenes bien redactados, que sirven como

«molde» estilístico para la respuesta. La ventaja del *few-shot* es que el modelo puede capturar el formato, la profundidad argumentativa y la citación esperada sin necesidad de ajustar pesos. Schulhoff et al. (2024) sistematizan estas técnicas en su taxonomía de prompting, identificando el few-shot como una de las técnicas más efectivas para tareas que requieren un formato de salida específico.

Chain-of-Thought (CoT) prompting. Introducido por (Wei et al., 2022b), el *Chain-of-Thought* consiste en inducir al LLM a descomponer su razonamiento en pasos intermedios antes de producir la respuesta final. En el dominio legal, donde el razonamiento es típicamente silogístico (premisa mayor: la norma; premisa menor: los hechos; conclusión: la aplicación), CoT permite que el modelo muestre explícitamente la cadena de inferencia, lo cual mejora la precisión y la auditabilidad. Thomson Reuters[14] destaca el CoT como una de las cinco técnicas de prompt engineering más útiles para trabajo jurídico, junto con few-shot, RAG, y dos variantes específicas del dominio.

ReAct (Reason + Act). La técnica ReAct, propuesta por (Yao et al., 2023a), entrelaza razonamiento y acción: el LLM razona sobre qué información necesita y, en el mismo paso, ejecuta una acción (por ejemplo, consultar una base de datos legal) para obtenerla. ReAct es la base conceptual del paradigma de agentes que se aborda en profundidad en el capítulo Sección 4, y es la técnica de prompting que subyace al sistema de este trabajo: el agente *razona* sobre qué artículo consultar y *actúa* recuperándolo de la base vectorial.

Structured prompting para documentos largos. El contexto legal es extenso, y los LLMs tienen ventanas de contexto limitadas. Para documentos largos se requieren técnicas de *structured prompting* que organizan la entrada en secciones explícitas: contexto, consulta, instrucciones, ejemplos, restricciones, formato de salida esperado. (Klem & Al Moubayed, 2025) proponen una metodología estructurada específicamente para documentos legales largos, mostrando que mejora la fidelidad de las respuestas y reduce las omisiones de información relevante. Esta metodología es particularmente relevante para el sistema de este trabajo, donde la entrada al LLM incluye tanto el contexto recuperado (uno o varios artículos del Código de Minería) como la consulta del usuario y las instrucciones sobre cómo responder.

Prompts con jurisprudencia y contexto legal. Una técnica específica del dominio que merece mención es el *prompt con jurisprudencia* (también conocido como *in-context RAG*): el LLM recibe no solo la consulta del usuario y los documentos relevantes, sino también uno o más fallos concordantes que ilustran el patrón argumentativo esperado. Esta técnica aprovecha la capacidad de los LLMs de imitar estilos y estructuras, y es particularmente útil para tareas como la redacción de borradores de escritos o la elaboración de dictámenes. Ariai et al. (2025) revisan esta técnica dentro del conjunto de estrategias de adaptación de modelos generalistas al dominio legal.

El sistema de este trabajo combina varias de estas técnicas: few-shot prompting para fijar el formato de respuesta, Chain-of-Thought para inducir razonamiento explícito, y

[14] https://legal.thomsonreuters.com/blog/prompt-engineering-best-ai-output/

structured prompting para organizar la entrada con contexto recuperado + consulta + restricciones. La descripción detallada del prompt concreto se presenta en el capítulo Sección 5.

3.6 Alineamiento, sesgos y equidad en modelos del lenguaje jurídicos

El uso de LLMs en el dominio legal introduce riesgos éticos y sociales que merecen consideración explícita. A diferencia de la mayoría de las aplicaciones de PLN, en el derecho los errores o sesgos del modelo pueden traducirse en decisiones que afectan derechos fundamentales de las personas. Esta sección revisa los principales riesgos identificados en la literatura.

Sesgos demográficos en embeddings legales. El estudio más sistemático sobre este problema es el de (Bommasani et al., 2023), quienes demostraron que *Legal-BERT* y otros modelos preentrenados sobre jurisprudencia estadounidense reproducen y amplifican sesgos de género presentes en los datos históricos. En particular, los embeddings asocian sistemáticamente profesiones y roles según estereotipos de género (por ejemplo, asocian *juez* con pronombres masculinos y *secretaria* con femeninos), y estas asociaciones correlacionan con decisiones judiciales reales que pueden estar sesgadas. Mitigar estos sesgos requiere intervenciones específicas (re-weighting, counterfactual data augmentation, debiasing de embeddings) y, fundamentalmente, auditoría continua.

Equidad en la clasificación automática de documentos. Si un sistema legal usa PLN para clasificar automáticamente presentaciones (por urgencia, por área del derecho, por tipo de proceso), los sesgos históricos del corpus pueden traducirse en una distribución desigual de la atención o los recursos. Por ejemplo, si históricamente las presentaciones de ciertos barrios o comunidades son sistemáticamente clasificadas como «menos urgentes», el modelo perpetuará esa asignación de recursos. Auditar la equidad del sistema requiere métricas específicas (igualdad de oportunidad, paridad demográfica) y, idealmente, datasets que representen la diversidad de usuarios afectados. Ariai et al. (2025) identifican la detección y mitigación de sesgos como uno de los dieciséis desafíos abiertos del PLN legal.

Sesgo jurisdiccional y dependencia cultural. Los modelos entrenados predominantemente con datos de jurisdicciones anglosajonas (EE.UU., Reino Unido) presentan un sesgo sistemático hacia el *common law* y hacia categorías jurídicas propias de esas tradiciones. Aplicados a jurisdicciones de *civil law* como la argentina, pueden producir respuestas que invocan principios inexistentes (por ejemplo, *stare decisis*) o que interpretan conceptos con categorías equivocadas. La solución no es «neutralizar» el modelo, sino entrenarlo o ajustarlo con datos de la jurisdicción objetivo, como se hace en este trabajo al restringir el corpus al Código de Minería Argentino. Niklaus et al. (2024) documentan esta caída de rendimiento al transferir modelos entre jurisdicciones.

Automation bias y responsabilidad profesional. Aun cuando un sistema de PLN legal sea técnicamente preciso, existe el riesgo documentado de *automation bias*: la tendencia de los profesionales a confiar ciegamente en la salida de un sistema

computacional, sin verificar. En el derecho, donde la responsabilidad profesional del abogado o del juez es ineludible, este riesgo es particularmente grave. Un sistema bien diseñado debe incluir disclaimers explícitos, citaciones verificables y señales claras de incertidumbre (por ejemplo, «no encuentro un artículo que responda directamente a esta pregunta»), y debe resistir la tentación de generar respuestas plausibles pero infundadas. Conley (2024) analizan este fenómeno en el contexto de la práctica jurídica con IA generativa y argumentan que la educación legal debe abordar explícitamente la prevención del sesgo de automatización.

Transparencia y auditabilidad. El principio de *publicidad* de las decisiones judiciales y de *motivación* de los actos administrativos exige que las decisiones automatizadas que afecten derechos sean trazables y auditables. Esto implica que un sistema legal basado en LLM debe poder responder a preguntas del tipo: «¿en qué artículos te basaste?», «¿qué versión de la norma usaste?», «¿hay jurisprudencia concordante o contradictoria?». La arquitectura RAG, al anclar cada respuesta en documentos recuperables, es naturalmente más auditable que un LLM cerrado. La arquitectura agéntica extiende esta auditabilidad al razonamiento: el agente puede mostrar el plan que siguió y las herramientas que invocó. Ariai et al. (2025) destacan la necesidad de modelos más interpretables y la mejora de la explicabilidad como desafíos centrales para manejar la complejidad del razonamiento legal.

3.7 Riesgos centrales: alucinaciones y citaciones falsas

El riesgo más conocido y más grave del uso de LLMs en el derecho es la generación de información jurídica falsa pero plausible: citas a leyes o precedentes que no existen, artículos derogados presentados como vigentes, o procedimientos descritos incorrectamente. Este fenómeno se enmarca en el problema más amplio de las *alucinaciones* de los LLMs, que (Islam et al., 2025) definen como «la generación de información que no está basada en datos reales o verificables».

Taxonomía de errores legales específicos. Aun cuando el problema general de las alucinaciones es transversal a todos los dominios, en el derecho adquiere formas específicas que es útil distinguir:

1. **Errores factuales.** El modelo invoca una ley, un artículo, un decreto o un código que no existe o que no corresponde a la jurisdicción. Por ejemplo, citar el «artículo 1234 del Código de Minería» cuando el código solo tiene 245 artículos.

2. **Citas jurisprudenciales falsas.** El modelo atribuye una doctrina a un fallo que no la sostiene, o inventa precedentes. Este fue el caso célebre de *Mata v. Avianca* (2023, Tribunal de Distrito de EE.UU., S.D.N.Y.), donde los abogados presentaron escritos citando precedentes generados por ChatGPT que no existían. El caso resultó en sanciones a los abogados y se convirtió en referencia mundial sobre los riesgos legales del uso de LLMs.

3. **Errores de versión temporal.** El modelo invoca una norma que fue derogada, modificada o sustituida, presentándola como vigente. En el derecho argentino, donde la legislación cambia con frecuencia mediante leyes, decretos y resoluciones adminis-

trativas, este riesgo es particularmente alto si la base de conocimiento no se actualiza regularmente.

4. **Errores de procedimiento.** El modelo describe plazos, requisitos o pasos procesales que no aplican, o que han cambiado. En el derecho procesal, estos errores pueden llevar a la pérdida de derechos (por ejemplo, sugerir un plazo de 30 días cuando el aplicable es de 15).

Detección de alucinaciones: estado del arte. Detectar automáticamente alucinaciones es un problema abierto. Las técnicas actuales incluyen: (a) verificación factual contra bases de conocimiento externas; (b) métricas de *factual consistency* que comparan la respuesta con documentos de referencia; (c) atribución de afirmaciones a fuentes específicas (*attribution accuracy*); (d) el uso de un segundo LLM como *verificador* que evalúa la consistencia de la respuesta generada. Sin embargo, ninguna de estas técnicas es infalible: el verificador puede tener los mismos sesgos que el modelo verificado, y las métricas de factual consistency pueden pasar por alto errores sutiles. Ariai et al. (2025) revisan estas técnicas y señalan que la mejora de la factualidad y la atribución correcta de fuentes siguen siendo desafíos abiertos.

RAG como mitigación estructural. (Shuster et al., 2021) demostraron que el uso de RAG reduce significativamente las alucinaciones, porque cada afirmación de la respuesta puede anclarse en un documento concreto de la base de conocimiento. En el dominio legal, esta propiedad es especialmente valiosa: una respuesta que cita un artículo específico del Código de Minería es verificable abriendo el código en ese artículo, mientras que una respuesta «desde el conocimiento del modelo» no lo es. La arquitectura RAG ofrece, además, una forma natural de *expresar incertidumbre*: si la base de conocimiento no contiene un artículo relevante para la consulta, el sistema puede decirlo explícitamente en lugar de inventar uno.

Agentic-RAG como mitigación extendida. La arquitectura agéntica extiende las propiedades de RAG con tres mecanismos adicionales de mitigación: (a) descomposición de la consulta en sub-preguntas más específicas, cada una anclada en un documento; (b) verificación cruzada entre los documentos recuperados y la respuesta generada; (c) registro explícito del razonamiento del agente, que permite auditar qué herramientas se invocaron y por qué. (Gosmar & Dahl, 2025) muestran que un sistema multi-agente con un protocolo estandarizado de comunicación reduce aún más las alucinaciones en comparación con un único LLM.

En el sistema de este trabajo, la mitigación de alucinaciones es una prioridad de diseño explícita. El agente está instruido para, ante la duda, declarar que no encuentra respuesta en la base documental en lugar de generarla libremente. La evaluación del sistema (capítulo Sección 6) incluye métricas diseñadas para detectar y cuantificar errores factuales y de citación, justamente porque estos son los modos de falla más peligrosos en el dominio legal.

3.8 Síntesis del capítulo

La decisión metodológica central de este trabajo —desarrollar un sistema Agentic-RAG sobre el Código de Minería Argentino, y no un LLM aislado ni un RAG clásico— se fundamenta en la complementariedad entre tres componentes. Los LLMs aportan versatilidad cognitiva: capacidad de razonar sobre texto, seguir instrucciones complejas y producir respuestas coherentes (Gao et al., 2023). Sin embargo, presentan limitaciones estructurales en el dominio legal: conocimiento estático, imposibilidad de atribuir afirmaciones a fuentes específicas, sensibilidad a la formulación del prompt y generación de alucinaciones (Islam et al., 2025).

RAG resuelve la mayoría de estas limitaciones al anclar la generación en documentos externos recuperables, garantizando actualización, trazabilidad y la posibilidad de expresar incertidumbre. No obstante, el RAG clásico carece de mecanismos explícitos para descomponer consultas compuestas, planificar la recuperación o verificar la coherencia interna de la respuesta —operaciones que en el dominio legal son frecuentemente necesarias.

La arquitectura Agentic-RAG (Singh et al., 2025) integra estas capacidades: el agente descompone la consulta en sub-preguntas, planifica la exploración de la base documental, recupera documentos relevantes para cada sub-pregunta, sintetiza una respuesta anclada en ellos y verifica la coherencia interna y la atribución correcta. Ehtesham et al. (2025) muestran que estos sistemas ofrecen consistentemente mejor relación entre precisión factual y costo computacional que los LLMs aislados o los RAG clásicos en tareas que requieren múltiples pasos de razonamiento.

En síntesis, el LLM aporta versatilidad cognitiva, la base documental aporta factualidad y trazabilidad, y el agente aporta planificación y verificación. Los capítulos siguientes desarrollan cada uno de estos componentes en detalle, comenzando por el marco teórico de RAG y agentes en el capítulo Sección 4.

4 Agentic-RAG: Generación Aumentada por Recuperación Agéntica

Este capítulo presenta el marco teórico de Agentic-RAG, describiendo sus conceptos fundamentales, arquitecturas, patrones de flujo de trabajo, agentes especializados, casos de uso prácticos y consideraciones de ingeniería.

4.1 Concepto de Agentic-RAG

La Generación Aumentada por Recuperación (RAG) es una técnica que combina un sistema de búsqueda (recuperador) con un modelo de lenguaje grande (LLM) para generar respuestas fundamentadas en información externa, en lugar de depender únicamente del conocimiento que el modelo aprendió durante su entrenamiento. El recuperador busca documentos relevantes en una base de conocimiento y el modelo los utiliza como contexto para responder. Agentic-RAG extiende esta idea al dotar al sistema de un planificador que decide de manera autónoma cuándo y cómo buscar información, permitiendo una recuperación iterativa y un razonamiento en múltiples pasos (Mishra et al., 2026).

Frente a RAG tradicional, donde la recuperación es un preprocesamiento estático *retrieve-then-read*, Agentic-RAG convierte al LLM en un agente que decide qué acciones realizar (por ejemplo, buscar, reformular la consulta, consultar otra base, usar un grafo, etc.), cuándo iterar y cuándo detenerse. Esta autonomía permite adaptar la estrategia de recuperación a la complejidad de la tarea y del entorno, reduciendo ruido, mejorando la cobertura y habilitando flujos de trabajo complejos (Ferrazzi et al., 2026; Shorten & Monigatti, 2024).

4.2 Arquitecturas de Agentic-RAG

4.2.1 De RAG clásico a Agentic-RAG

En RAG clásico, la arquitectura se suele modelar como dos módulos: *Retriever* (vector search, BM25, etc.) y *Generator* (LLM) unidos en un pipeline estático de una sola pasada. Varios trabajos muestran que esta arquitectura sufre cuando las consultas son multi-hop, requieren desambiguación, o necesitan combinar múltiples fuentes heterogéneas (Mishra et al., 2026; Singh et al., 2025; Xiong et al., 2025).

Agentic-RAG introduce un tercer componente central: un *Planner/Controller* que convierte el pipeline en un proceso de decisión secuencial, donde cada paso puede desencadenar nuevas acciones de búsqueda, filtrado, verificación o reformulación. Este bucle de planificación-acción-observación se formaliza como un proceso de decisión parcialmente observable de horizonte finito (POMDP, por sus siglas en inglés *Partially Observable Markov Decision Process*) en el estudio de conocimiento (del inglés *Systematization of Knowledge*, SoK) «Agentic Retrieval-Augmented Generation» (Mishra et al., 2026).

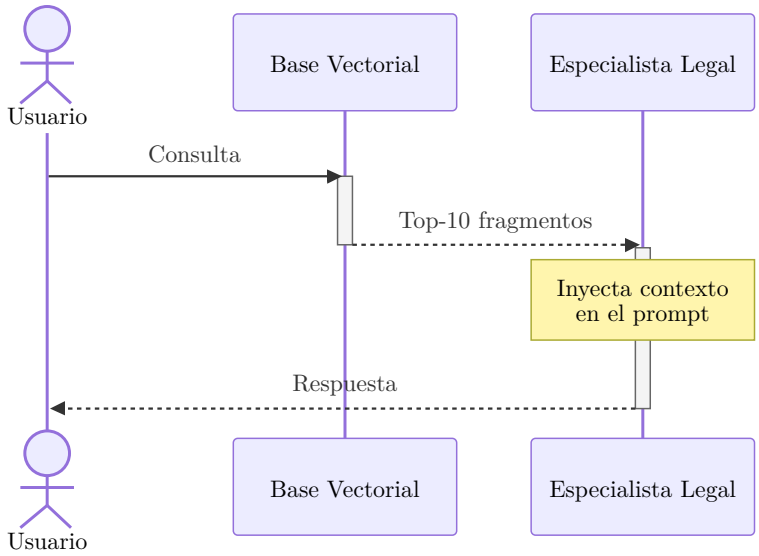


Figura 12: Arquitectura básica de RAG clásico.

4.2.2 Descomposición modular típica

La literatura reciente propone una descomposición modular de Agentic-RAG en al menos cuatro subsistemas (Mishra et al., 2026; Singh et al., 2025):

- **Planner / Policy Controller:** decide la siguiente acción (por ejemplo, llamar a una herramienta de búsqueda, leer un fragmento, cambiar de base de conocimiento, pedir verificación, etc.).
- **Motor de Recuperación (Retrieval Engine):** deja de ser un simple filtro pasivo y pasa a actuar como coprocesador lógico, combinando índices densos, dispersos, bases SQL y grafos de conocimiento mediante interfaces programables (Mishra et al., 2026).
- **Memoria / Estado:** almacena trayectoria de acciones, evidencias recuperadas, hipótesis intermedias y feedback del usuario o de otros agentes (Bandara et al., 2025; Singh et al., 2025).
- **Ejecutor de Herramientas / Entorno:** encapsula APIs externas (web search, bases vectoriales, bases relacionales, calculadoras, simuladores, etc.) y devuelve observaciones al agente (Achanta, 2025; Shorten & Monigatti, 2024).

4.2.3 Patrones de flujo de trabajo agéntico aplicados a RAG

La revisión bibliográfica de Agentic-RAG identifica varios patrones de flujos de trabajo agénticos reutilizados dentro de arquitecturas RAG (Singh et al., 2025):

- **Encadenamiento de instrucciones (*prompt chaining*):** secuencia de pasos donde salidas intermedias se convierten en entradas del siguiente (por ejemplo, reformular consulta → recuperar → agrupar → generar respuesta).
- **Enrutamiento (*routing*):** un agente clasificador deriva la consulta a distintos sub-flujos especializados (por ejemplo, factual simple vs. multi-salto vs. cálculo numérico) (Singh et al., 2025).

- **Paralelización (*parallelization*):** ejecución concurrente de múltiples recuperadores o agentes sobre variantes de la consulta, seguida de agregación (Singh et al., 2025).
- **Orquestador–Trabajadores (*orchestrator–workers*):** un orquestador central descompone la tarea en subtareas y asigna a agentes trabajadores especializados (por ejemplo, agente de búsqueda web, de grafo, de base SQL) (Salve et al., 2024; Singh et al., 2025).
- **Evaluador–Optimizador (*evaluator–optimizer*):** un evaluador (a menudo otro LLM) juzga la calidad de la evidencia y la respuesta, y un optimizador ajusta instrucciones, rutas o políticas (Singh et al., 2025; Xiong et al., 2025).

Estos patrones se implementan en marcos de trabajo como LangGraph, AutoGen o Hugging Face Agents, que proporcionan nodos y aristas para describir estados y transiciones del flujo de trabajo (Achanta, 2025; Singh et al., 2025).

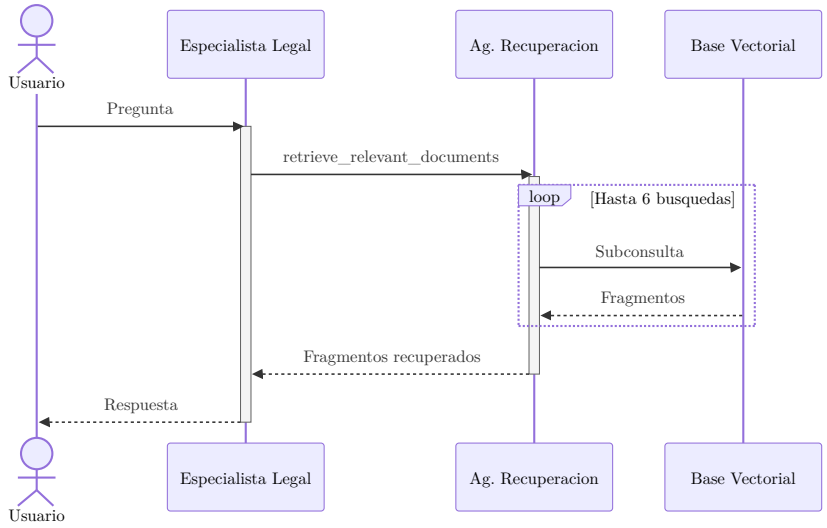


Figura 13: Arquitectura de Agentic-RAG con componentes agénticos.

4.2.4 Arquitecturas jerárquicas y multi-herramienta

Du et al. (2026) propone una arquitectura de Agentic-RAG basada en **interfaces de recuperación jerárquicas** expuestas directamente al modelo. Define tres herramientas de recuperación: *keyword search*, *semantic search* y *chunk read*, que el agente aprende a combinar de forma adaptativa según la tarea, escalando en tamaño de modelo y cómputo de inferencia.

Esta idea generaliza al concepto de **múltiples herramientas de recuperación**: el agente puede elegir entre distintos índices (vectorial, BM25, grafos, SQL) y granularidades (documento, sección, párrafo, frase), lo que le permite navegar un espacio de búsqueda mucho más expresivo que el de un único vector store (Mishra et al., 2026; Shorten & Monigatti, 2024).

4.2.5 Arquitecturas multi-agente

Varios trabajos implementan Agentic-RAG mediante colaboración de múltiples agentes especializados que comparten memoria y se coordinan mediante un orquestador (Salve et al., 2024; Singh et al., 2025; Yousefi Maragheh et al., 2025):

- ARAG (Personalized Recommendation) define cuatro agentes: *Agente de Comprensión del Usuario* (*User Understanding Agent*), *Agente de NLI* (*NLI Agent*)[15], *Agente de Resumen de Contexto* (*Context Summary Agent*) y *Agente de Ranking de Ítems* (*Item Ranker Agent*); cada uno procesa una parte distinta de la información (preferencias de usuario, alineación semántica, resumen contextual y ranking final) (Yousefi Maragheh et al., 2025).
- Sistemas de RAG multi-fuente asignan un agente por tipo de fuente (relacional, NoSQL, documentos, grafos), que generan subconsultas y devuelven vistas parciales al orquestador (Salve et al., 2024).

Este diseño mejora escalabilidad y permite extensibilidad incremental: añadir un nuevo tipo de fuente implica definir un nuevo agente y su herramienta.

4.2.6 Comparativa de arquitecturas

La Tabla 2 sintetiza las diferencias entre las principales variantes arquitectónicas de RAG (Du et al., 2026; Mishra et al., 2026; Salve et al., 2024; Singh et al., 2025; Yousefi Maragheh et al., 2025).

Dimensión	RAG clásico	Agentic-RAG monolítico	Agentic-RAG multi-agente
Control del flujo	Pipeline fijo 1–2 pasos	LLM decide secuencia de acciones	Orquestador + múltiples agentes coordinados
Recuperación	Una llamada a un índice	Múltiples llamadas iterativas y adaptativas	Recuperadores especializados por agente/ fuente
Herramientas	Normalmente solo búsqueda	Búsqueda + otras herramientas (cálculo, APIs)	Conjunto amplio de herramientas heterogéneas
Escalabilidad semántica	Limitada por el embedding	Mejora vía planificación y re-consulta	Mejora adicional vía especialización y paralelización
Complejidad de impl.	Baja	Media (diseño de policy y herramientas)	Alta (orquestación, memoria compartida, concurrencia)

Tabla 2: Comparativa de arquitecturas RAG.

[15]NLI: *Natural Language Inference* (Inferencia de Lenguaje Natural). Este agente evalúa, mediante un LLM, el grado de alineación semántica entre cada ítem candidato recuperado y la intención inferida del usuario, produciendo un puntaje que permite filtrar aquellos ítems que realmente respaldan la consulta o preferencia expresada (Yousefi Maragheh et al., 2025).

4.3 Agentes especializados en Agentic-RAG

4.3.1 Tipos de agentes por rol en el pipeline

La literatura y la práctica industrial convergen en una tipología de agentes especializados relativamente estable (Achanta, 2025; Shorten & Monigatti, 2024; Singh et al., 2025):

- **Agente Planner / Orquestador:** recibe la consulta del usuario, la analiza, define subtareas y planea la secuencia de acciones (por ejemplo, «reformule la consulta», «busque en el grafo», «combine resultados»).
- **Agentes de Recuperación (Retrieval Agents):** encapsulan estrategias de búsqueda concretas, como vector search, BM25, web search, bases SQL, grafos, APIs de dominio (Mishra et al., 2026; Shorten & Monigatti, 2024).
- **Agentes de Razonamiento y Fusión:** combinan evidencias, realizan razonamiento multi-hop, verifican consistencia y sintetizan una respuesta intermedia o final (Mishra et al., 2026).
- **Agentes Evaluadores / Críticos:** revisan respuestas y cadenas de pensamiento, detectan alucinaciones, miden relevancia de pasajes y pueden gatillar re-búsquedas (Singh et al., 2025; Xiong et al., 2025).
- **Agentes de Dominio Específico:** especializados en un dominio (sanidad, finanzas, testing software, religión, etc.), con acceso a herramientas y conocimiento específicos (Bhatia et al., 2026; Hariharan et al., 2025; Yousefi Maragheh et al., 2025; Zeggai et al., 2025; Zheng et al., 2025).

4.3.2 Ejemplos concretos de agentes especializados

- **ARAG (recomendación personalizada)** (Yousefi Maragheh et al., 2025):
  - *User Understanding Agent*: resume preferencias del usuario combinando contexto a largo plazo y de sesión.
  - *NLI Agent*: evalúa la alineación semántica entre ítems candidatos y la intención inferida.
  - *Context Summary Agent*: condensa las evidencias del NLI para reducir longitud de contexto.
  - *Item Ranker Agent*: genera el ranking final de recomendaciones.
- **AI-VaxGuide (decisiones de vacunación):** utiliza un marco Agentic-RAG para convertir guías de vacunación extensas en un sistema interactivo de preguntas y respuestas, donde agentes razonadores buscan y combinan recomendaciones nacionales y de la OMS (Zeggai et al., 2025).
- **Deep-DxSearch (diagnóstico médico):** entrena un sistema Agentic-RAG end-to-end mediante RL, donde el LLM actúa como investigador activo sobre perfiles de enfermedad, historiales clínicos y literatura biomédica; el agente formula queries, evalúa evidencia y refina hipótesis diagnósticas (Zheng et al., 2025).

- **IslamicFaithQA:** construye un marco Agentic-RAG para fundamentación coránica, utilizando llamadas estructuradas a herramientas de búsqueda sobre un corpus de versículos y recursos islámicos; el agente itera entre recuperar evidencias y revisar la respuesta para minimizar alucinaciones y abstenerse cuando no hay evidencia (Bhatia et al., 2026).
- **Agentic-RAG para testing de software:** combina agentes que trabajan sobre un sistema híbrido vector-grafo para automatizar generación de planes y casos de prueba, logrando importantes mejoras de precisión y eficiencia (Hariharan et al., 2025).

4.3.3 Agentes de recuperación avanzados

Trabajos de Agentic-RAG como los de Du et al. (2026) y Mishra et al. (2026) enfatizan que el agente de recuperación evoluciona de simple indexador a **motor de búsqueda semántico-lógico** que integra múltiples índices y estrategias:

- Soporta **interfaces jerárquicas** (buscar documentos, luego secciones, luego chunks) con herramientas diferenciadas (Du et al., 2026).
- Combina **vector search** con recuperación dispersa, queries estructuradas y grafos de conocimiento para superar las limitaciones de separabilidad lineal de embeddings densos en consultas lógicas complejas (Mishra et al., 2026).

En la práctica industrial, proveedores como Weaviate, IBM y Azure hablan de *agentic retrieval* como un pipeline especializado donde el LLM descompone la consulta en subconsultas, las ejecuta en paralelo sobre distintos recursos y luego reensambla las respuestas (Belcic & Stryker, 2025; Microsoft, 2026; Shorten & Monigatti, 2024).

4.3.4 Evaluadores, jueces y supervisores

Con el auge de RL para Agentic-RAG, emergen agentes especializados que actúan como **jueces de proceso** (process reward models) evaluando la calidad de cada paso de la trayectoria en lugar de solo el resultado final (Xiong et al., 2025; Zhang et al., 2026; 2025):

- ReasonRAG y TreePS-RAG utilizan recompensas de proceso para supervisar generación de queries, extracción de evidencias y síntesis de respuesta, mejorando estabilidad de entrenamiento y asignación de crédito (Zhang et al., 2026; 2025).
- En RAG-Gym se explora el uso de LLMs avanzados como jueces de proceso para evaluar la calidad de trayectorias de agentes, cuya señal se usa luego para entrenar políticas de búsqueda más eficientes (Xiong et al., 2025).

4.4 Flujo de trabajo en Agentic-RAG

4.4.1 Comparación de flujos: RAG estático vs. Agentic-RAG

En RAG estático, el flujo típico es: (1) indexación fuera de línea, (2) embedding y recuperación de top-k pasajes, (3) concatenación al prompt, (4) generación de respuesta. Este flujo no se adapta durante la inferencia: no hay verificación dinámica de si la

evidencia es suficiente, ni reformulación de la consulta (Mishra et al., 2026; Shorten & Monigatti, 2024).

En Agentic-RAG, el flujo se convierte en un bucle plan-act-observe-reflect donde el LLM (o conjunto de agentes) puede decidir: cuándo recuperar, cómo reformular, qué fuentes consultar, cuándo detener la búsqueda, y cómo fusionar evidencia (Ferrazzi et al., 2026; Mishra et al., 2026; Shorten & Monigatti, 2024).

4.4.2 Flujo de trabajo genérico de Agentic-RAG

Un flujo de trabajo genérico, extraído de los surveys y de guías industriales de IBM, Weaviate, Google y Microsoft, se puede describir así (Belcic & Stryker, 2025; Kolesnikov, 2026; Microsoft, 2026; Shorten & Monigatti, 2024; Singh et al., 2025):

1. **Comprensión y clasificación de la consulta.** Un agente planner analiza la consulta, detecta su tipo (factual simple, multi-salto, pregunta de dominio, tarea de síntesis, etc.) y decide si necesita recuperación externa. En algunos sistemas, un clasificador ligero estima la complejidad y escoge entre cero ejemplos (sin RAG), RAG simple o Agentic-RAG multi-paso (Singh et al., 2025).

2. **Planificación inicial.** El planner decide un plan de alto nivel: qué fuentes consultar (vector DB, web, grafo, SQL), qué herramientas usar y en qué orden (por ejemplo, primero ampliar contexto, luego focalizar) (Achanta, 2025; Shorten & Monigatti, 2024; Singh et al., 2025).

3. **Ciclo iterativo de recuperación y razonamiento.** El agente ejecuta un bucle estilo ReAct: *Thought* (razona sobre el siguiente paso), *Action* (invoca una herramienta: búsqueda, cálculo, ejecución de código, llamada API, etc.) y *Observation* (recibe resultados y actualiza su estado) (Patronus AI, 2025; Shorten & Monigatti, 2024). Este ciclo se repite hasta cumplir criterios de parada (confianza suficiente, límite de pasos/tokens, retroalimentación del usuario) (Mishra et al., 2026; Xiong et al., 2025).

4. **Fusión, verificación y corrección.** Un agente de fusión integra evidencias heterogéneas, aplica validaciones cruzadas (por ejemplo, comparar varias fuentes para detectar inconsistencias) y descarta ruido (Achanta, 2025; Mishra et al., 2026). Un evaluador puede puntuar la respuesta candidata y, si la calidad es baja, pedir más evidencia o reformular la consulta (Singh et al., 2025; Zhang et al., 2026; 2025).

5. **Síntesis y generación final.** El agente genera una respuesta final, potencialmente acompañada de citas, explicaciones de trazabilidad y advertencias cuando la evidencia es insuficiente (Bhatia et al., 2026; Mishra et al., 2026; Zheng et al., 2025).

6. **Actualización de memoria y aprendizaje.** El sistema puede actualizar memorias largas (por ejemplo, perfil de usuario en recomendación) y registros de trayectoria usados después para ajuste fino supervisado o aprendizaje por refuerzo (por ejemplo, ReasonRAG, TreePS-RAG) (Xiong et al., 2025; Yousefi Maragheh et al., 2025; Zhang et al., 2026; 2025).

4.4.3 Patrones de flujos de trabajo específicos

Las revisiones bibliográficas de Agentic-RAG y de flujos de trabajo agéntico identifican patrones recurrentes que se concretan en implementaciones prácticas (Bandara et al., 2025; Kirkovska & Vargas, 2025; Singh et al., 2025):

- **Multi-query / Multi-HyDE:** generar múltiples reformulaciones de una misma consulta para cubrir aspectos complementarios, ejecutar recuperaciones en paralelo y fusionar resultados (por ejemplo, en Financial RAG con Multi-HyDE) (Govind Srinivasan et al., 2025).
- **Finite State Machine (FSM):** representar el flujo de trabajo agéntico como una máquina de estados finitos donde cada estado es una fase (reformulación, búsqueda, verificación, síntesis) y las transiciones se controlan con reglas o políticas aprendidas (Govind Srinivasan et al., 2025).
- **Grafos de flujo de trabajo (LangGraph, StateFlow):** modelar cada nodo como un agente o herramienta y usar edges condicionales basadas en resultados intermedios (Achanta, 2025; Kirkovska & Vargas, 2025; Singh et al., 2025).
- **Orchestrator-Workers:** un agente orquestador divide la tarea en subtareas (por ejemplo, «extraer requisitos», «buscar casos similares», «proponer plan»), que se asignan a workers especializados; luego recoge y combina las salidas (Jeong, 2024; Salve et al., 2024; Singh et al., 2025).

4.4.4 Entrenamiento y optimización del flujo

Una tendencia clara es el uso de **RL y supervisión de proceso** para optimizar políticas Agentic-RAG, más allá de simple ingeniería de instrucciones (Xiong et al., 2025; Zhang et al., 2026; 2025):

- RAG-Gym proporciona un entorno unificado para entrenar agentes de búsqueda y razonamiento con supervisión de proceso, destacando la eficacia de LLMs como jueces de trayectoria (Xiong et al., 2025).
- ReasonRAG (Process vs. Outcome Reward) muestra que recompensas de proceso para generación de queries, extracción de evidencias y respuesta final mejoran estabilidad y eficiencia frente a RL solo con recompensa final (Zhang et al., 2025).
- TreePS-RAG modela el razonamiento agentic como un árbol de rollouts, estimando la utilidad de cada paso a partir de resultados de sus descendientes, lo que permite crédito fino sin anotaciones intermedias (Zhang et al., 2026).

Estos enfoques apuntan a flujos de trabajo de Agentic-RAG cada vez más aprendidos y menos manuales.

4.5 Casos de uso de Agentic-RAG

4.5.1 Dominios donde Agentic-RAG aporta ventajas claras

Las revisiones bibliográficas y estudios experimentales señalan que Agentic-RAG es especialmente útil cuando (Ferrazzi et al., 2026; Mishra et al., 2026; Singh et al., 2025):

- Las consultas requieren **razonamiento multi-hop** (por ejemplo, combinar múltiples documentos para llegar a una conclusión).
- Se necesitan **fuentes heterogéneas** (documentos, bases estructuradas, grafos, web en tiempo real).
- Hay **requisitos fuertes de trazabilidad y verificación**, como en salud, derecho o finanzas.
- La **complejidad de la tarea es variable**, por lo que conviene adaptar el grado de búsqueda y razonamiento.

El paper «Is Agentic RAG Worth It?» realiza una comparación experimental rigurosa entre RAG mejorado (Enhanced RAG) y Agentic-RAG, concluyendo que Agentic-RAG tiende a ofrecer mayores mejoras en escenarios complejos, aunque con mayor coste computacional (Ferrazzi et al., 2026).

4.5.2 Salud y diagnóstico clínico

- **Deep-DxSearch:** un sistema de diagnóstico basado en Agentic-RAG que entrena end-to-end con RL sobre perfiles de enfermedad, historiales y 27M de documentos biomédicos (Zheng et al., 2025). El agente aprende a formular consultas, evaluar evidencias y refinar búsquedas para cerrar brechas diagnósticas, superando ampliamente marcos RAG de ingeniería de prompts. En validación con 150 casos reales, mejora la precisión diagnóstica de médicos de 45.6% a 69.1% (Zheng et al., 2025).

- **AI-VaxGuide:** sistema de preguntas y respuestas agéntico para decisiones de vacunación que transforma guías extensas en un sistema interactivo multilingüe. Agentic-RAG resulta especialmente superior frente a métodos tradicionales en preguntas multi-paso o ambiguas (Zeggai et al., 2025).

4.5.3 Recomendadores y personalización

ARAG demuestra un marco Agentic-RAG multi-agente para recomendación personalizada, donde agentes dedicados entienden al usuario, validan alineación semántica y ordenan ítems (Yousefi Maragheh et al., 2025). En tres conjuntos de datos de recomendación, ARAG supera RAG estándar y baselines por recencia, con mejoras de hasta 42.1% en NDCG@5 y 35.5% en Hit@5. El diseño agentic permite capturar comportamientos de sesión y largo plazo de forma más rica que heurísticas estáticas (Yousefi Maragheh et al., 2025).

4.5.4 Pruebas de software y entornos empresariales

- **Agentic-RAG para pruebas de software:** un enfoque que combina agentes autónomos con un sistema híbrido vector-grafo para automatizar generación de planes de prueba, casos y métricas de QE, usando modelos como Gemini y Mistral (Hariharan et al., 2025). Reporta mejoras de precisión de 65% a 94.8%, reducción del 85% del cronograma de pruebas y ahorros de coste del 35% en proyectos de migración SAP y sistemas corporativos (Hariharan et al., 2025).

- **Resolución de problemas empresarial:** trabajos sobre RAG agéntico ponderado para soporte técnico describen agentes que priorizan dinámicamente fuentes como

manuales, bases de conocimiento internas y guías de resolución de problemas según el contexto, mejorando tiempo de resolución (Khanda, 2024).

4.5.5 Dominios regulados y de alto riesgo

El marco agéntico para preguntas y respuestas islámicas (IslamicFaithQA) muestra cómo Agentic-RAG puede reducir alucinaciones y mejorar la capacidad de abstención en dominios donde respuestas incorrectas tienen consecuencias graves (Bhatia et al., 2026). El sistema incluye 25.000 pares de razonamiento SFT[16], 5.000 muestras de preferencia y un corpus de versículos; el agente llama herramientas estructuradas para buscar evidencia y revisa iterativamente las respuestas. Los resultados muestran que Agentic-RAG supera RAG estándar en corrección y robustez árabe-inglés (Bhatia et al., 2026).

De forma análoga, IBM y otros proveedores destacan Agentic-RAG como patrón recomendado para dominios donde se requieran respuestas explicables, auditables y actualizables sobre bases de conocimiento en constante cambio (Achanta, 2025; Belcic & Stryker, 2025; Microsoft, 2026).

4.5.6 Investigación académica y análisis político

Trabajos sobre LLMs como asistentes de investigación política describen Agentic-RAG como mecanismo central para dotar a los modelos de capacidades de acción sobre funciones y herramientas, desde recopilación automatizada de datos hasta codificación de transcripciones y modelado estadístico (Loffredo & Yun, 2025). El agente no solo recupera documentos, sino que también invoca herramientas para resumir, clasificar y cuantificar información, operando sobre flujos de trabajo multi-paso típicos de investigación en ciencias sociales (Loffredo & Yun, 2025).

4.5.7 Benchmarks y plataformas de evaluación

Para medir de forma sistemática Agentic-RAG, se han propuesto:

- **RAG-Gym:** entorno para optimizar agentes de búsqueda y razonamiento con supervisión de proceso, evaluando costes y calidad de forma uniforme (Xiong et al., 2025).
- **ARC (Agentic RAG Capabilities):** métrica fina que descompone capacidades agentic en razonamiento, coordinación de búsqueda y síntesis de respuesta, usada para estudiar cómo modelos compactos preservan comportamientos agentic mediante RL con distillation-guided policy optimization (Kotoge et al., 2025).

Estos recursos son clave para comparar arquitecturas y políticas de Agentic-RAG de forma rigurosa más allá de casos de uso ad hoc.

[16]SFT: *Supervised Fine-Tuning* (Ajuste Fino Supervisado), técnica de entrenamiento donde un modelo pre-entrenado se ajusta con pares de entrada-salida etiquetados, empleada en la primera etapa del pipeline de InstructGPT (Ouyang et al., 2022).

4.6 Consideraciones prácticas para ingeniería

4.6.1 Diseño de arquitecturas en producción

Las guías de IBM, Weaviate, Microsoft y Google Cloud coinciden en varias recomendaciones prácticas (Achanta, 2025; Belcic & Stryker, 2025; Kolesnikov, 2026; Microsoft, 2026; Shorten & Monigatti, 2024):

- Empezar por un RAG bien instrumentado y evolucionar a Agentic-RAG donde haya puntos críticos evidentes (multi-hop, heterogeneidad de fuentes, requisitos de auditoría).
- Modelar explícitamente el flujo de trabajo como grafo o FSM (por ejemplo, LangGraph, StateFlow, orquestadores propios) para controlar costes y trazabilidad.
- Limitar el espacio de herramientas y definir contratos claros (inputs/outputs tipados) para reducir errores de invocación de herramientas.
- Introducir agentes evaluadores (LLM como juez, reglas, métricas) que puedan rechazar respuestas de baja calidad y gatillar bucles de corrección.

4.6.2 Compromisos clave

Los estudios comparativos resaltan varios trade-offs (Ferrazzi et al., 2026; Kotoge et al., 2025; Mishra et al., 2026; Xiong et al., 2025):

- **Calidad vs. coste:** más pasos agentic implican más tokens y latencia; conviene adoptar políticas adaptativas que escalen el esfuerzo con la dificultad de la consulta.
- **Autonomía vs. control:** dar demasiada libertad al agente puede aumentar riesgos de alucinación compuesta, *uso indebido de herramientas* y comportamiento impredecible; por ello se recomiendan políticas con límites de profundidad y herramientas, además de supervisores (Mishra et al., 2026).
- **Complejidad de ingeniería:** multi-agente y multi-herramienta simplifican extensibilidad pero aumentan coste de observabilidad, testing y debugging.

4.6.3 Direcciones abiertas de investigación

Mishra et al. (2026) y trabajos recientes identifican varias líneas abiertas (Chen et al., 2026; Kotoge et al., 2025; Tian et al., 2025; Zhang et al., 2026):

- **Retrieval adaptativo y estable:** políticas que ajusten dinámicamente intensidad y tipo de búsqueda según señales de calidad de query y de resultados (Tian et al., 2025).
- **Orquestación consciente de costos:** optimizar políticas de Agentic-RAG bajo restricciones de coste/latencia, quizá con RL multi-objetivo.
- **Evaluación de trayectorias:** métricas y benchmarks que evalúen no solo la respuesta final, sino la calidad del camino seguido (relevancia de evidencias, robustez a ruido, etc.) (Mishra et al., 2026; Xiong et al., 2025).

- **Seguridad y robustez:** mitigación de riesgos como contaminación de memoria, propagación de alucinaciones, vulnerabilidades en ejecución de herramientas y ataques adversarios sobre el pipeline (Mishra et al., 2026).

Estas líneas son altamente relevantes para trabajos de tesis que busquen unir aportes teóricos (por ejemplo, formulación POMDP[17], garantías de convergencia) con ingeniería de sistemas (por ejemplo, frameworks de orquestación, métricas operativas).

4.7 LLM-as-a-judge

La evaluación automática de sistemas de lenguaje natural ha sido históricamente un desafío. Las métricas tradicionales como BLEU (Papineni et al., 2002) y ROUGE (Lin, 2004) miden calidad mediante solapamiento léxico entre la salida y un texto de referencia, pero fallan en escenarios abiertos y dinámicos donde múltiples respuestas pueden ser igualmente válidas (Liu et al., 2016). Con el auge de los modelos de lenguaje grandes (LLMs), surgió el paradigma *LLM-as-a-judge*, donde un LLM actúa como evaluador, asignando puntuaciones, produciendo rankings o seleccionando la mejor opción entre candidatos (Li et al., 2024a; Zheng et al., 2023).

4.7.1 Definición y fundamentos

Formalmente, el proceso de evaluación con LLM-as-a-judge se puede definir como $R = J(C_1, ..., C_n)$, donde J es el LLM juez, C_i son los candidatos a evaluar y R es el resultado del juicio (Li et al., 2024a). Las modalidades de entrada se clasifican en:

- **Point-wise** ($n = 1$): el juez evalúa un único candidato de forma absoluta, asignando una puntuación numérica o categórica.
- **Pair-wise** ($n = 2$): el juez compara dos candidatos y determina cuál es superior.
- **List-wise** ($n > 2$): el juez ordena o selecciona entre múltiples candidatos.

En cuanto a la salida, el juez puede producir: (a) una **puntuación** numérica (continua o discreta), (b) un **ranking** de los candidatos, o (c) una **selección** del mejor candidato (Li et al., 2024a).

4.7.2 Taxonomía de atributos evaluables

La literatura identifica al menos seis atributos que LLM-as-a-judge puede evaluar de forma diferenciada (Li et al., 2024a):

- **Utilidad (Helpfulness)**: mide la utilidad e informatividad de una respuesta generada. Es el criterio más empleado en evaluación de asistentes conversacionales y alineamiento (Zheng et al., 2023).
- **Seguridad (Safety & Security)**: evalúa si el modelo evita generar contenido dañino, tóxico o sesgado. Se ha validado que LLMs pueden actuar como moderadores de contenido siguiendo guías de políticas (Bai et al., 2022).

[17]POMDP: *Partially Observable Markov Decision Process* (Proceso de Decisión de Markov Parcialmente Observable). Modelo matemático para problemas de decisión secuencial donde el agente no observa completamente el estado del entorno, sino que recibe observaciones parciales y debe mantener una creencia (distribución de probabilidad) sobre el estado real. En Agentic-RAG, se usa para formalizar el bucle de planificación-acción-observación donde el agente decide qué buscar basándose en evidencias parciales (Mishra et al., 2026).

- **Confiabilidad (Reliability)**: mide la fidelidad de la respuesta a fuentes verificables y la calibración de la incertidumbre expresada. Incluye la detección de alucinaciones (Min et al., 2023).
- **Relevancia (Relevance)**: evalúa qué tan bien se alinea la respuesta con la consulta o tarea original. Es particularmente importante en sistemas RAG (Li et al., 2024b).
- **Lógica (Logic)**: evalúa la coherencia interna y corrección del razonamiento, independientemente de la precisión factual (Yao et al., 2023b).
- **Calidad general (Overall Quality)**: evaluación holística que integra múltiples dimensiones en una puntuación compuesta (Lin & Chen, 2023).

4.7.3 Metodologías de prompting

El diseño de las instrucciones (*prompts*) para el LLM juez es crítico para la calidad de la evaluación. Las principales estrategias incluyen (Gu et al., 2024; Li et al., 2024a):

- **Aumento con reglas (Rule Augmentation)**: incorporar principios, rúbricas y criterios de evaluación directamente en el prompt. Por ejemplo, en Constitutional AI (Bai et al., 2022) se introduce una lista de principios (utilidad, inocuidad, honestidad) para guiar la comparación.
- **Operación de intercambio (Swapping Operation)**: para mitigar el sesgo de posición —donde el LLM favorece la primera o última opción presentada— se invoca al juez dos veces intercambiando el orden de los candidatos. Si los resultados son inconsistentes, se declara un empate (Zheng et al., 2023).
- **Demostraciones (In-Context Learning)**: proporcionar ejemplos concretos de evaluaciones correctas para que el LLM aprenda los estándares de evaluación (Jain et al., 2023).
- **Cadena de pensamiento (Chain-of-Thought)**: descomponer la evaluación en pasos intermedios, como en G-Eval (Liu et al., 2023a), donde el LLM primero genera una explicación y luego asigna una puntuación.
- **Colaboración multi-agente**: utilizar múltiples LLMs como jueces y agregar sus decisiones mediante votación, debate o ranking entre pares para reducir sesgos individuales (Chan et al., 2023; R. Li et al., 2023).
- **Interacción multi-turno**: el juez interactúa dinámicamente con el candidato, formulando preguntas de seguimiento para una evaluación más profunda (Bai et al., 2023).

4.7.4 Métodos de ajuste fino

Más allá del prompting, se han desarrollado LLMs especializados como jueces mediante ajuste fino supervisado (SFT) y aprendizaje por preferencias (Li et al., 2024a):

- **JudgeLM** (Zhu et al., 2023): ajusta modelos Vicuna con 100K pares de juicios generados por GPT-4, abordando sesgos mediante aumento de datos con intercambio.
- **Prometheus** (Kim et al., 2024a) y **Prometheus 2** (Kim et al., 2024b): modelos open-source especializados en evaluación fine-grained, capaces de evaluar según rúbricas personalizadas y soportar tanto evaluación directa como comparación por pares.

- **AUTO-J** (J. Li et al., 2023): entrenado con datos sintéticos de GPT-4 para generar evaluaciones tanto point-wise como pair-wise con explicaciones detalladas.
- **FLAMe** (Vu et al., 2024): modelo entrenado con más de 5 millones de juicios humanos en 100+ tareas, demostrando capacidad de generalización a tareas no vistas.
- **Self-Taught Evaluator** (T. Wang et al., 2024): utiliza aprendizaje iterativo con datos sintéticos para mejorar sin depender de anotaciones humanas.

4.7.5 Sesgos y vulnerabilidades

A pesar de su efectividad, LLM-as-a-judge presenta sesgos significativos que deben ser considerados (Gu et al., 2024; Li et al., 2024a):

- **Sesgo de posición (Position Bias)**: el LLM tiende a favorecer respuestas según su orden de presentación (P. Wang et al., 2024; Zheng et al., 2023).
- **Sesgo de verbosidad (Verbosity Bias)**: favorece respuestas más largas, equiparando extensión con calidad (Koo et al., 2023).
- **Sesgo egoísta (Egocentric Bias)**: el LLM juez favorece respuestas generadas por sí mismo (Liu et al., 2023b).
- **Sesgo de autoridad (Authority Bias)**: favorece afirmaciones que citan fuentes que parecen autoritarias (Chen et al., 2024).
- **Vulnerabilidad adversarial**: secuencias adversariales cuidadosamente diseñadas pueden manipular las puntuaciones del juez (Raina et al., 2024; Shi et al., 2024).

Para mitigar estos sesgos, se han propuesto técnicas como calibración mediante intercambio de posición (Zheng et al., 2023), uso de múltiples jueces (R. Li et al., 2023), y entrenamiento con datos debiased como OFFSETBIAS (Park et al., 2024).

4.7.6 LLM-as-a-judge en sistemas RAG

En el contexto de RAG, LLM-as-a-judge se ha convertido en el método de evaluación predominante, ya que permite evaluar dimensiones que las métricas tradicionales no capturan (Gu et al., 2024; H. Li et al., 2024):

- **RAGAS** (Es et al., 2024): framework que utiliza LLMs para evaluar sistemas RAG en tres dimensiones: relevancia del contexto, fidelidad de la respuesta y relevancia de la respuesta.
- **ARES** (Saad-Falcon et al., 2024): combina LLMs con clasificadores entrenados para evaluar RAG en las mismas tres dimensiones, utilizando PPI (*Prediction-Powered Inference*) para intervalos de confianza estadística.
- **Self-RAG** (Asai et al., 2023): el propio LLM genera *tokens reflectantes* que indican si necesita recuperación y evalúa la calidad de sus propias salidas, integrando el juez dentro del pipeline de generación.
- **RAG-Gym** (Xiong et al., 2025): entorno unificado para entrenar agentes de búsqueda y razonamiento con supervisión de proceso, donde LLMs avanzados actúan como jueces de trayectoria.
- **ConsJudge** (Liu et al., 2025): método que mejora la consistencia del juez LLM en RAG utilizando consistencia entre juicios (*judge-consistency*) para seleccionar pares de preferencia y entrenar con DPO.

4.7.7 Meta-evaluación de LLM-as-a-judge

La evaluación de los propios LLM jueces —meta-evaluación— es un área activa de investigación. Los benchmarks más relevantes incluyen (Li et al., 2024a):

- **MT-Bench y Chatbot Arena** (Zheng et al., 2023): el primero es un conjunto de preguntas multi-turno para evaluar la capacidad de los LLMs de seguir instrucciones; el segundo es una plataforma crowdsourced donde usuarios votan anónimamente entre respuestas de distintos modelos, generando rankings mediante puntuaciones Elo.
- **JudgeBench** (Tan et al., 2024): benchmark que evalúa LLM jueces en pares de respuestas desafiantes en dominios como conocimiento, razonamiento, matemáticas y código, con etiquetas de preferencia que reflejan corrección objetiva.
- **RewardBench** (Lambert et al., 2024): benchmark para evaluar modelos de recompensa en áreas como chat, razonamiento y seguridad.
- **LLM-as-a-Judge Meta-Evaluation** (Bavaresco et al., 2024): estudio a gran escala que evalúa la correlación entre LLM jueces y evaluadores humanos en 20 tareas de NLP, encontrando que la concordancia varía significativamente según la tarea y el modelo.

Las métricas de meta-evaluación más utilizadas son la correlación de Pearson, Spearman, Kendall's τ y el coeficiente Kappa de Cohen, que miden el grado de acuerdo entre el LLM juez y los evaluadores humanos (Li et al., 2024a). Soft Pairwise Accuracy (SPA) (Thompson et al., 2024) ha surgido como una métrica más robusta que asigna crédito parcial por rankings casi correctos.

4.7.8 Implicaciones para esta tesis

En esta tesis, LLM-as-a-judge se emplea como método principal de evaluación del sistema Agentic-RAG para consulta de normativa argentina. La elección responde a varias consideraciones:

1. Las respuestas del sistema son inherentemente abiertas y no existe una única respuesta correcta, lo que hace que métricas basadas en solapamiento léxico (BLEU, ROUGE) sean inadecuadas.
2. El dominio legal requiere evaluación de dimensiones como precisión factual, completitud y relevancia, que LLM-as-a-judge puede capturar mediante rúbricas personalizadas.
3. La escalabilidad del método permite evaluar sistemáticamente múltiples configuraciones del sistema sin el costo prohibitivo de anotadores humanos expertos.

Para mitigar los sesgos inherentes, se adoptan las siguientes estrategias: (a) uso de múltiples LLM jueces y agregación de resultados, y (b) prompts con rúbricas detalladas específicas del dominio legal que definan explícitamente los criterios de puntuación.

4.8 Síntesis del capítulo

Este capítulo presentó el marco teórico de Agentic-RAG, partiendo del concepto de RAG clásico y evolucionando hacia arquitecturas agénticas donde un planificador autónomo decide cuándo y cómo recuperar información. Se describieron las principales

arquitecturas —monolítica, multi-agente y jerárquica—, los patrones de flujo de trabajo reutilizables (encadenamiento, enrutamiento, paralelización, orquestador-trabajadores, evaluador-optimizador) y los agentes especializados que componen estos sistemas: planificadores, recuperadores, razonadores, evaluadores y agentes de dominio específico.

Se dedicó una sección al paradigma LLM-as-a-judge, que constituye el método de evaluación principal empleado en esta tesis. Se presentaron sus fundamentos formales, la taxonomía de atributos evaluables (utilidad, seguridad, confiabilidad, relevancia, lógica y calidad general), las metodologías de prompting y ajuste fino para construir jueces LLM, los sesgos y vulnerabilidades que afectan a estos evaluadores, y su aplicación específica en sistemas RAG mediante frameworks como RAGAS, ARES, Self-RAG y ConsJudge. Se discutieron también los benchmarks y métricas de meta-evaluación para validar la calidad de los propios jueces LLM.

Se analizaron casos de uso concretos en salud, recomendación, pruebas de software, dominios regulados e investigación académica, así como las consideraciones prácticas de ingeniería para llevar estos sistemas a producción. Finalmente, se identificaron direcciones abiertas de investigación, entre las que se destacan la orquestación consciente de costos, la evaluación de trayectorias y la seguridad y robustez de los pipelines agénticos.

El siguiente capítulo (Sección 5) describe la metodología empleada para diseñar, implementar y evaluar el sistema Agentic-RAG para consulta de normativa argentina, detallando el diseño experimental, las fases del proyecto y los criterios de selección de herramientas y métricas.

5 Metodología e Implementación

5.1 Introducción

Este capítulo describe la metodología empleada para diseñar, implementar y evaluar el sistema Agentic-RAG para consulta de normativa argentina. Se detalla el enfoque metodológico adoptado, las fases del proyecto, las decisiones de diseño, los criterios de selección de herramientas y modelos, así como los detalles técnicos de la implementación. La implementación se organizó en módulos independientes ejecutados como notebooks de Jupyter, lo que favorece la reproducibilidad y la iteración.

5.2 Enfoque Metodológico

5.2.1 Diseño de la Investigación

Los experimentos descritos en este capítulo fueron realizados durante el segundo semestre de 2025. Los modelos, versiones de librerías y servicios de API empleados corresponden a las versiones disponibles en dicho período. El campo de los grandes modelos de lenguaje se caracteriza por ciclos de lanzamiento cortos; los resultados deben interpretarse en ese contexto temporal.

La investigación sigue un enfoque experimental cuantitativo con diseño factorial en el que se comparan tres estrategias de generación de respuestas —sin recuperación (no RAG), RAG básico y Agentic-RAG— combinadas con tres modelos de lenguaje de la familia GPT-5 (nano, mini y estándar), lo que da lugar a nueve configuraciones experimentales. Este diseño permite evaluar tanto los efectos principales de cada factor como su interacción, tal como se resume en la Tabla 3.

Estrategia	**gpt-5-nano**	**gpt-5-mini**	**gpt-5**
Sin RAG	✓	✓	✓
RAG básico	✓	✓	✓
Agentic-RAG	✓	✓	✓

Tabla 3: Diseño factorial 3 × 3 del experimento. Cada celda marcada representa una configuración experimental evaluada.

5.2.2 Fases del Proyecto

El proyecto se estructura en las siguientes fases:

1. **Extracción de datos**: descarga de textos legales, conversión de texto original a texto estructurado y normalización del corpus.
2. **Generación de preguntas**: creación de un benchmark de preguntas con respuestas esperadas a partir de los títulos del Código de Minería.
3. **Base de datos vectorial**: diseño del esquema, generación de embeddings y almacenamiento en Milvus.
4. **Generación de respuestas**: consulta a los LLM bajo las tres estrategias de recuperación.

5. **Evaluación**: cálculo de métricas de similitud textual, semántica y evaluación por jueces LLM.
6. **Análisis estadístico**: aplicación de pruebas de normalidad, comparaciones no paramétricas (Kruskal-Wallis, Dunn), análisis factorial (Scheirer-Ray-Hare) y análisis de acuerdo inter-juez (Krippendorff's alpha).

5.3 Selección y Extracción del Corpus

5.3.1 Características del Código de Minería Argentino

Se seleccionó el Código de Minería de la Nación Argentina como corpus de prueba de concepto. Este cuerpo normativo presenta una estructura jerárquica definida (títulos, capítulos, secciones y artículos) que facilita la segmentación y vectorización. Además, su extensión es acotada (comparada con la totalidad de la legislación nacional), lo que permite iterar sobre el diseño del sistema sin incurrir en costos prohibitivos de procesamiento.

5.3.2 Obtención del Texto Legal

El Código de Minería fue descargado manualmente desde el sitio del Boletín Oficial de la República Argentina (www.boletinoficial.gob.ar) en formato PDF. Este documento constituye la fuente primaria del corpus.

Se exploraron dos estrategias adicionales de descarga desde el sistema Infoleg del Ministerio de Justicia de la Nación, que provee los textos legislativos en formato HTML:

1. **Docling**: librería especializada en descarga y conversión de documentos legales a formato Markdown.
2. **Requests + BeautifulSoup**: enfoque alternativo para descarga y parsing de HTML con extracción de metadatos estructurados (artículos individuales, fechas de sanción, anexos).

Ambas estrategias basadas en Infoleg resultaron en documentos incompletos o con errores de parsing, por lo que se descartaron en favor del procesamiento directo del PDF oficial. Ambos enfoques se implementaron como notebooks independientes, permitiendo contrastar y validar los resultados de extracción.

5.3.3 Extracción de Texto mediante Modelos de Visión

El texto del PDF fue extraído mediante un pipeline automatizado que convierte cada página del PDF a imagen y procesa las imágenes mediante el modelo de visión o4-mini de OpenAI. Este enfoque permite preservar la estructura jerárquica del documento legal (títulos, capítulos, artículos) y manejar documentos escaneados que carecen de una capa de texto digital.

El pipeline realiza las siguientes tareas:

1. Convierte cada página del PDF a formato PNG mediante la librería `pdf2image`.
2. Envía cada imagen al modelo o4-mini de OpenAI para extraer el texto estructurado.
3. Almacena los resultados en una base de datos SQLite para su posterior procesamiento y para evitar llamadas redundantes a la API.

Se desarrolló un módulo específico de parsing (`02_html_infoleg_parser`) que transforma el HTML original en texto estructurado, preservando los metadatos de cada artículo (número, título, capítulo, sección).

5.3.4 Validación de Calidad de Extracción

La calidad de la extracción se verificó mediante inspección manual de una muestra de artículos, comparando el texto extraído con el original en Infoleg. Se establecieron criterios de calidad basados en la completitud del texto, la preservación de la estructura jerárquica y la correcta identificación de los límites entre artículos.

5.3.5 Segmentación y Estructuración del Corpus

El Código de Minería fue segmentado en títulos individuales, y el texto completo fue dividido en 22 archivos Markdown correspondientes a cada título (Título I al Título XXII), más el apéndice y el título final. Esta segmentación por título constituye la base para la generación del benchmark de preguntas.

Posteriormente, para la indexación en la base de datos vectorial, el corpus fue dividido en unidades siguiendo su estructura jerárquica natural. Cada artículo se almacena como una unidad independiente, preservando metadatos como el título, capítulo y sección a los que pertenece, lo que permite tanto la recuperación granular como la contextualización de los fragmentos recuperados.

El corpus resultante está compuesto por un total de 482 partes distribuidas según su tipo en la Tabla 4.

Tipo de parte	**Cantidad**
Títulos	22
Secciones	6
Acápites	55
Artículos	398
Apéndice	1
Total	**482**

Tabla 4: Estadísticas del corpus del Código de Minería Argentino. Desglose por tipo de parte estructural.

El Código de Minería se organiza en 22 títulos que abarcan desde el dominio de las minas y su clasificación (Título I) hasta disposiciones sobre la caducidad de las concesiones (Título XXII). Los 398 artículos constituyen las unidades normativas fundamentales, cada uno con contenido jurídico específico. Los 55 acápites agrupan artículos bajo subtemas dentro de cada título, mientras que las 6 secciones y el apéndice completan la estructura jerárquica del cuerpo legal. Esta organización jerárquica facilita la segmentación del corpus en fragmentos semánticamente coherentes para su indexación y recuperación en el sistema RAG.

5.4 Base de Datos Vectorial

5.4.1 Selección de la Base de Datos Vectorial

Se seleccionó Milvus (versión 2.5.9) como base de datos vectorial por su capacidad de escalar a grandes volúmenes, soporte para múltiples tipos de índices y su disponibilidad como software de código abierto (Wang et al., 2021) (véase el capítulo Sección 4 para los fundamentos de bases de datos vectoriales en sistemas RAG). La instancia se despliega mediante el script de instalación oficial de Milvus, que genera automáticamente la configuración de Docker Compose con los servicios de Milvus standalone y etcd como almacén de metadatos, lo que permite reproducir el entorno con un único comando.

5.4.2 Esquema de Datos

El esquema de la colección en Milvus contempla campos para:

1. Identificador único del fragmento.
2. Texto original del artículo o sección.
3. Metadatos estructurales (título, capítulo, sección, número de artículo).
4. Vector de embeddings (768 dimensiones).

La Tabla 5 ilustra la estructura de la colección en Milvus.

Campo	Tipo	Descripción
`id`	`INT64`	Identificador único del fragmento
`contenido_texto`	`STRING`	Texto original del artículo o sección
`titulo`	`STRING`	Título del código al que pertenece
`capitulo`	`STRING`	Capítulo dentro del título
`seccion`	`STRING`	Sección dentro del capítulo
`numero_articulo`	`INT64`	Número del artículo
`vector_contenido_texto`	`FLOAT_VECTOR[768]`	Embedding semántico de 768 dimensiones

Tabla 5: Esquema de la colección en Milvus. Cada artículo se almacena con su texto, metadatos estructurales y vector de embeddings de 768 dimensiones.

5.4.3 Creación de Índices

Se creó un índice de tipo IVF_FLAT sobre el campo de embeddings, que ofrece un balance entre velocidad de búsqueda y precisión de los resultados. La métrica de distancia empleada fue la similitud coseno (COSINE), adecuada para comparar representaciones semánticas.

5.4.4 Estrategia de Chunking

Se adoptó una estrategia de segmentación basada en la estructura natural del documento legal. Cada artículo constituye una unidad de chunking, lo que respeta los límites semánticos del texto jurídico y evita la fragmentación arbitraria que podría separar información contextualmente vinculada.

La Figura 14 ilustra el proceso de generación de embeddings e inserción en la base de datos vectorial.

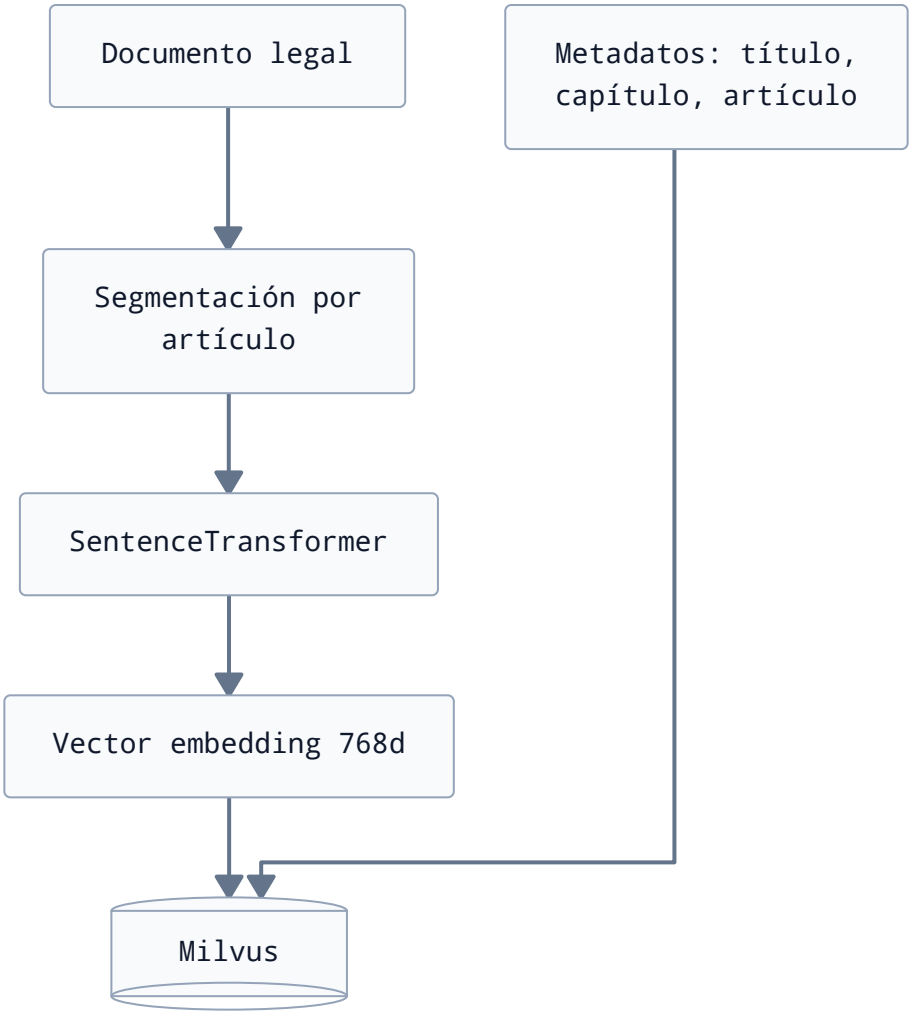


Figura 14: Diagrama del flujo de generación de embeddings e inserción en la base de datos vectorial Milvus.

5.5 Modelo de Embeddings

5.5.1 Criterios de Selección

Los criterios para la selección del modelo de embeddings fueron:

1. Soporte para el idioma español.
2. Desempeño en tareas de similitud semántica.
3. Dimensionalidad de los vectores resultantes.
4. Disponibilidad como modelo de código abierto.

5.5.2 Modelo Seleccionado

Se seleccionó el modelo `hiiamsid/sentence_similarity_spanish_es` (Siddhartha, 2023), un modelo de la familia sentence-transformers especializado en similitud semántica para textos en español (Reimers & Gurevych, 2019). Genera vectores de 768 dimensiones, ofreciendo un balance adecuado entre capacidad representativa y eficiencia computacio-

nal (véase el capítulo Sección 2 para los fundamentos de representaciones vectoriales y embeddings).

5.5.3 Generación e Inserción de Embeddings

Los embeddings se generaron procesando cada fragmento de texto mediante el modelo `SentenceTransformer('hiiamsid/sentence_similarity_spanish_es')` cargado desde Hugging Face. La vectorización del corpus se realizó por lotes (batches) para optimizar el uso de memoria y CPU. Cada lote procesa un conjunto de artículos, genera los embeddings correspondientes y los prepara para la inserción en la base de datos.

Los vectores generados se insertaron en la colección de Milvus junto con sus metadatos textuales y estructurales. Se realizó una verificación post-inserción mediante consultas de prueba para confirmar la correcta indexación y recuperabilidad de los documentos. Los fragmentos sin contenido textual fueron representados con un vector nulo para mantener la consistencia del índice.

La generación de embeddings se realizó sobre una máquina con procesador x86_64 de 16 núcleos y 32 GB de RAM, ejecutando el modelo en CPU. El tiempo total de procesamiento para los 398 artículos del corpus fue del orden de minutos, dado el tamaño acotado del conjunto de datos.

5.6 Sistema RAG Básico

El sistema RAG básico constituye una de las configuraciones de referencia del diseño experimental. A diferencia de la respuesta directa sin recuperación, incorpora una etapa de búsqueda vectorial para proveer contexto legal al modelo.

5.6.1 Módulo de Recuperación

El módulo de recuperación del RAG básico realiza una búsqueda vectorial por similitud coseno en Milvus, retornando los top-10 documentos más relevantes para cada consulta. Los documentos recuperados incluyen su texto completo y metadatos estructurales (título, capítulo, sección, número de artículo).

5.6.2 Módulo de Generación

Los fragmentos recuperados se inyectan en el prompt del sistema del LLM como contexto. El prompt instruye al modelo para responder basándose exclusivamente en la información proporcionada y citar los artículos relevantes. Se emplea la misma configuración de agente (Pydantic AI con temperatura 0) que en el baseline sin RAG, para mantener la comparabilidad.

5.6.3 Pipeline Completo

El pipeline completo del RAG básico sigue la secuencia: recepción de la pregunta → búsqueda vectorial → inyección de contexto en el prompt → generación de respuesta por el LLM. Se registran los metadatos completos de cada respuesta (tokens consumidos, costo, fragmentos recuperados, tiempos de ejecución).

5.7 Sistema Agentic-RAG

5.7.1 Arquitectura General

El sistema Agentic-RAG implementa una arquitectura multi-agente compuesta por dos agentes especializados que coordinan las tareas de recuperación y generación de respuestas: el Agente Especialista Legal y el Agente de Recuperación. Durante el diseño se contempló un tercer agente —el Agente de Calidad de Recuperación— encargado de evaluar la pertinencia de los fragmentos recuperados, pero fue desactivado en la configuración experimental final debido a que su complejidad adicional no se tradujo en mejoras marginales significativas respecto al pipeline de dos agentes. Esta arquitectura se compara con dos configuraciones de referencia: respuesta directa sin recuperación (no RAG) y RAG básico con recuperación vectorial simple.

La Figura 15 presenta el diagrama de secuencia de la configuración sin RAG.

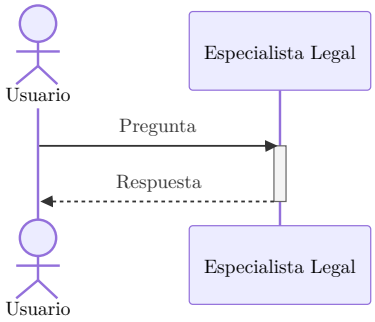


Figura 15: Configuración *Sin RAG*: el modelo genera la respuesta directamente a partir de su conocimiento paramétrico, sin acceso a documentos externos.

5.7.2 Framework de Orquestación (Pydantic AI)

La implementación de los agentes se realizó mediante el framework Pydantic AI (Colvin & others, 2024), cuya selección se fundamenta en los criterios presentados en el capítulo Sección 4 sobre frameworks de orquestación de agentes. Pydantic AI provee:

1. Definición declarativa de agentes con validación de tipos.
2. Gestión de herramientas (tools) con límites de invocación configurables.
3. Soporte para salidas estructuradas mediante modelos Pydantic.
4. Compatibilidad nativa con múltiples proveedores de LLM.
5. Ejecución asíncrona para optimizar las llamadas concurrentes a APIs.

Cada agente se define como una instancia con su propio prompt del sistema, conjunto de herramientas y restricciones de ejecución. Se habilitó la ejecución asíncrona para optimizar las llamadas concurrentes a las APIs de los modelos.

5.7.3 Agente Especialista Legal

El Agente Especialista Legal es el componente principal responsable de generar la respuesta final al usuario. Recibe la pregunta del usuario y, en la configuración Agentic-RAG, tiene acceso a herramientas que le permiten solicitar búsquedas al Agente de Recuperación. Su prompt del sistema está diseñado para orientar las respuestas hacia el dominio legal argentino:

> *Eres un asistente legal experto en derecho Argentino. Respondes preguntas legales basadas en la Constitución Nacional Argentina, Códigos, Leyes y jurisprudencia. Analizas la pregunta del usuario, extraes los puntos importantes y utilizas la herramienta* `retrieve_relevant_documents` *una vez para obtener información de la base de datos legislativa. Analizas los documentos para generar la respuesta final. Siempre citas las fuentes legales consultadas. Si no sabes la respuesta y los documentos legales no son suficientes para crear una respuesta, di que no cuentas con información para responderla. Las respuestas deben ser claras, concisas y directas al punto. Respondes en español al menos que el usuario solicite otro idioma.*

El agente tiene límites configurables de invocaciones de herramientas para controlar costos y evitar ciclos infinitos.

La Figura 16 presenta el diagrama de secuencia de la configuración RAG básico, donde la consulta se emplea directamente para recuperar fragmentos de la base vectorial.

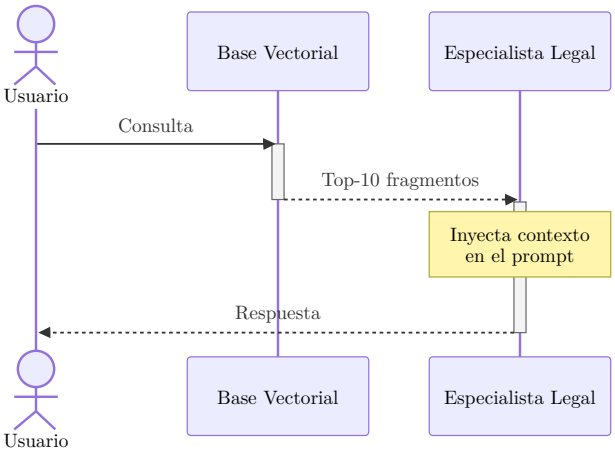


Figura 16: Configuración *RAG Básico*: la consulta del usuario se emplea para recuperar los 10 fragmentos más similares de la base vectorial, que se inyectan como contexto en el prompt del modelo.

5.7.4 Agente de Recuperación

El Agente de Recuperación se encarga de realizar búsquedas semánticas en la base de datos vectorial. Tiene la capacidad de descomponer consultas complejas en subconsultas más específicas, realizar múltiples búsquedas y consolidar los resultados. Retorna los fragmentos más relevantes junto con sus metadatos estructurales (artículo, capítulo, título). Su prompt del sistema es el siguiente:

> *Eres un especialista en búsqueda de información legislativa que: recibes una consulta relacionada con cuestiones legales de derecho Argentino; la consulta puede dividirse en máximo 3 subconsultas para mejorar la recuperación; realizas una búsqueda por cada subconsulta en una base de datos de vectores empleando la herramienta* `cosine_distance_vector_search` *como máximo 6 veces; seleccionas los documentos que mejor se ajustan a la consulta inicial y respondes con los resultados de la búsqueda.*

Sus capacidades incluyen:

1. Descomposición de consultas complejas en subconsultas semánticamente más específicas.
2. Ejecución de múltiples búsquedas vectoriales en paralelo.

3. Consolidación y deduplicación de resultados.
4. Retorno de fragmentos con metadatos completos (artículo, capítulo, título, distancia de similitud).

La Figura 17 presenta el diagrama de secuencia de la configuración Agentic-RAG, ilustrando la interacción entre ambos agentes.

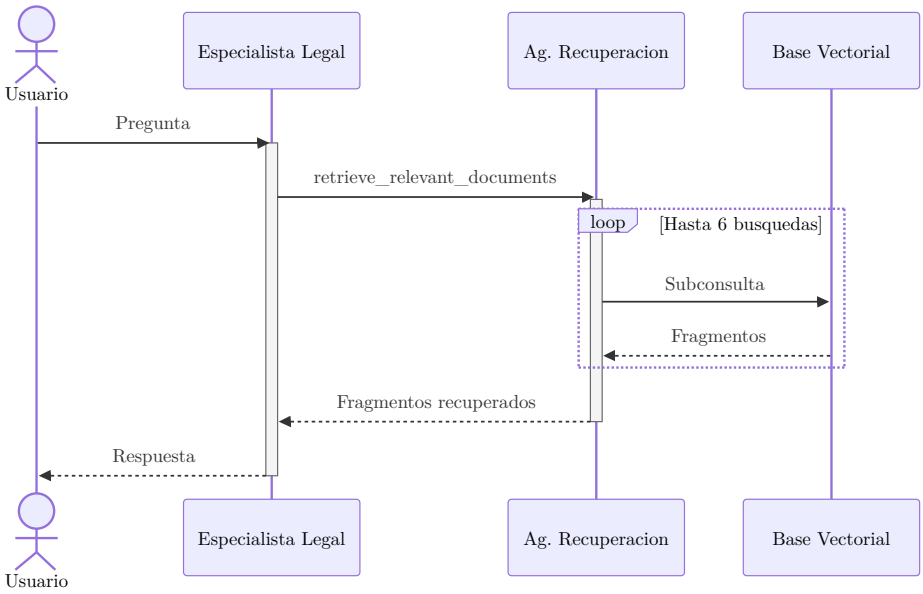


Figura 17: Configuración *Agentic-RAG*: el Agente Especialista Legal delega la recuperación al Agente de Recuperación, que descompone la consulta en subconsultas y ejecuta múltiples búsquedas vectoriales (bucle). Los fragmentos recuperados se retornan directamente al Agente Especialista Legal para la generación de la respuesta.

5.7.5 Agente de Calidad de Recuperación (Diseñado pero no Activado)

El Agente de Calidad de Recuperación fue diseñado para evaluar la pertinencia de los fragmentos recuperados antes de enviarlos al Agente Especialista Legal. Su función prevista era verificar que los documentos recuperados fueran relevantes para la consulta y solicitar búsquedas adicionales si la calidad de la recuperación inicial era insuficiente. Sin embargo, este agente fue desactivado en la configuración experimental final por dos motivos principales: (1) agregaba complejidad significativa en la gestión de llamadas a APIs, incrementando los costos operativos al requerir una capa adicional de invocación al LLM; y (2) requería la selección de un modelo adicional, introduciendo otra variable de configuración. En su lugar, se optó por actualizar los prompts del Agente Especialista Legal para que este pudiera solicitar búsquedas adicionales al Agente de Recuperación en caso de considerar que la información recuperada inicialmente era insuficiente, logrando un equilibrio entre calidad de recuperación y eficiencia. Su prompt del sistema era el siguiente:

> *Eres un especialista en «retrieval augmented generation» encargado de evaluar la calidad de la recuperación. Comparas la consulta original con la información recuperada y decides si la información es suficiente para responder la consulta. Provees feedback conciso para mejorar la consulta.*

En el código de orquestación, su herramienta (`evaluate_retrieval`) y su prompt del sistema fueron comentados, por lo que no participó en la generación de los resultados reportados.

5.7.6 Orquestación y Flujo de Comunicación

La orquestación se implementó como un flujo secuencial donde el Agente Especialista Legal invoca herramientas que internamente disparan la ejecución del Agente de Recuperación. Pydantic AI gestiona automáticamente el ciclo de herramientas (tool call → resultado → siguiente invocación) hasta que el agente principal produce la respuesta final o se alcanza el límite de herramientas. El flujo de comunicación sigue la siguiente secuencia:

1. El usuario formula una pregunta.
2. El Agente Especialista Legal recibe la consulta e invoca al Agente de Recuperación.
3. El Agente de Recuperación ejecuta búsquedas vectoriales, potencialmente descomponiendo la consulta en subconsultas.
4. Los fragmentos recuperados se retornan al Agente Especialista Legal.
5. El Agente Especialista Legal genera la respuesta final integrando el contexto recuperado.

Para las configuraciones sin RAG y RAG básico, se empleó un prompt simplificado que instruye al modelo como asistente legal experto en derecho argentino, con indicación de citar fuentes legales y responder de manera concisa. En la configuración RAG básico, el prompt incluye además una sección con los fragmentos recuperados de la base de datos vectorial inyectados como contexto.

5.7.7 Logging y Trazabilidad

Se implementó un sistema de logging que registra la conversación completa entre agentes, incluyendo:

1. Todas las invocaciones de herramientas con sus parámetros y resultados.
2. Los fragmentos recuperados en cada iteración de búsqueda.
3. Los metadatos de tokens consumidos y costos asociados a cada llamada a la API.

Los logs se almacenan en formato JSON para facilitar el análisis posterior del comportamiento del sistema. Nota: las decisiones del Agente de Calidad no aparecen en los logs de la configuración experimental final, dado que este agente fue desactivado.

5.7.8 Definición de Agentes y Herramientas

A continuación se presentan fragmentos representativos de la implementación de cada agente, incluyendo su instanciación, herramientas y mecanismos de control de ejecución. El código fuente completo se encuentra en el directorio `codigo_experimentos/part_g/`, y los logs detallados de cada interacción se almacenan en `codigo_experimentos/part_g/chat_data/`.

5.7.8.1 Agente Especialista Legal

```
law_specialist_agent = Agent(
    model=law_specialist_agent_model,
    instructions=INSTRUCTIONS["law_specialist_agent"],
    prepare_tools=cap_document_retrieval_tool_calls,
    tools=[retrieve_relevant_documents],
    model_settings=ModelSettings(temperature=0.0),
)
```

El prompt del sistema (`INSTRUCTIONS["law_specialist_agent"]`) fue presentado en la sección Sección 5.7.3. La herramienta `retrieve_relevant_documents` actúa como fachada que internamente invoca al Agente de Recuperación:

```
def retrieve_relevant_documents(question: str) -> dict:
    """Recupera documentos para responder la pregunta."""
    retrieval_deps = RetrievalDependencies(
        vector_db_client=get_db_client(),
    )
    result = retrieval_agent.run_sync(
        f"Please retrieve documents for: {question}",
        usage_limits=VECTOR_SEARCH_LIMIT,
        deps=retrieval_deps,
    )
    return result.output
```

El límite de una única invocación se impone mediante la función `cap_document_retrieval_tool_calls`, que deshabilita todas las herramientas después de la primera llamada.

5.7.8.2 Agente de Recuperación

```
retrieval_agent = Agent(
    model=retrieval_agent_model,
    instructions=INSTRUCTIONS["retrieval_agent"],
    prepare_tools=cap_total_tool_calls,
    tools=[cosine_distance_vector_search],
    model_settings=ModelSettings(temperature=0.0),
    deps_type=RetrievalDependencies,
)
```

La herramienta `cosine_distance_vector_search` ejecuta la búsqueda por similitud coseno contra Milvus:

```
def cosine_distance_vector_search(
    ctx: RunContext[RetrievalDependencies], query: str
) -> dict:
    """Busca documentos en la base vectorial
    por similitud coseno."""
    result = get_document_matches(
        ctx.deps.vector_db_client, query,
        max_documents=5
    )
    return {
        "usage_counts": vector_search_counter,
        "search_result": result
    }
```

La función `get_document_matches` codifica la consulta mediante `SentenceTransformer`, ejecuta `client.search()` sobre la colección con métrica `COSINE` y retorna los cinco documentos con mayor similitud junto con sus metadatos estructurales.

El límite global de 6 invocaciones se implementa mediante `cap_total_tool_calls`, que contabiliza todas las llamadas a herramientas en la sesión y deshabilita las herramientas al alcanzar el umbral. Esto permite que el agente descomponga la consulta en hasta 3 subconsultas con hasta 2 búsquedas cada una, o realice hasta 6 búsquedas con una única formulación.

5.7.9 Ejemplo de Interacción entre Agentes

Para ilustrar el flujo completo de comunicación, se presenta a continuación un ejemplo concreto extraído de los logs de ejecución correspondientes a la pregunta 1.1 del benchmark (modelo gpt-5-mini como Agente Especialista Legal, gpt-5 como Agente de Recuperación). Los logs completos de todas las interacciones se encuentran en `codigo_experimentos/part_g/chat_data/`.

Pregunta del usuario: «El ingeniero Juan Pérez ha descubierto un importante yacimiento de litio en una finca privada ubicada en territorio de la Provincia de Salta. El propietario de la finca, la Sra. Marta Gómez, sostiene que, por ser dueña del suelo, es también dueña exclusiva del litio allí existente y que nadie puede explotarlo sin su consentimiento. Juan Pérez pretende solicitar la concesión minera ante la autoridad competente y argumenta que el litio es de dominio estatal y que la propiedad del suelo es accesoria respecto de la mina.»

Paso 1 — Invocación del Agente de Recuperación: El Agente Especialista Legal recibe la pregunta y estructura una consulta para la herramienta `retrieve_relevant_documents` con los términos clave: «Código de Minería Argentino dominio de los minerales litio propiedad del suelo acceso a minas categoría del litio normas aplicables explotación concesión minera artículos pertinentes régimen entre Estado y propietarios privados».

Paso 2 — Descomposición en subconsultas: El Agente de Recuperación recibe la consulta y la descompone en tres subconsultas más específicas: (a) clasificación del litio y sustancias mineras, (b) dominio originario de las minas y relación con la propiedad del suelo, y (c) régimen de concesiones y adquisición de minas. Para cada subconsulta, invoca `cosine_distance_vector_search`.

Paso 3 — Búsqueda vectorial: Cada subconsulta se codifica mediante el modelo de embeddings y se ejecuta una búsqueda por similitud coseno en Milvus, retornando los 5 documentos más similares por subconsulta. Los artículos recuperados incluyen: Art. 3 (clasificación del litio en primera categoría), Art. 7 (dominio originario del Estado), Art. 8 (facultad de buscar minas), Art. 10 (concesión legal), Art. 11 (propiedad distinta del terreno), Art. 44 (adquisición por concesión) y Art. 351 (multiplicador de pertenencias para litio).

Paso 4 — Consolidación y respuesta: El Agente de Recuperación selecciona los 10 documentos más pertinentes y los retorna al Agente Especialista Legal. Este integra

el contexto recuperado y genera la respuesta final, citando los artículos 3, 7, 8, 10, 11, 44 y 351 del Código de Minería, y concluyendo que el dominio del litio corresponde a la Provincia de Salta, que el propietario del suelo no tiene derecho exclusivo sobre el mineral, y que Juan Pérez puede solicitar la concesión minera ante la autoridad competente.

Este ejemplo ilustra el flujo completo de delegación, descomposición de consultas, búsqueda vectorial múltiple e integración de contexto que caracteriza a la configuración Agentic-RAG. Los logs JSON de cada paso —incluyendo tokens consumidos, costo, documentos recuperados y sus distancias de similitud— están disponibles para su inspección en el directorio `codigo_experimentos/part_g/chat_data/`.

5.7.10 Hiperparámetros de Configuración

La temperatura controla la aleatoriedad de las respuestas del modelo (0,0 = determinista). El límite de llamadas a herramientas restringe cuántas veces cada agente puede invocar sus herramientas. La herramienta disponible especifica la función que cada agente puede ejecutar. El máximo de subconsultas indica cuántas preguntas derivadas puede generar el Agente de Recuperación a partir de una consulta original. El modelo por defecto define el LLM asignado a cada agente.

La Tabla 6 presenta los hiperparámetros de configuración de los agentes activos en la configuración Agentic-RAG experimental.

Parámetro	**Agente Especialista Legal**	**Agente de Recuperación**
Temperatura	0,0	0,0
Límite de llamadas a herramientas	1	6
Herramienta disponible	`retrieve_relevant_documents`	`cosine_distance_vector_search`
Máx. subconsultas por consulta	—	3
Modelo por defecto	Variable[†]	gpt-5

Tabla 6: Hiperparámetros de configuración de los agentes activos del sistema Agentic-RAG. [†]El modelo del Agente Especialista Legal varía según la configuración experimental (gpt-5-nano, gpt-5-mini o gpt-5).

El límite total de llamadas a herramientas del sistema está fijado en 6 invocaciones, lo que permite al Agente de Recuperación descomponer la consulta en hasta 3 subconsultas y realizar hasta 2 búsquedas por subconsulta, o realizar hasta 6 búsquedas con una única formulación.

5.8 Selección de Modelos LLM

5.8.1 Criterios de Selección

La selección de los modelos de lenguaje consideró:

1. Disponibilidad mediante API pública.
2. Costo por token de entrada y salida.

3. Desempeño en benchmarks generales de lenguaje.
4. Capacidad de seguir instrucciones complejas (instruction following).
5. Soporte para generación con temperatura cero (respuestas determinísticas).

5.8.2 Modelos Evaluados

Se evaluaron tres modelos de la familia GPT-5 de OpenAI, lo que permite analizar el efecto del tamaño del modelo manteniendo constante la arquitectura:

1. **gpt-5-nano**: modelo de menor tamaño y costo, orientado a tareas simples.
2. **gpt-5-mini**: modelo intermedio que balancea capacidad y costo.
3. **gpt-5**: modelo de mayor capacidad, referencia de desempeño superior.

Todos los modelos fueron configurados con temperatura 0 para maximizar la reproducibilidad de las respuestas.

5.8.3 Costos por Token

La Tabla 7 presenta los costos por token de los tres modelos empleados, según la tarificación vigente de OpenAI al momento de la experimentación (agosto-noviembre 2025).

Modelo	Entrada (USD/Mtok)	Salida (USD/Mtok)	Ratio vs. nano
gpt-5-nano	\$0,05	\$0,40	1×
gpt-5-mini	\$0,25	\$2,00	5×
gpt-5	\$1,25	\$10,00	25×

Tabla 7: Costos por millón de tokens (Mtok) de los modelos GPT-5 empleados. El ratio indica el factor de costo relativo al modelo más económico (gpt-5-nano).

La diferencia de costos entre modelos es significativa: gpt-5 es 25 veces más costoso que gpt-5-nano por token de entrada y salida. Esta variación permite analizar la relación costo-beneficio de cada configuración en el capítulo de evaluación. Cabe señalar que en la configuración Agentic-RAG, los agentes de Recuperación y Calidad emplean gpt-5 independientemente del modelo del Agente Especialista Legal, lo que incrementa el costo base de esta estrategia.

5.9 Diseño del Benchmark

5.9.1 Generación de Preguntas

Las preguntas del benchmark fueron generadas empleando GPT-4.1, a partir de cada título del Código de Minería. El proceso consistió en:

1. Segmentación del Código de Minería en títulos individuales (Título I, II, etc.).
2. Generación automatizada de preguntas tipo caso de estudio legal con respuestas esperadas para cada título.
3. Compilación de las preguntas en un documento unificado para revisión.

5.9.2 Estructura del Benchmark

El benchmark contiene 25 preguntas distribuidas entre los títulos del Código de Minería, que a su vez se descomponen en 79 subpreguntas que constituyen las unidades de evaluación independientes. Cada pregunta incluye:

1. Texto de la pregunta (formulada como caso de estudio legal).
2. Contexto relevante (artículos y secciones de referencia).
3. Respuesta esperada elaborada por el LLM generador.
4. Identificadores de los artículos fuente.

Las preguntas se normalizaron a un formato unificado donde cada subpregunta constituye una unidad de evaluación independiente, con su respectivo texto, contexto y respuesta esperada.

5.9.3 Colaboración con Profesionales del Derecho

Las preguntas generadas y sus respuestas esperadas fueron revisadas por profesionales del derecho con experiencia en legislación minera argentina, con el objetivo de validar la pertinencia, corrección y completitud de las respuestas de referencia. El proceso de validación se estructuró en las siguientes etapas:

1. **Revisión inicial**: los profesionales examinaron la totalidad de las preguntas generadas por GPT-4.1, verificando que los escenarios planteados fueran jurídicamente plausibles y que las subpreguntas abordaran aspectos relevantes de la normativa.
2. **Verificación de respuestas esperadas**: se contrastaron las respuestas de referencia con el texto vigente del Código de Minería, evaluando tres criterios: (a) correcta identificación de los artículos aplicables, (b) interpretación adecuada de las disposiciones normativas, y (c) coherencia entre la fundamentación y la conclusión jurídica.
3. **Ajustes y correcciones**: se realizaron correcciones en los casos donde la respuesta generada contenía imprecisiones en la citación de artículos, omisiones de disposiciones relevantes o interpretaciones ambiguas.
4. **Validación final**: las respuestas corregidas fueron revisadas por segunda vez para confirmar la consistencia del benchmark completo. Se verificó que las respuestas esperadas fueran autosuficientes y que los artículos de referencia citados correspondieran efectivamente a las disposiciones invocadas.

5.10 Métricas y Sistema de Evaluación

La evaluación del sistema emplea tres familias de métricas complementarias que capturan diferentes aspectos de la calidad de las respuestas.

5.10.1 Métricas de Similitud Textual

Se implementaron métricas de solapamiento léxico y a nivel de n-gramas mediante un módulo reutilizable (`03_compare_texts.py`) que calcula más de 11 métricas de similitud textual entre la respuesta generada y la respuesta esperada:

1. **Token F1**: media armónica de precisión y recall a nivel de tokens (Chinchor, 1992).
2. **Similitud de Jaccard**: proporción de tokens compartidos respecto al total de tokens únicos (Jaccard, 1901).
3. **Similitud de Levenshtein**: complemento normalizado de la distancia de edición (Levenshtein, 1966).

4. **BLEU** (1-4): precisión de n-gramas con penalización por brevedad (Papineni et al., 2002).
5. **ROUGE** (1, 2, L): recall de n-gramas y subsecuencia común más larga (Lin, 2004).
6. **Tasa de error de caracteres (CER)**: proporción de operaciones de edición a nivel de caracteres.
7. **Exact Match**: coincidencia exacta entre respuestas.
8. **Ratios de longitud y conteo de palabras**: métricas complementarias de formato.

5.10.2 Métricas Semánticas

Se emplearon métricas basadas en modelos de lenguaje preentrenados:

1. **SBERT Similarity (Spanish)**: similitud coseno entre embeddings generados por un modelo sentence-transformers optimizado para español (Reimers & Gurevych, 2019).
2. **SBERT Similarity (Multilingual)**: variante multilingüe del mismo enfoque (Reimers & Gurevych, 2020).
3. **BERTScore**: similitud contextualizada basada en alineamiento de tokens mediante embeddings BERT, reportando precisión, recall y F1 (Zhang et al., 2020).

5.10.3 Evaluación LLM-as-a-Judge

Complementariamente a las métricas automatizadas, se implementó un esquema de evaluación LLM-as-a-Judge (véase el capítulo Sección 4, sección Sección 4.7 para los fundamentos teóricos de esta técnica de evaluación) empleando la librería `pydantic_evals`, que permite definir rúbricas estructuradas y ejecutar evaluaciones de forma automatizada. Se definieron dos rúbricas de evaluación:

1. **Respuesta correcta** (`score_respuesta_correcta`): evalúa el grado de alineamiento de la respuesta con la respuesta esperada, en una escala de 0.0 a 1.0 con valores discretos {0.0, 0.25, 0.5, 0.75, 1.0}.
2. **Precisión legal** (`score_precision_legal`): evalúa la correcta citación de artículos del Código de Minería argentino, en la misma escala.

Se emplearon tres modelos como jueces, seleccionados para diversificar las arquitecturas y proveedores, lo que permite evaluar la confiabilidad de las evaluaciones mediante métricas de acuerdo inter-juez:

1. **DeepSeek-V3** (`openrouter:deepseek/deepseek-chat-v3.1`): modelo de mezcla de expertos (MoE) con 671B parámetros totales y 37B activados por token (DeepSeek-AI, 2024). No posee un modo de razonamiento nativo, pero puede seguir instrucciones de cadena de pensamiento mediante prompting estándar.
2. **Grok 4.1 Fast** (`openrouter:x-ai/grok-4.1-fast:free`): modelo con modo de razonamiento nativo que puede activarse mediante el parámetro `reasoning: enabled`, generando cadenas de pensamiento detalladas antes de producir la respuesta final (xAI, 2025).
3. **Kimi K2 0905** (`openrouter:moonshotai/kimi-k2-0905`): modelo de mezcla de expertos con 1T parámetros totales y 32B activados, optimizado para uso agentivo

(Kimi Team, 2025). Es un modelo *reflex-grade* sin modo de razonamiento nativo, aunque puede seguir instrucciones de cadena de pensamiento mediante prompting.

Los tres modelos fueron configurados con temperatura 0,0 y un límite de 5.000 tokens de salida. Los prompts de evaluación se construyeron a partir de las rúbricas definidas, solicitando al juez que asigne un puntaje y proporcione una justificación textual de la calificación mediante el parámetro `include_reason=True`. Cabe señalar que los prompts no incluyeron instrucciones explícitas de razonamiento paso a paso (cadena de pensamiento), lo que constituye una limitación discutida en Sección 8.3.2. Los resultados se almacenan como reportes JSON con scores y justificaciones por caso.

La Tabla 8 y la Tabla 9 presentan los criterios detallados de cada rúbrica.

Puntaje	**Criterio**
0,00	La respuesta está en blanco o no coincide en absoluto con la respuesta esperada.
0,25	La respuesta tiene poca relación con la respuesta esperada y contiene errores significativos.
0,50	La respuesta es parcialmente correcta, pero omite información clave o tiene errores menores.
0,75	La respuesta es mayormente correcta, con solo errores menores.
1,00	La respuesta es completamente correcta y precisa.

Tabla 8: Rúbrica de evaluación `respuesta_correcta`: «La respuesta provista está alineada con la respuesta esperada.» Evaluada en español.

Puntaje	**Criterio**
0,00	No cita ningún artículo relevante o cita artículos incorrectos.
0,25	Cita algunos artículos relevantes, pero omite otros importantes.
0,50	Cita la mayoría de los artículos relevantes y otros irrelevantes.
0,75	Cita casi todos los artículos relevantes con precisión y pocos irrelevantes.
1,00	Cita todos los artículos relevantes y ninguno irrelevante de manera precisa y completa.

Tabla 9: Rúbrica de evaluación `precision_legal`: «La respuesta cita correctamente los artículos relevantes del Código de Minería Argentino.» Evaluada en español.

5.11 Gestión de Costos y Tokens

5.11.1 Tracking de Uso

Cada llamada a la API de los proveedores de LLM registra los tokens de entrada y salida consumidos, así como el costo asociado. Esta información se almacena en los metadatos de cada respuesta y se consolida en un archivo CSV (`metadata_tokens_cost_cleaned.csv`) para el análisis agregado.

5.11.2 Optimización de Llamadas

Se implementaron las siguientes estrategias para optimizar el uso de la API y controlar costos:

1. Temperatura 0 para respuestas determinísticas (evita múltiples ejecuciones).
2. Límites de invocaciones de herramientas en los agentes.
3. Procesamiento secuencial con almacenamiento incremental de resultados para permitir la reanudación ante fallos.
4. Almacenamiento en caché de respuestas intermedias (embeddings, búsquedas vectoriales).

5.12 Herramientas y Tecnologías

5.12.1 Lenguajes y Frameworks

1. **Python**: lenguaje principal de desarrollo (Python Software Foundation, 2023).
2. **Pydantic AI**: framework para la implementación de agentes con validación de tipos (Colvin & others, 2024).
3. **pydantic_evals**: librería para evaluación LLM-as-a-Judge con rúbricas estructuradas (Colvin & others, 2024).
4. **Polars**: librería de procesamiento de datos tabulares de alto rendimiento (Polars Development Team, 2024).
5. **sentence-transformers**: generación de embeddings semánticos (Reimers & Gurevych, 2019).
6. **scipy**: análisis estadístico (Virtanen & others, 2020).
7. **matplotlib**: visualización de resultados (Hunter, 2007).

Las versiones exactas de todas las librerías empleadas, incluyendo dependencias transitivas, pueden consultarse en el archivo `codigo_experimentos/uv.lock`, que registra el estado completo del entorno Python gestionado mediante uv.

5.12.2 Infraestructura

1. **Milvus**: base de datos vectorial desplegada mediante Docker Compose.
2. **OpenAI API**: acceso a los modelos GPT-5 (nano, mini, estándar), GPT-4.1 para generación de preguntas y o4-mini para extracción de contenido de archivos PDF.
3. **APIs de terceros**: DeepSeek, Grok y Kimi K2 como modelos jueces.

5.12.3 Análisis Estadístico

El análisis estadístico de los resultados se estructuró en varias etapas, cada una con un propósito específico y respaldada por métodos estadísticos establecidos en la literatura.

5.12.3.1 Verificación de Normalidad (Shapiro-Wilk)

Previo a la selección de pruebas de hipótesis, se verificó la normalidad de las distribuciones de las métricas mediante el test de Shapiro-Wilk (Shapiro & Wilk, 1965). Este test evalúa la hipótesis nula de que la muestra proviene de una distribución normal. Si el valor p es inferior a 0,05, se rechaza la hipótesis de normalidad, lo que determina la necesidad de emplear métodos no paramétricos para los análisis subsiguientes.

5.12.3.2 Comparaciones No Paramétricas (Kruskal-Wallis y Dunn)

Para comparar las métricas entre los tres sistemas de recuperación (no RAG, RAG básico y Agentic-RAG) y entre los tres modelos (gpt-5-nano, gpt-5-mini, gpt-5), se empleó el test de Kruskal-Wallis (Kruskal & Wallis, 1952), una alternativa no paramétrica al ANOVA de un factor que no requiere el supuesto de normalidad. Este test evalúa si al menos uno de los grupos difiere significativamente de los demás.

Ante resultados significativos en Kruskal-Wallis, se aplicó el test post-hoc de Dunn (Dunn, 1964) con corrección de Bonferroni (Bonferroni, 1936) para identificar qué pares de grupos presentan diferencias significativas, controlando la tasa de error familiar (family-wise error rate).

Para los análisis con más de cinco comparaciones —en particular, los efectos simples de estrategia dentro de cada modelo (9 comparaciones)— se aplicó adicionalmente la corrección de Holm-Bonferroni (Holm, 1979), un procedimiento step-down secuencialmente rejectivo que ofrece mayor potencia estadística que Bonferroni manteniendo el control de la tasa de error familiar. El método de Holm ordena los p-values de menor a mayor y compara cada uno contra un umbral decreciente $\frac{\alpha}{m-k+1}$, donde m es el número total de comparaciones y k la posición en el ordenamiento. Ambas correcciones se implementaron mediante la función `multipletests` de la librería statsmodels (Seabold & Perktold, 2010).

5.12.3.3 Análisis de Interacción Factorial (Scheirer-Ray-Hare)

Para evaluar la interacción entre los factores «sistema de recuperación» y «modelo» sobre las métricas de calidad, se empleó el test de Scheirer-Ray-Hare (Scheirer et al., 1976). Este test constituye la alternativa no paramétrica al ANOVA de dos factores y se basa en aplicar ANOVA sobre los rangos de los datos. Permite evaluar los efectos principales de cada factor y su interacción, siendo particularmente útil cuando no se cumplen los supuestos de normalidad y homocedasticidad del ANOVA paramétrico.

5.12.3.4 Análisis de Acuerdo Inter-Juez (Krippendorff's Alpha)

Para evaluar la confiabilidad de las evaluaciones LLM-as-a-Judge, se calculó el coeficiente alpha de Krippendorff (Hayes & Krippendorff, 2007; Krippendorff, 2018) entre los tres modelos jueces (DeepSeek-V3, Grok 4.1 Fast y Kimi K2 0905). Este coeficiente mide el acuerdo entre múltiples codificadores (en este caso, modelos LLM actuando como jueces) corrigiendo el acuerdo esperado por azar, y es aplicable a cualquier escala de medición (nominal, ordinal, intervalo o razón). Se complementó con el análisis de correlación de Spearman (Spearman, 1904) entre pares de jueces para identificar posibles sesgos sistemáticos.

5.13 Desafíos Técnicos y Soluciones

5.13.1 Manejo de Errores y Reintentos

Se implementó un sistema de manejo de errores que registra las respuestas fallidas y permite su reprocesamiento selectivo. Los errores más frecuentes fueron timeouts de la

API y límites de tasa (rate limits), que se gestionaron mediante reintentos con backoff exponencial.

5.13.2 Optimización de Rendimiento

Se empleó ejecución asíncrona para las llamadas a APIs externas, lo que permite solapar tiempos de espera. La generación de embeddings se paralelizó por lotes y los resultados intermedios se persistieron en disco para evitar pérdida de progreso ante interrupciones.

5.14 Repositorio y Documentación

5.14.1 Estructura del Repositorio

El código se organizó en directorios temáticos que siguen el flujo del pipeline:

1. `notebooks/`: exploración y pruebas de tecnologías.
2. `part_a/`: generación de preguntas del benchmark.
3. `part_b/`: base de datos de preguntas del benchmark (enfoque cancelado; se optó por archivos JSON en su lugar).
4. `part_c/`: creación de la base de datos vectorial.
5. `part_d/`: pruebas de recuperación baseline.
6. `part_e/`: respuestas sin RAG y cálculo de métricas.
7. `part_f/`: respuestas con RAG básico.
8. `part_g/`: respuestas con Agentic-RAG.
9. `part_h/`: integración de datos.
10. `part_h_1/`: evaluación LLM-as-a-Judge.
11. `part_i/`: preparación de datos para análisis.
12. `part_j/`: análisis estadístico.

Los logs de interacción entre agentes correspondientes a la configuración Agentic-RAG se almacenan en `part_g/chat_data/`, organizados por modelo del Agente Especialista Legal y por número de pregunta. Cada sesión contiene archivos JSON independientes para el Agente Especialista Legal (`law_specialist_agent.json`) y el Agente de Recuperación (`retrieval_agent.json`), junto con el historial de búsquedas vectoriales (`retrieval_history/`). Los logs de las configuraciones sin RAG y RAG básico se encuentran en `part_e/` y `part_f/` respectivamente.

5.14.2 Documentación Técnica

Cada directorio contiene un archivo README con instrucciones de ejecución. Se mantiene un archivo de resumen de tareas (`performed_tasks_summary.md`) que documenta el propósito y resultado de cada notebook del proyecto.

5.15 Conclusiones del Capítulo

Este capítulo presentó el diseño metodológico y la implementación del sistema Agentic-RAG, abarcando todo el pipeline desde la extracción de datos del Código de Minería hasta el análisis estadístico de los resultados. Se detallaron las decisiones de diseño fundamentales sobre la selección del corpus, la arquitectura multi-agente, la estrategia de evaluación y las herramientas empleadas, así como los detalles técnicos de imple-

mentación incluyendo la configuración de la base de datos vectorial, la orquestación de agentes mediante Pydantic AI y el sistema de logging y trazabilidad.

El diseño factorial de 3 estrategias $\times$ 3 modelos permite un análisis riguroso de los efectos de cada factor y su interacción, mientras que la combinación de métricas textuales, semánticas y evaluación por jueces LLM proporciona una visión multidimensional de la calidad de las respuestas. El análisis estadístico detallado —que comprende pruebas de normalidad (Shapiro-Wilk), comparaciones no paramétricas (Kruskal-Wallis con post-hoc de Dunn), análisis de interacción factorial (Scheirer-Ray-Hare) y acuerdo inter-juez (Krippendorff's alpha)— proporciona un marco riguroso para la interpretación de los resultados experimentales.

La organización modular en notebooks independientes facilita la reproducibilidad y permite iterar sobre componentes específicos sin afectar al resto del sistema. El uso de frameworks como Pydantic AI para la orquestación de agentes y `pydantic_evals` para la evaluación automatizada simplifica la gestión de la complejidad inherente a un sistema multi-agente.

El siguiente capítulo (Sección 6) presenta los resultados de la evaluación experimental, incluyendo el análisis comparativo de las nueve configuraciones mediante métricas de similitud textual y semántica, la evaluación por jueces LLM, el análisis de acuerdo inter-juez y el estudio de costos. Los resultados se presentan mediante tablas resumen, gráficos de caja (boxplots) y mapas de calor (heatmaps) que facilitan la interpretación de los efectos principales y las interacciones entre factores.

6 Evaluación y Resultados

6.1 Introducción

Este capítulo presenta la evaluación sistemática del sistema Agentic-RAG y los resultados obtenidos. Se describen el diseño experimental, las métricas empleadas, los resultados de cada configuración y el análisis estadístico que fundamenta las conclusiones. El análisis sigue una progresión desde la exploración descriptiva hasta pruebas de hipótesis y análisis de interacción factorial.

6.2 Diseño Experimental

El diseño experimental sigue un esquema factorial 3 × 3 detallado en el capítulo Sección 5 (Tabla 3): tres estrategias de recuperación (sin RAG, RAG básico, Agentic-RAG) × tres modelos de lenguaje (gpt-5-nano, gpt-5-mini, gpt-5), con temperatura 0 en todas las configuraciones para garantizar reproducibilidad.

El benchmark de evaluación —descrito en Sección 5— está compuesto por 25 preguntas (76 subpreguntas explícitas más 3 preguntas evaluadas como unidad única, totalizando 79 unidades de evaluación × 9 configuraciones = 711 observaciones). Las métricas empleadas abarcan tres familias complementarias presentadas en dicho capítulo: solapamiento léxico (Token F1, Jaccard, Levenshtein, BLEU, ROUGE, CER), similitud semántica (SBERT Similarity, BERTScore) y evaluación LLM-as-a-Judge (`score_respuesta_correcta`, `score_precision_legal`) mediante tres jueces independientes (DeepSeek, Grok, Kimi K2).

6.3 Exploración Descriptiva y Selección de Variables de Respuesta

6.3.1 Análisis de Correlación entre Métricas

Se calcularon correlaciones de Spearman entre las 13 métricas textuales y las 2 métricas de evaluación por jueces. El análisis reveló cuatro clústeres de métricas:

1. **Clúster de volumen**: `word_count_ratio`, `length_ratio`, `character_error_rate` — métricas ortogonales a la calidad del contenido.
2. **Clúster de embeddings neuronales**: `bertscore`, `sbert_similarity_spanish`, `sbert_similarity_multilingual` — similitud semántica vía modelos de lenguaje.
3. **Clúster léxico/n-gramas**: `token_f1`, `jaccard_similarity`, `levenshtein_similarity`, `bleu`, `rouge_*` — solapamiento a nivel de tokens y n-gramas.
4. **Clúster agéntico**: `respuesta_correcta_avg` (alineada con el clúster semántico) y `precision_legal_avg` (alineada con el clúster léxico).

La Figura 18 presenta la matriz de correlación completa, donde los bloques de alta correlación intra-clúster (rojo) y las líneas divisorias confirman visualmente la estructura de agrupamiento.

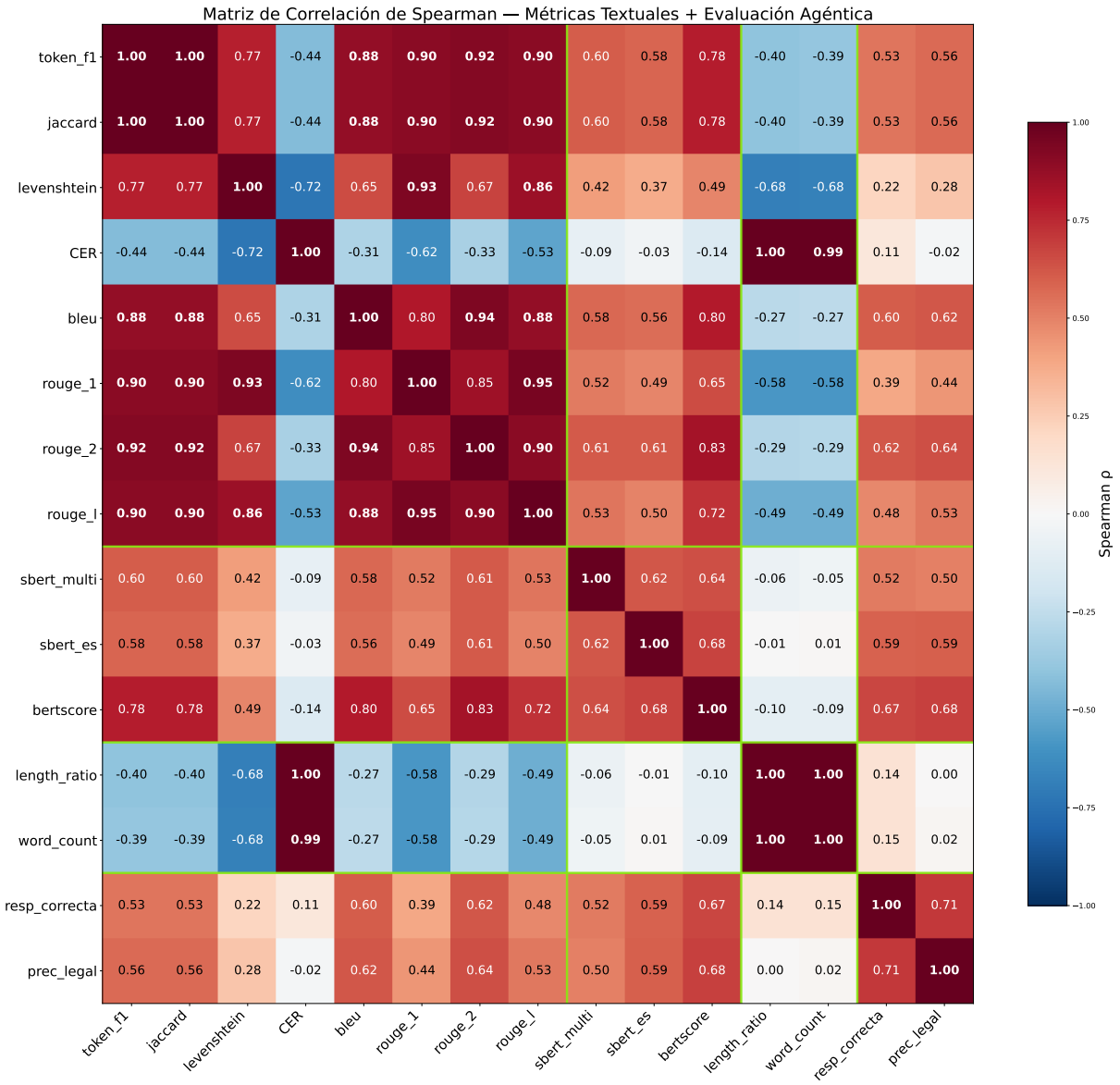


Figura 18: Matriz de correlación de Spearman entre las 13 métricas textuales y las 2 métricas de evaluación agéntica. Los bloques de correlación alta (rojo intenso) delimitan los cuatro clústeres: léxico/n-gramas (recuadro superior-izquierdo), embeddings neuronales (recuadro central), volumen (recuadro inferior-derecho) y agéntico (últimas dos filas/columnas). Las líneas punteadas separan los clústeres identificados.

6.3.2 Selección de Variables de Respuesta

Empleando un umbral de $|\rho| > 0.85$ para identificar redundancias, se identificaron seis candidatas a variables de respuesta (VR) representativas de los clústeres:

1. `token_f1`: representante del clúster léxico.
2. `rouge_l`: representante de métricas de subsecuencia.
3. `sbert_similarity_spanish`: representante del clúster semántico.
4. `bertscore`: similitud contextualizada.
5. `respuesta_correcta_avg`: evaluación global de qué tan correcta es una respuesta de acuerdo al método LLM-as-a-judge.
6. `precision_legal_avg`: evaluación de precisión en citación legal.

El análisis de componentes principales (PCA) con biplot confirmó la estructura de clústeres, ubicando `respuesta_correcta_avg` con correlación moderada con PC1 ($r = +0.488$) y PC2 ($r = +0.429$), y `precision_legal_avg` con correlación predominante con PC1 ($r = +0.545$). La Figura 19 presenta el biplot con las 13 métricas textuales como variables activas (azul) y las 2 métricas agénticas como variables suplementarias (rojo).

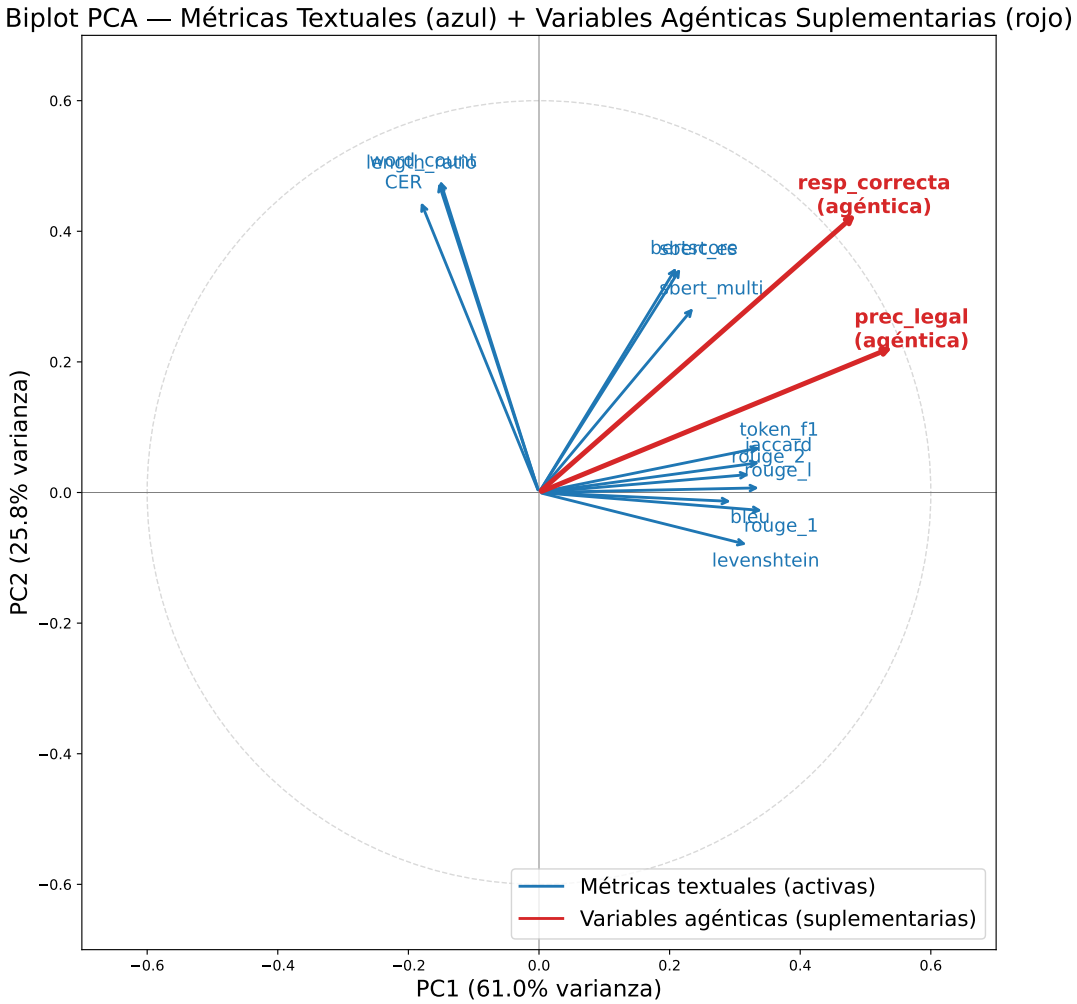


Figura 19: Biplot del análisis de componentes principales (PCA) sobre las 13 métricas textuales estandarizadas. Las flechas azules representan las métricas activas; las flechas rojas representan las variables agénticas proyectadas como suplementarias. PC1 (61,0%) captura principalmente el contraste entre métricas de similitud (derecha) y métricas de error/volumen (izquierda). PC2 (25,8%) separa las métricas de volumen del resto. `respuesta_correcta_avg` se ubica en una posición intermedia entre los clústeres semántico y léxico, mientras que `precision_legal_avg` se alinea más estrechamente con el clúster léxico. El círculo punteado indica el umbral de correlación $r = 0.6$ con las componentes principales.

Para las pruebas de hipótesis subsiguientes se retuvo exclusivamente `respuesta_correcta_avg` como variable de respuesta principal. Esta decisión se fundamenta en tres consideraciones: (1) es la métrica más directamente alineada con el objetivo de evaluar la calidad de las respuestas legales; (2) el análisis de acuerdo inter-juez (Sección 6.9) reveló que `precision_legal_avg` no alcanza confiabilidad suficiente; y (3) las métricas textuales y semánticas, aunque informativas, presentan alta redundancia interna y correlación moderada con la evaluación de corrección. Las cinco métricas

restantes se emplearon con fines descriptivos y complementarios, en particular en el análisis de costos.

La Figura 20 presenta la distribución de `respuesta_correcta_avg` agrupada por estrategia de recuperación.

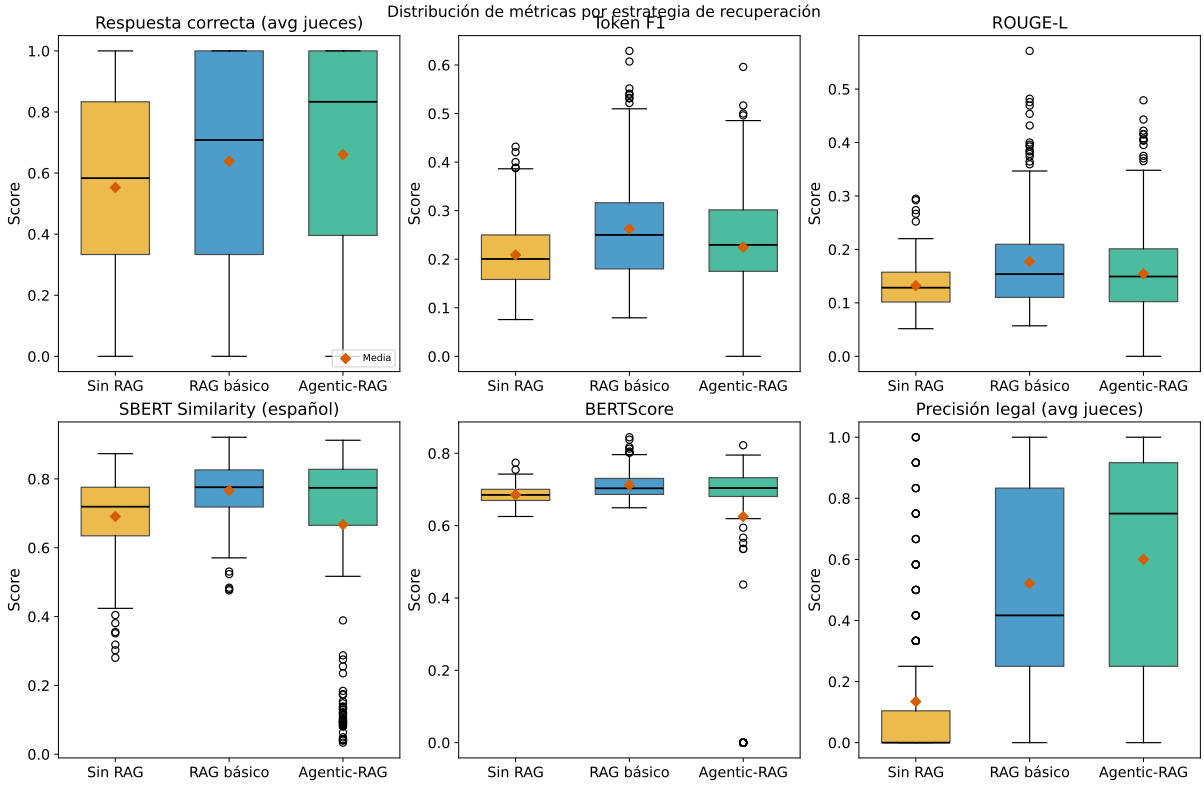


Figura 20: Distribución de `respuesta_correcta_avg` por estrategia de recuperación. Las cajas representan el rango intercuartílico (IQR); la línea central indica la mediana; los diamantes rojos señalan la media. Se observa una tendencia general de mejora desde la configuración sin RAG hacia Agentic-RAG, con mayor variabilidad en esta última debido a la interacción con el modelo.

6.3.3 Imputación de Valores Fuera de Rúbrica

Se detectaron valores de evaluación por jueces fuera del conjunto válido {0.0, 0.25, 0.5, 0.75, 1.0} (11 combinaciones únicas juez-métrica-valor). Se aplicó una estrategia de imputación al valor de rúbrica más cercano, resolviendo empates en favor del valor mayor (sesgo conservador hacia el reconocimiento parcial):

$$r^* = \arg\min_{r \in \mathcal{R}} |x - r|$$

donde $\mathcal{R} = \{0.0, 0.25, 0.5, 0.75, 1.0\}$.

Adicionalmente, se identificaron 27 observaciones (3.8% del total) con valores ausentes en las métricas de evaluación por jueces, atribuibles a fallos puntuales en la generación de respuestas para ciertas combinaciones subpregunta-configuración. Estos valores fueron excluidos de las pruebas de hipótesis mediante eliminación por lista (listwise deletion), reduciendo el conjunto efectivo a 684 observaciones para los análisis que involucran `respuesta_correcta_avg`.

6.4 Verificación de Normalidad

6.4.1 Test de Shapiro-Wilk

Se aplicó el test de Shapiro-Wilk (H_0: los datos siguen una distribución normal, $\alpha = 0.05$) a nivel global (todas las observaciones por métrica) y a nivel de grupos (dentro de cada combinación estrategia × modelo).

6.4.2 Resultados

La mayoría de las métricas rechazan la hipótesis nula de normalidad tanto a nivel global como por grupos. Los scores de evaluación por jueces, al ser mediciones ordinales en escala discreta {0.0, 0.25, 0.5, 0.75, 1.0}, no satisfacen el supuesto de normalidad por construcción.

6.4.3 Implicación Metodológica

Los resultados justifican el uso de pruebas no paramétricas para las comparaciones posteriores:

1. **Kruskal-Wallis** en lugar de ANOVA unilateral.
2. **Dunn con corrección de Bonferroni** como prueba post-hoc.
3. **Scheirer-Ray-Hare** en lugar de ANOVA factorial para evaluar interacciones.

6.5 Comparaciones entre Estrategias y Modelos

6.5.1 Test de Kruskal-Wallis

Se aplicó el test de Kruskal-Wallis como alternativa no paramétrica al ANOVA para comparar las distribuciones de cada VR entre los niveles de cada factor.

6.5.1.1 Efecto de la Estrategia de Recuperación

Para la métrica `respuesta_correcta_avg`, el test arrojó $H = 21.83$, $p < 0.001$, indicando diferencias significativas entre las tres estrategias de recuperación. Las medianas observadas fueron:

1. Agentic-RAG: 0.8333
2. RAG básico: 0.7083
3. Sin RAG: 0.5833

6.5.1.2 Efecto del Modelo

Para la misma métrica, $H = 49.01$, $p < 0.001$, confirmando diferencias significativas entre modelos. Las medianas fueron:

1. gpt-5: 0.8333
2. gpt-5-mini: 0.5417
3. gpt-5-nano: 0.5833

6.5.2 Prueba Post-Hoc de Dunn

Las comparaciones por pares con corrección de Bonferroni revelaron los siguientes patrones para `respuesta_correcta_avg`:

Entre estrategias:

1. Agentic-RAG vs. sin RAG: $p = 0.000014$ — diferencia significativa.
2. RAG básico vs. sin RAG: $p = 0.0066$ — diferencia significativa.
3. Agentic-RAG vs. RAG básico: $p = 0.383$ — sin diferencia significativa.

Entre modelos:

1. gpt-5 vs. gpt-5-nano: $p < 0.0001$ — diferencia significativa.
2. gpt-5 vs. gpt-5-mini: $p = 0.000007$ — diferencia significativa.
3. gpt-5-mini vs. gpt-5-nano: $p = 0.101$ — sin diferencia significativa.

6.5.3 Análisis Estratificado: Efecto del Modelo dentro de cada Sistema

Se realizaron tests de Kruskal-Wallis estratificados por sistema de recuperación, con nivel de significación ajustado por Bonferroni ($\alpha = \frac{0.05}{3} = 0.0167$):

1. **Agentic-RAG**: $H = 40.64$, $p < 0.0001$ — los modelos difieren significativamente.
2. **Sin RAG**: $H = 59.67$, $p < 0.0001$ — los modelos difieren significativamente.
3. **RAG básico**: $H = 4.71$, $p = 0.095$ — los modelos **no** difieren significativamente.

Este último resultado sugiere un **efecto igualador** del RAG básico: al proveer contexto relevante, las diferencias de capacidad entre modelos se atenúan, permitiendo que modelos más pequeños y económicos alcancen un desempeño comparable al del modelo de mayor tamaño.

6.5.4 Efectos Simples: Estrategia dentro de cada Modelo

El test de Scheirer-Ray-Hare reveló una interacción significativa entre estrategia y modelo ($H = 56.62, p < 10^{-11}$). Cuando existe interacción, los efectos principales no son interpretables de forma aislada: el efecto de la estrategia depende del modelo empleado. El análisis estratificado anterior evaluó el efecto del modelo dentro de cada estrategia; como complemento necesario, se evaluó el efecto de la estrategia dentro de cada modelo mediante tests de Mann-Whitney U (efectos simples de estrategia).

Dado que se realizan 9 comparaciones (3 modelos $\times$ 3 pares de estrategias), se aplicaron correcciones de Bonferroni y Holm-Bonferroni (Holm, 1979) mediante `statsmodels.stats.multitest.multipletests` (Seabold & Perktold, 2010). La Tabla 10 presenta los resultados completos.

Modelo	Estrategia 1	Estrategia 2	Med. 1	Med. 2	U	p	p corr. Holm	Sig. Holm
gpt-5	agentic_rag	base_rag	0.9583	0.8333	3475.5	0.024606	0.073819	
gpt-5	agentic_rag	no_rag	0.9583	0.8333	3545.5	0.012841	0.063887	
gpt-5	base_rag	no_rag	0.8333	0.8333	2850.5	0.889993	1.000000	
gpt-5-mini	agentic_rag	base_rag	0.9167	0.6250	3666.0	0.003055	0.018331	✓
gpt-5-mini	agentic_rag	no_rag	0.9167	0.3333	4964.0	< 0.0001	< 0.0001	✓
gpt-5-mini	base_rag	no_rag	0.6250	0.3333	4349.0	< 0.0001	< 0.0001	✓
gpt-5-nano	agentic_rag	base_rag	0.0833	0.5833	2076.0	0.002382	0.016676	✓
gpt-5-nano	agentic_rag	no_rag	0.0833	0.5833	2221.5	0.012777	0.063887	
gpt-5-nano	base_rag	no_rag	0.5833	0.5833	2923.5	0.896927	1.000000	

Tabla 10: Efectos simples de estrategia dentro de cada modelo (Mann-Whitney U). `Med. 1` y `Med. 2` son las medianas de `respuesta_correcta_avg` para cada estrategia. `p corr. Holm` es el p-value corregido por Holm-Bonferroni; ✓ indica significativo con corrección de Holm ($p_{\text{corr}} < 0.05$). $m = 9$ comparaciones.

Los resultados revelan patrones diferenciados por modelo:

1. **gpt-5-mini** es el modelo más sensible a la estrategia: las tres comparaciones son significativas incluso con corrección de Holm-Bonferroni. Agentic-RAG (mediana 0.92) supera ampliamente a RAG básico (0.63) y a sin RAG (0.33), evidenciando una dependencia crítica del contexto externo.
2. **gpt-5-nano** muestra un comportamiento contraintuitivo: Agentic-RAG (0.08) degrada significativamente el desempeño respecto a RAG básico (0.58, $p_{\text{corr}} = 0.017$), pero la degradación frente a sin RAG (0.58, $p_{\text{corr}} = 0.064$) no alcanza significación con corrección de Holm. Esto sugiere que la complejidad agéntica es excesiva para modelos de capacidad reducida.
3. **gpt-5** se beneficia de Agentic-RAG (0.96 vs. 0.83 sin RAG, $p = 0.013$), aunque esta mejora no sobrevive a la corrección por comparaciones múltiples ($p_{\text{corr}} = 0.064$), indicando un efecto positivo pero moderado.

Estos resultados refuerzan el hallazgo de interacción: el efecto de Agentic-RAG es marcadamente beneficioso para modelos de capacidad media-alta pero perjudicial para el modelo más pequeño.

La Tabla 11 presenta las medianas de `respuesta_correcta_avg` y, con fines descriptivos, de las cinco métricas complementarias para las nueve configuraciones evaluadas. Los valores de las métricas textuales y semánticas se incluyen como referencia, pero las pruebas de hipótesis se realizaron exclusivamente sobre `respuesta_correcta_avg`.

Estrategia	Modelo	Resp. correcta	Token F1	ROUGE-L	SBERT (es)	BERTScore	Prec. legal
Sin RAG	gpt-5-nano	0,583	0,200	0,123	0,740	0,688	0,000
Sin RAG	gpt-5-mini	0,333	0,176	0,123	0,629	0,674	0,000
Sin RAG	gpt-5	0,833	0,243	0,141	0,760	0,694	0,333
RAG básico	gpt-5-nano	0,583	0,253	0,150	0,770	0,703	0,333
RAG básico	gpt-5-mini	0,625	0,213	0,135	0,760	0,700	0,417
RAG básico	gpt-5	0,833	0,257	0,174	0,794	0,706	0,667
Agentic-RAG	gpt-5-nano	0,083	0,125	0,083	0,389	0,648	0,125
Agentic-RAG	gpt-5-mini	0,917	0,215	0,130	0,795	0,706	0,833
Agentic-RAG	gpt-5	0,958	0,277	0,190	0,791	0,708	0,875

Tabla 11: Medianas por configuración (estrategia × modelo). `Resp. correcta` es la variable de respuesta principal utilizada en las pruebas de hipótesis; las columnas restantes (`Token F1`, `ROUGE-L`, `SBERT (es)`, `BERTScore`, `Prec. legal`) se incluyen con fines descriptivos. Los valores más altos indican mejor desempeño. Destaca la mediana de 0,958 de Agentic-RAG + gpt-5 en respuesta correcta, y el valor anómalamente bajo de 0,083 para Agentic-RAG + gpt-5-nano.

6.6 Análisis Factorial: Interacción Estrategia × Modelo

6.6.1 Test de Scheirer-Ray-Hare

Para evaluar si los efectos de la estrategia de recuperación y del modelo operan de forma independiente o interactúan, se aplicó el test de Scheirer-Ray-Hare (SRH), análogo no paramétrico al ANOVA factorial de dos vías. El procedimiento consiste en:

1. Asignar rangos globales a todas las observaciones.
2. Aplicar ANOVA de dos vías sobre los rangos.
3. Calcular el estadístico H para cada efecto:

$$H = \frac{SS_{\text{efecto}}}{MS_{\text{total rangos}}}$$

donde H sigue una distribución χ^2 con los grados de libertad correspondientes.

6.6.2 Resultados de la Interacción

Los resultados para `respuesta_correcta_avg` fueron:

Efecto	H	gl	p	Sig.
Sistema	21.83	2	0.000018	Sí
Modelo	49.01	2	< 0.0001	Sí
Sistema × Modelo	56.62	4	1.48×10^{-11}	Sí

Tabla 12: Resultados del test de Scheirer-Ray-Hare para `respuesta_correcta_avg`.

La interacción significativa ($H = 56.62$, $p < 10^{-11}$) indica que el efecto de la estrategia de recuperación **depende** del modelo empleado, y viceversa. El estadístico de interacción supera a ambos efectos principales, lo que implica que las combinaciones explican más varianza que los factores individuales.

6.6.3 Patrón de Interacción

La Figura 21 presenta el gráfico de interacción que visualiza cómo las líneas de los modelos se cruzan entre estrategias, evidenciando la interacción disordinal.

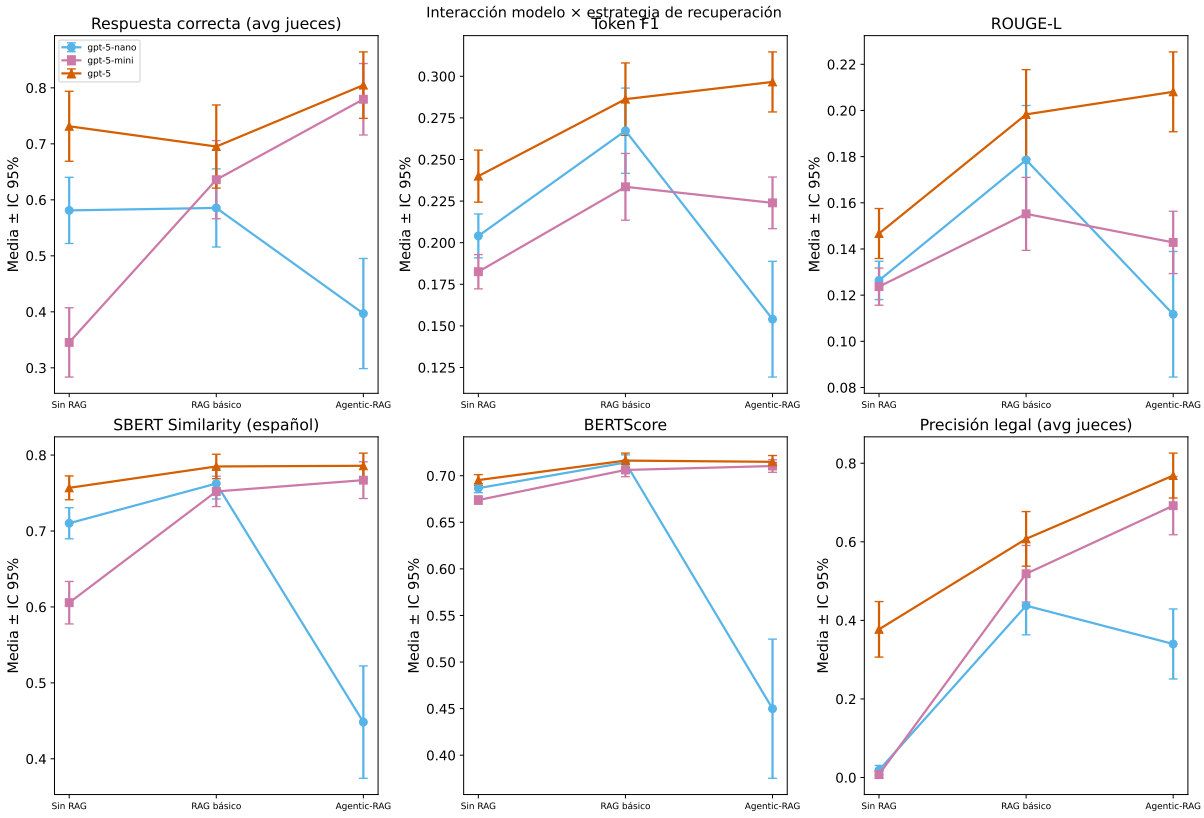


Figura 21: Gráficos de interacción modelo × estrategia de recuperación. Las líneas representan la media de cada modelo con intervalos de confianza del 95%. El cruce de las líneas confirma la interacción disordinal: el modelo óptimo depende de la estrategia empleada.

El análisis de las medias por combinación revela un patrón de interacción disordinal (líneas cruzadas):

Modelo	agentic_rag	base_rag	no_rag
gpt-5	0.80	0.70	0.73
gpt-5-mini	0.78	0.64	0.35
gpt-5-nano	0.40	0.59	0.58

Tabla 13: Medias de `respuesta_correcta_avg` por combinación modelo × sistema.

Las principales observaciones son:

1. **gpt-5 es robusto**: mantiene un desempeño alto (0.70–0.80) en todas las condiciones, compensando parcialmente la ausencia de contexto mediante su capacidad de razonamiento interna.
2. **gpt-5-mini es sensible al RAG**: con Agentic-RAG alcanza 0.78 (comparable a gpt-5), pero sin RAG cae a 0.35, evidenciando una dependencia crítica del contexto externo.
3. **gpt-5-nano muestra un comportamiento contraintuitivo**: su desempeño es **peor** con Agentic-RAG (0.40) que con RAG básico (0.59), lo que sugiere que el sistema agéntico genera instrucciones y contextos de complejidad excesiva para un modelo de capacidad reducida.

6.6.4 Implicaciones para Resultados Previos

La interacción significativa matiza las conclusiones del análisis de Kruskal-Wallis:

1. La no-diferencia marginal entre Agentic-RAG y RAG básico enmascara diferencias condicionales al modelo: para gpt-5-mini la diferencia es sustancial (0.78 vs. 0.64), mientras que para gpt-5-nano la relación se invierte (0.40 vs. 0.59).
2. El efecto igualador del RAG básico se confirma, pero no aplica al sistema agéntico, que amplifica las diferencias entre modelos.

6.7 Análisis de Costos

6.7.1 Costos Descriptivos por Modelo y Sistema

Se calculó el costo total de cada consulta sumando el costo de generación de la respuesta y el costo de recuperación (cuando aplica). Los costos se analizaron por modelo, por sistema y por combinación modelo × sistema. La Figura 22 presenta el costo medio por configuración.

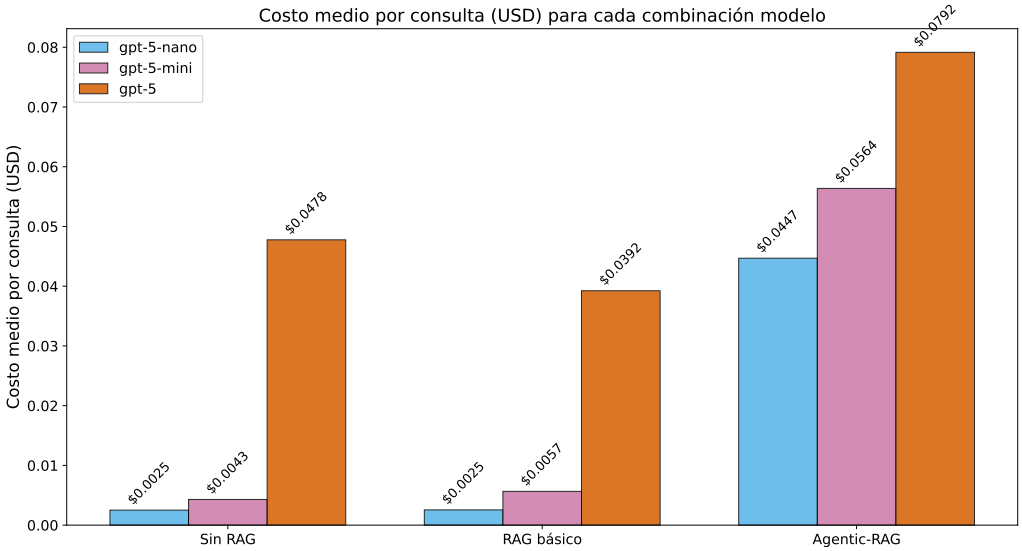


Figura 22: Costo medio por consulta (USD) para cada combinación modelo × estrategia de recuperación. El costo del sistema Agentic-RAG incluye las llamadas a los agentes de Recuperación y Calidad, que emplean gpt-5 independientemente del modelo del Agente Especialista.

6.7.2 Relación Costo-Desempeño

Para cada combinación se graficó el costo medio versus el desempeño medio en `respuesta_correcta_avg` y, complementariamente, en las cinco métricas textuales y semánticas (Figura 23). Este análisis permite identificar las configuraciones con mejor balance de eficiencia.

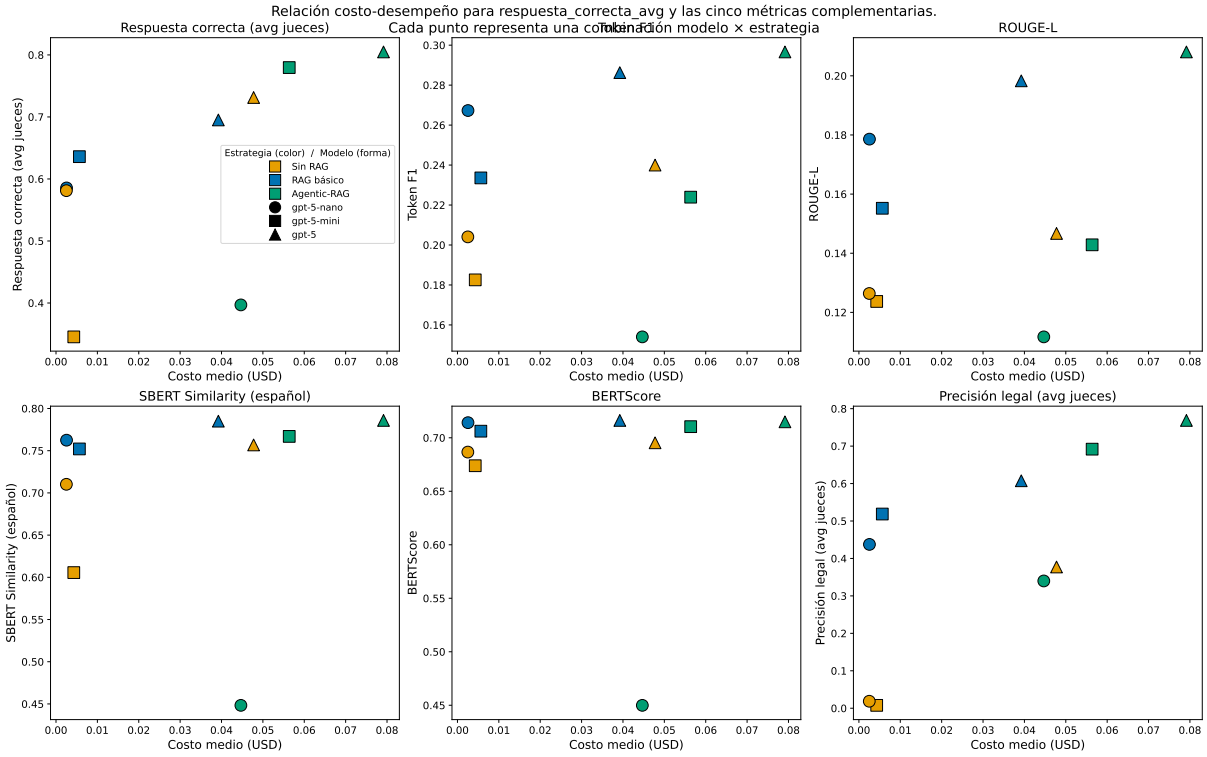


Figura 23: Relación costo-desempeño para `respuesta_correcta_avg` y las cinco métricas complementarias. Cada punto representa una combinación modelo × estrategia. El cuadrante superior-izquierdo (alto desempeño, bajo costo) contiene las configuraciones más eficientes.

6.7.3 Índice de Eficiencia

Se definió un índice de eficiencia como el cociente entre el desempeño y el costo normalizado (normalización min-max):

$$\text{Eficiencia} = \frac{\text{Desempeño}}{\text{Costo normalizado}}$$

donde el costo normalizado se calcula como:

$$c_{\text{norm}} = \max\left(\frac{c - c_{\min}}{c_{\max} - c_{\min}}, 0.01\right)$$

El piso de 0.01 evita la división por cero para la configuración de menor costo. Este índice permite comparar qué combinaciones ofrecen mayor desempeño por unidad de costo.

La Figura 24 presenta el índice de eficiencia para las nueve configuraciones, ordenadas de menor a mayor.

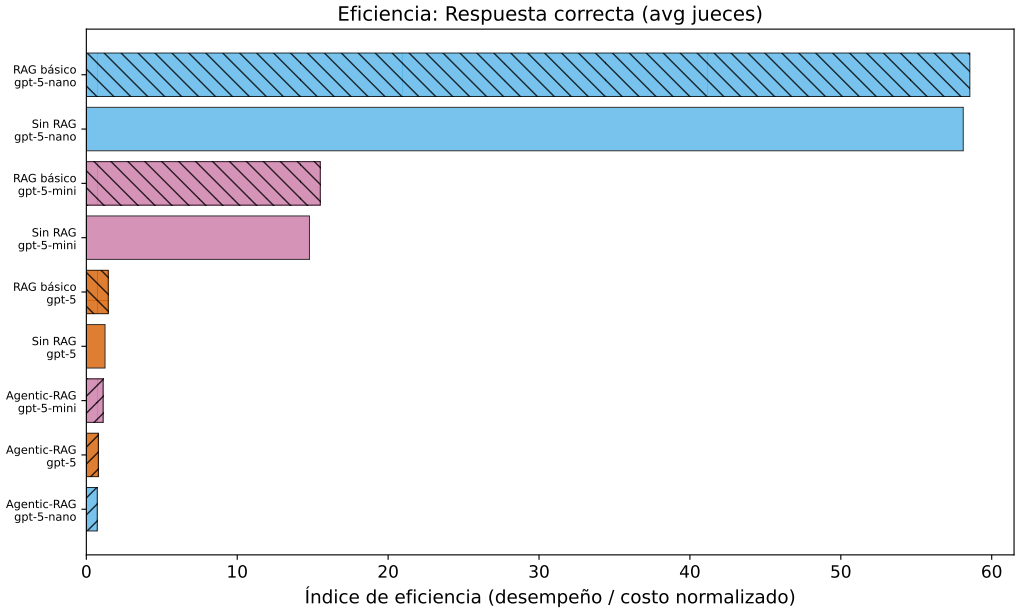


Figura 24: Índice de eficiencia (desempeño / costo normalizado) para la métrica `respuesta_correcta_avg`. Las barras se colorean según el modelo y se texturizan según la estrategia. Las configuraciones más eficientes combinan alto desempeño con bajo costo relativo.

6.7.4 Resumen de Tokens y Costos del Experimento

Se reportaron los tokens totales consumidos y el costo acumulado por modelo y por estrategia. El costo total del experimento fue de USD 22.29, distribuido en Agentic-RAG (USD 14.24), RAG básico (USD 3.75) y sin RAG (USD 4.31). El mayor costo de Agentic-RAG refleja las llamadas adicionales a los agentes de Recuperación, que emplean gpt-5 independientemente del modelo del Agente Especialista.

6.8 Evaluación LLM-as-a-Judge

6.8.1 Modelos de Juez

Se emplearon tres modelos como jueces independientes, cuyas características y configuración se detallan en el capítulo Sección 5: DeepSeek-V3, Grok 4.1 Fast y Kimi K2 0905. Cada juez evaluó las 9 configuraciones usando las dos rúbricas definidas en dicho capítulo (`score_respuesta_correcta`, Tabla 8; `score_precision_legal`, Tabla 9).

6.8.2 Resultados de «respuesta_correcta»

Los scores de `respuesta_correcta` muestran una distribución concentrada en los valores $\{0.5, 0.75, 1.0\}$ para los sistemas con RAG, mientras que el sistema sin RAG presenta mayor dispersión hacia valores bajos, consistente con la menor calidad de las respuestas sin contexto externo.

6.8.3 Resultados de Precisión Legal

Los scores de `precision_legal` presentan mayor variabilidad entre jueces que los de corrección, reflejando la mayor subjetividad inherente en evaluar la precisión de citación legal.

La Figura 25 presenta la distribución de los puntajes asignados por cada juez para ambas rúbricas.

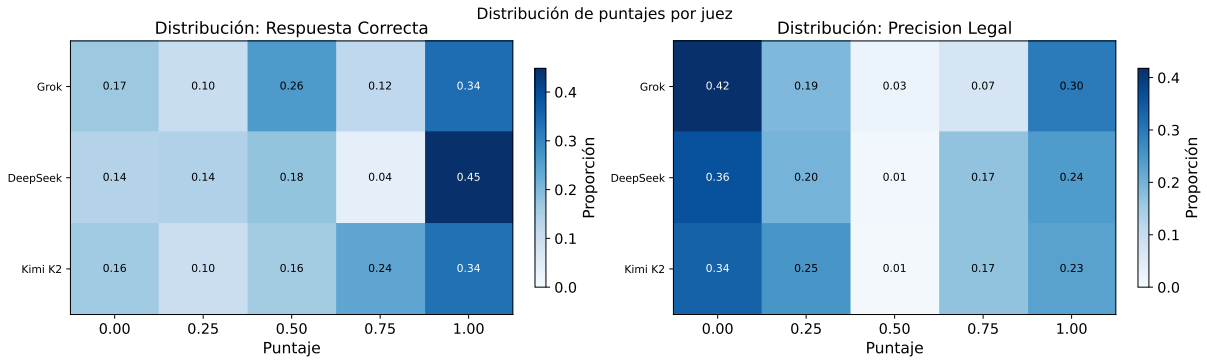


Figura 25: Distribución proporcional de puntajes por juez y rúbrica. Cada celda indica la proporción de evaluaciones que recibieron cada puntaje. Se observan diferencias en los patrones de distribución: algunos jueces tienden a concentrar sus evaluaciones en valores centrales (0,50–0,75), mientras que otros exhiben distribuciones más polarizadas.

6.9 Acuerdo Inter-Juez

El análisis de acuerdo inter-juez evalúa la confiabilidad de las evaluaciones LLM-as-a-Judge.

6.9.1 Correlación de Spearman entre Jueces

Las correlaciones de Spearman entre pares de jueces resultaron altas y significativas:

1. Para `score_respuesta_correcta`: $\rho = 0.79$–0.84 entre todos los pares.
2. Para `score_precision_legal`: $\rho = 0.77$–0.82 entre todos los pares.

Esto indica un ordenamiento relativo consistente de las respuestas entre los tres jueces.

6.9.2 Alpha de Krippendorff

El Alpha de Krippendorff (α) es un coeficiente de acuerdo inter-anotadores que corrige el acuerdo esperado por azar, soporta múltiples anotadores y datos faltantes, y admite escala ordinal. Los umbrales de interpretación según Krippendorff (2004) son: $\alpha \geq 0.800$ (bueno), $0.667 \leq \alpha < 0.800$ (aceptable), $\alpha < 0.667$ (insuficiente).

Los resultados globales fueron:

Rúbrica	α ordinal	Interpretación
`score_respuesta_correcta`	0.8078	Bueno
`score_precision_legal`	0.7846	Aceptable

Tabla 14: Alpha de Krippendorff global por rúbrica.

6.9.2.1 Alpha por Sistema de Recuperación

Rúbrica	agentic_rag	base_rag	no_rag
`score_respuesta_correcta`	0.8693	0.8219	0.6979
`score_precision_legal`	0.7550	0.6389	0.6211

Tabla 15: Alpha de Krippendorff ordinal por sistema de recuperación.

La Figura 26 visualiza estos resultados, permitiendo comparar de un vistazo la consistencia entre sistemas y rúbricas con respecto al umbral de aceptabilidad.

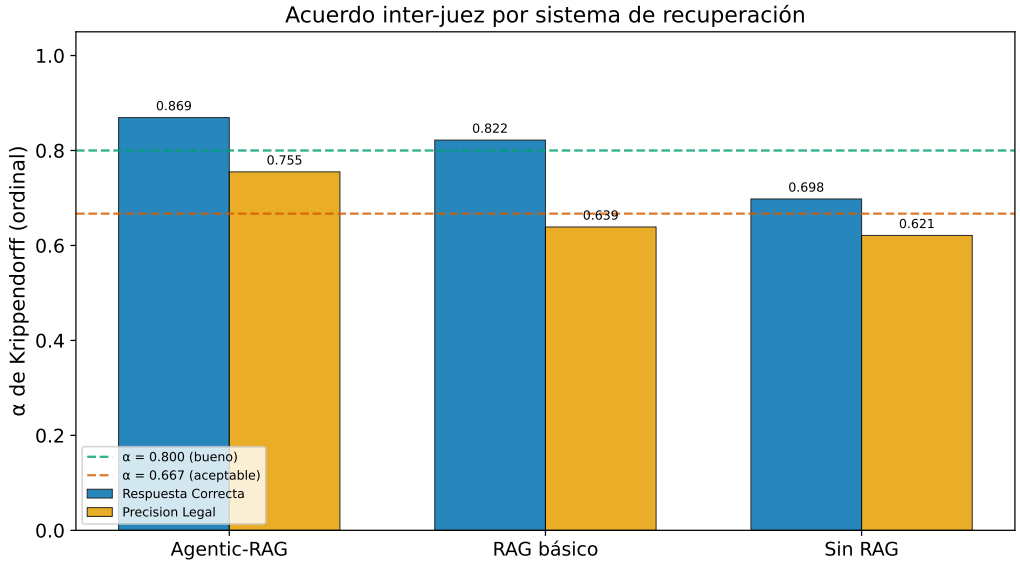


Figura 26: Alpha de Krippendorff ordinal por sistema de recuperación y rúbrica. Las líneas punteadas indican los umbrales de acuerdo aceptable ($\alpha = 0.667$) y bueno ($\alpha = 0.8$). Se observa que `score_respuesta_correcta` supera el umbral aceptable en todos los sistemas, mientras que `score_precision_legal` cae por debajo en *base_rag* y *no_rag*.

Para `score_respuesta_correcta`, el acuerdo es bueno o cercano al umbral en los tres sistemas. Para `score_precision_legal`, dos de los tres sistemas caen por debajo del umbral aceptable ($\alpha < 0.667$), lo que indica que los jueces no miden de forma consistente la precisión legal en esas condiciones.

6.9.3 Test Chi-Cuadrado de Homogeneidad

Se aplicó el test chi-cuadrado de homogeneidad sobre la tabla de contingencia jueces × valores de score para evaluar si los jueces distribuyen los scores de la misma manera. El test rechazó la hipótesis nula para ambas rúbricas ($p < 0.001$), confirmando diferencias en los patrones de distribución de scores entre jueces.

No obstante, este resultado es complementario al α de Krippendorff: el chi-cuadrado detecta diferencias estadísticamente significativas (sensible al tamaño muestral), mientras que el α cuantifica la magnitud del acuerdo y es el estadístico de decisión sobre la confiabilidad.

6.9.4 Decisión sobre `score_precision_legal`

Basándose en los resultados del acuerdo inter-juez, se decidió emplear exclusivamente `score_respuesta_correcta` para las comparaciones entre sistemas. Las razones principales son:

1. α ordinal de `score_precision_legal` cae por debajo de 0.667 en base_rag y no_rag.
2. Su coeficiente de variación inter-juez (95–105%) duplica al de `score_respuesta_correcta` (58–62%).
3. La ambigüedad conceptual inherente a la «precisión legal» no es resoluble sin una reformulación de la rúbrica con anclas específicas del dominio.
4. `score_respuesta_correcta` es suficiente para el objetivo de comparar estrategias, con $\alpha \geq 0.82$ en los sistemas RAG.

6.9.5 Análisis de Sesgo Sistemático

Se analizó el sesgo de cada juez mediante la media, mediana, desviación estándar y coeficiente de variación de sus scores. Se identificaron patrones de lenidad/severidad leves pero consistentes. El sesgo se analizó también de forma estratificada por sistema de recuperación, confirmando que las diferencias entre jueces son uniformes a través de los sistemas (sesgo no diferencial).

6.10 Discusión de Resultados

6.10.1 Fortalezas del Sistema

1. Los sistemas con recuperación (RAG básico y Agentic-RAG) superan significativamente al baseline sin RAG en la métrica de corrección.
2. El sistema Agentic-RAG combinado con gpt-5-mini alcanza un desempeño comparable al de gpt-5 (0.78 vs. 0.80), ofreciendo una alternativa de menor costo.
3. El RAG básico presenta un efecto igualador sobre los modelos, eliminando las diferencias significativas de desempeño entre gpt-5, gpt-5-mini y gpt-5-nano.
4. El acuerdo inter-juez para `score_respuesta_correcta` es bueno ($\alpha = 0.81$), validando la confiabilidad de las evaluaciones.

6.10.2 Debilidades y Limitaciones

1. El sistema Agentic-RAG degrada el desempeño de gpt-5-nano frente a RAG básico (0.40 vs. 0.59, $p_{\text{corr}} = 0.017$, Holm-Bonferroni), indicando que la complejidad agéntica

es excesiva para modelos de capacidad reducida. La diferencia frente a sin RAG no es significativa tras la corrección ($p_{\text{corr}} = 0.064$).
2. La métrica `score_precision_legal` no alcanza confiabilidad inter-juez suficiente para ser empleada en comparaciones, lo que limita la evaluación de la precisión de citación.
3. No se encontró diferencia marginal significativa entre Agentic-RAG y RAG básico, aunque el análisis de interacción revela que esta conclusión global enmascara patrones condicionales al modelo.
4. El benchmark se limita al Código de Minería Argentino, por lo que la generalización a otros cuerpos normativos requiere validación adicional.

La ausencia de diferencias significativas para gpt-5 entre estrategias tras la corrección de Holm-Bonferroni admite múltiples interpretaciones no mutuamente excluyentes. En primer lugar, el modelo podría poseer conocimiento paramétrico suficiente del Código de Minería —cuya última modificación data de 2024 y cuyos textos son de acceso público— para responder correctamente sin necesidad de recuperación externa, lo que explicaría su elevada mediana sin RAG (0.83). En segundo lugar, la escala discreta de cinco niveles de la rúbrica de evaluación podría generar un efecto de techo: si las respuestas de gpt-5 se concentran en los valores superiores (0.75–1.00), la resolución de la rúbrica resulta insuficiente para discriminar mejoras marginales que el RAG podría aportar. En tercer lugar, la potencia estadística disponible (n = 79 observaciones por celda, 9 comparaciones) podría ser insuficiente para detectar efectos moderados tras una corrección exigente como Holm-Bonferroni; la diferencia cruda entre Agentic-RAG y sin RAG ($p = 0.013$) es significativa sin corregir, pero no sobrevive a la corrección. Finalmente, la capacidad discriminatoria de los jueces LLM —que no fueron configurados con cadenas de razonamiento explícitas y cuya alineación con criterios humanos no ha sido validada— podría ser limitada para distinguir matices de corrección jurídica entre respuestas de alta calidad. La validación con expertos legales humanos constituye el único camino para dirimir entre estas hipótesis.

6.10.3 Implicaciones Prácticas

1. **Combinación óptima**: Agentic-RAG + gpt-5 (media 0.80), pero con costo superior.
2. **Combinación costo-eficiente**: Agentic-RAG + gpt-5-mini (media 0.78) para escenarios con restricciones presupuestarias, siempre que el sistema RAG esté activo.
3. **No existe una recomendación universal**: la interacción significativa entre factores implica que la selección de modelo y estrategia debe hacerse conjuntamente, no de forma independiente.
4. **Advertencia para modelos pequeños**: evitar la combinación Agentic-RAG + gpt-5-nano, que constituye la peor configuración evaluada.

6.11 Conclusiones del Capítulo

Los resultados confirman que la incorporación de un sistema de recuperación de información mejora significativamente la calidad de las respuestas de los LLM en el dominio legal argentino. La interacción significativa entre la estrategia de recuperación y el modelo ($H = 56.62$, $p < 10^{-11}$) es el hallazgo más relevante: demuestra que la efectividad de una estrategia depende del modelo con el que se combine, y viceversa.

El análisis de efectos simples mediante Mann-Whitney U con corrección de Holm-Bonferroni revela que Agentic-RAG beneficia significativamente a gpt-5-mini (las tres comparaciones entre estrategias son significativas, $p_{\text{corr}} < 0.05$) pero degrada a gpt-5--nano frente a RAG básico ($p_{\text{corr}} = 0.017$). Para gpt-5, no se detectan diferencias significativas entre estrategias tras la corrección por comparaciones múltiples. El RAG básico actúa como igualador de capacidades entre modelos, mientras que el sistema agéntico amplifica las diferencias. La evaluación LLM-as-a-Judge con tres jueces es confiable para la métrica de corrección ($\alpha = 0.81$) pero no para la de precisión legal ($\alpha = 0.78$ global, con valores insuficientes por sistema).

7 Trabajos Relacionados

7.1 Introducción

El presente capítulo revisa los trabajos previos y las plataformas existentes que guardan relación con el sistema Agentic-RAG propuesto en este trabajo final. La revisión se organiza en cuatro ejes: sistemas de búsqueda legal (tanto tradicionales como asistidos por inteligencia artificial), aplicaciones de RAG en dominios especializados con énfasis en el ámbito jurídico, sistemas agénticos y su integración con RAG, y benchmarks para la evaluación de sistemas de procesamiento de lenguaje natural en el dominio legal. Finalmente, se presenta una comparación del presente trabajo con la literatura analizada, identificando las contribuciones originales.

7.2 Sistemas de Búsqueda Legal

7.2.1 Sistemas Tradicionales Basados en Palabras Clave

Los sistemas de consulta legal más establecidos en Argentina operan mediante búsquedas basadas en coincidencia de palabras clave, filtros por tipo de norma, jurisdicción y fecha.

Infoleg (Ministerio de Justicia de la Nación, 2025) es una base de datos legislativa nacional de acceso libre y gratuito, administrada por la Dirección Nacional del Sistema Argentino de Información Jurídica del Ministerio de Justicia de la Nación. Contiene leyes, decretos, decisiones administrativas, resoluciones, disposiciones y todo acto cuya publicación sea obligatoria en la primera sección del Boletín Oficial de la República Argentina. Sus contenidos están licenciados bajo Creative Commons Reconocimiento 2.5 Argentina.

SAIJ (Sistema Argentino de Información Jurídica) (Ministerio de Justicia y Derechos Humanos, 2025) es un servicio de la Dirección Nacional del SAIJ, dependiente del Ministerio de Justicia y Derechos Humanos de la Nación. Fundado en 1981 con apoyo de la UNESCO, incluye todas las leyes nacionales sancionadas desde 1853, tratados internacionales, decretos nacionales y provinciales, jurisprudencia de tribunales federales, nacionales y provinciales, proyectos legislativos, dictámenes de la Procuración del Tesoro de la Nación, artículos de doctrina y publicaciones de Ediciones SAIJ. Su motor de búsqueda permite filtrar por tipo de contenido (legislación, fallos, sumarios, doctrina), tipo de norma (ley, decreto, resolución, disposición), jurisdicción y fecha.

A nivel internacional, plataformas como **Westlaw** (Thomson Reuters, 2025) y **LexisNexis** (LexisNexis, 2025) han sido durante décadas los referentes en investigación legal, operando inicialmente con motores de búsqueda por palabras clave sobre extensas bases de datos de jurisprudencia y legislación, principalmente de Estados Unidos y otros países anglosajones.

Si bien estos sistemas cumplen una función esencial al garantizar el acceso a la información jurídica, su dependencia de la coincidencia exacta de términos limita su capacidad para comprender la semántica de las consultas, lo que frecuentemente produce

resultados irrelevantes o incompletos cuando el usuario no emplea los términos técnicos precisos.

7.2.2 Sistemas Comerciales con IA

En los últimos años, múltiples plataformas comerciales han integrado inteligencia artificial para mejorar la búsqueda y el análisis jurídico. A continuación se describen las más relevantes.

Lexis+ con Protégé (LexisNexis, 2025) es la evolución de la plataforma de LexisNexis que integra un asistente de IA agéntico denominado Protégé. Este sistema es capaz de completar tareas de forma autónoma, revisar su propio trabajo y sugerir mejoras. Entre sus capacidades se incluyen la redacción de documentos legales (mociones, demandas, memorandos), la investigación legal con validación de citas mediante Shepard's, el análisis de documentos de hasta 300 páginas y la generación de cronologías a partir de materiales cargados. En agosto de 2025, LexisNexis incorporó Protégé General AI, que provee acceso seguro a múltiples modelos de IA de propósito general (GPT-5, Claude Sonnet 4, GPT-4o) junto con herramientas de IA específicas para el dominio legal. La plataforma emplea una arquitectura RAG con integración del grafo de conocimiento Shepard's (GraphRAG) para generar respuestas fundamentadas en fuentes verificables.

Westlaw con CoCounsel (Thomson Reuters, 2025) es la plataforma de Thomson Reuters que lanzó CoCounsel Legal en agosto de 2025, representando su oferta más avanzada de IA. Integrada con Westlaw Advantage, incluye la funcionalidad Deep Research, que en lugar de devolver simplemente resultados de búsqueda, razona a través de preguntas legales, genera planes de investigación de múltiples pasos, explica su lógica y entrega informes completos fundamentados en el contenido de Westlaw y Practical Law. La tecnología se basa en los desarrollos de CoCounsel, construidos sobre GPT-4. Un estudio de la Universidad de Stanford publicado en el Journal of Empirical Legal Studies en 2025 encontró que Westlaw AI presentaba alucinaciones en el 33% de las consultas, mientras que Lexis+ AI lo hacía en el 17%, en comparación con el 43% de GPT-4 sin RAG (Dahl et al., 2024).

vLex con Vincent AI (vLex, 2025) es una plataforma de inteligencia legal generativa que combina una de las mayores bases de datos legales a nivel global —con más de mil millones de documentos de más de 100 países— con IA avanzada. Vincent AI interpreta consultas en lenguaje natural para producir respuestas legales con citas completas, generar argumentos legales, comparar leyes entre jurisdicciones y automatizar flujos de trabajo complejos. En ensayos controlados aleatorizados, Vincent mostró 3,67 veces más fiabilidad que los LLM de propósito general. La plataforma emplea búsqueda vectorial (machine learning) para organizar documentos mediante búsqueda conceptual en lugar de coincidencia de palabras clave. Su alcance internacional, que incluye cobertura de Estados Unidos, Reino Unido, España, la Unión Europea, Singapur, México, Chile, Nueva Zelanda y Colombia, lo diferencia de sus competidores.

Lexius (Lexius, 2025) es un sistema jurídico con IA de origen latinoamericano que opera en 18 países. En Argentina cuenta con acceso a más de 32.500 textos legales

y 52.500 sentencias. Su motor de búsqueda inteligente identifica la intención de la consulta del usuario y ofrece funcionalidades como chat y llamadas con IA para resolver dudas legales, generación de video podcasts legales narrados por IA, mapas mentales interactivos y exportación de documentos en múltiples formatos.

Juztina (Juztina, 2025) es una legaltech peruana que utiliza inteligencia artificial para democratizar el acceso a información jurídica en América Latina. Su plataforma ofrece un chat de IA con acceso a más de 3 millones de documentos legales, permitiendo a abogados y usuarios realizar búsquedas especializadas de jurisprudencia, normativa y doctrina. Con presencia en Perú, Colombia y México, Juztina ha expandido su operación a Argentina. La búsqueda de información empleando la plataforma toma de diez segundos a un minuto, reduciendo significativamente los tiempos de investigación legal.

Veredicta (Veredicta, 2025) es una plataforma argentina que integra gestión de expedientes con IA. Su motor de búsqueda semántica analiza más de 500.000 fuentes judiciales argentinas, permite buscar describiendo el caso en lenguaje natural, genera resúmenes automáticos de fallos y está integrado con el Poder Judicial de la Nación (PJN), expedientes y WhatsApp. A diferencia de herramientas como Lexius o Juztina, Veredicta permite encontrar jurisprudencia y vincularla directamente al expediente en curso.

Lexium (Lexium, 2025) es una plataforma de IA de gestión jurídica especializada en derecho argentino. Ofrece búsqueda inteligente en SAIJ, Infoleg y jurisprudencia, automatización de escritos y gestión de vencimientos procesales. La propia plataforma reconoce las limitaciones inherentes a los modelos de IA generativa, incluyendo la posibilidad de generar información incompleta, imprecisa o no actualizada con la legislación vigente, recomendando verificar siempre las fuentes oficiales.

En Argentina también se destaca el sistema de **Búsqueda de Jurisprudencia Asistida por IA** del Tribunal Fiscal de la Nación (Tribunal Fiscal de la Nación, 2025; World Law Group, 2025), que constituye una de las primeras iniciativas gubernamentales en el país en incorporar inteligencia artificial a la búsqueda de documentos legales, aunque limitado a documentos emitidos por dicho tribunal.

7.3 Sistemas RAG en Dominios Especializados

7.3.1 RAG para el Dominio Legal

La aplicación de RAG al dominio legal ha sido objeto de creciente investigación. Hindi et al. (Hindi et al., 2025) presentaron una revisión exhaustiva sobre la precisión e interpretabilidad de los sistemas RAG en tecnología legal, identificando que la combinación de recuperación semántica con generación de texto permite anclar las respuestas en documentos reales, reduciendo significativamente las alucinaciones en comparación con el uso de LLMs sin RAG. La revisión cataloga diversos métodos y aplicaciones de RAG legal, incluyendo análisis de contratos, protección de privacidad y evaluación de sistemas mediante LegalBench-RAG (Pipitone & Alami, 2024).

Siino et al. (Siino et al., 2025) realizaron una revisión de la literatura sobre aplicaciones de LLMs en el ámbito legal, cubriendo tareas como clasificación de textos legales, reconocimiento de entidades nombradas, resumidización de documentos y respuesta a preguntas legales. Los autores identifican que si bien los avances son significativos, la mayoría de los trabajos se concentran en el idioma inglés y en los sistemas jurídicos del common law, dejando subrepresentados los sistemas de derecho civil como el argentino.

LexisNexis incorporó a su plataforma Lexis+ AI mejoras en su pipeline de RAG, incluyendo la integración del grafo de conocimiento Shepard's (GraphRAG), que utiliza información de relaciones entre casos para generar respuestas más completas y autoritativas.

A nivel académico, se han desarrollado sistemas como JusBuild, que emplea un modelo de campos aleatorios condicionales (CRF) para la segmentación supervisada de documentos legales en secciones funcionales, una base de datos vectorial para almacenar representaciones semánticas y RAG con LLMs para sugerir secciones relevantes durante la redacción de documentos legales.

7.3.2 RAG para Documentación Técnica y Otros Dominios

Los sistemas RAG han encontrado aplicaciones significativas en múltiples dominios especializados más allá del legal. En el ámbito médico, una revisión narrativa publicada en PubMed Central (Gao et al., 2023) exploró la aplicación de RAG en contextos médicos, identificando su utilidad para soporte en decisiones clínicas, asistencia diagnóstica, interpretación de guías médicas y evaluación de elegibilidad para ensayos clínicos. Un estudio publicado en npj Digital Medicine evaluó modelos LLM-RAG para determinar la aptitud quirúrgica utilizando 35 guías locales y 23 internacionales, demostrando que RAG permite personalizar LLMs integrando conocimiento especializado con mayor precisión y consistencia que los modelos base.

En el ámbito financiero, se han explorado aplicaciones de RAG para la recuperación y consulta de documentación regulatoria, reportes financieros y normativa contable (Govind Srinivasan et al., 2025). En contextos de gobernanza electrónica, se ha evaluado la fidelidad de sistemas Agentic-RAG utilizando frameworks de evaluación basados en LLM-as-a-Judge con múltiples modelos evaluadores.

Estos trabajos demuestran que la arquitectura RAG es transferible entre dominios especializados, y que las técnicas de fragmentación de texto, embeddings semánticos y recuperación por similitud desarrolladas en un dominio pueden informar el diseño de sistemas en otros.

7.4 Sistemas Agénticos

7.4.1 Agentes para Tareas Legales

La aplicación de agentes basados en LLM al dominio legal ha sido revisada de forma comprehensiva por Wan et al. (Wan & others, 2025), quienes trazan la evolución desde sistemas de agente único hacia arquitecturas multi-agente. Los autores identifican que

los agentes legales pueden clasificarse según sus capacidades: consulta legal, análisis de documentos, asistencia en litigios y simulación judicial.

He et al. (He et al., 2024) propusieron **AgentsCourt**, un sistema que construye agentes de toma de decisiones judiciales mediante simulación de debates judiciales y aumentación de conocimiento legal. El sistema simula el proceso judicial completo, con agentes que representan diferentes roles (juez, fiscal, defensor), demostrando que la interacción multi-agente puede mejorar la calidad del razonamiento legal.

En una dirección complementaria, se ha propuesto una **arquitectura multi-agente para IA legal confiable** (various, 2025) que emplea tres componentes principales: enrutamiento de consultas a agentes especializados para actos legales específicos, recuperación de regulaciones y razonamiento para generar respuestas fundamentadas. El sistema fue evaluado utilizando un examen de ingreso a la abogacía de 2024 con 150 preguntas, demostrando que la combinación de agentes especializados con RAG mejora significativamente la precisión respecto a modelos de propósito general.

7.4.2 Agentic-RAG en la Literatura

El concepto de Agentic-RAG ha sido formalizado por Singh et al. (Singh et al., 2025), quienes presentaron una revisión comprensiva que distingue este paradigma del RAG tradicional. Mientras que el RAG convencional sigue flujos de trabajo estáticos (consulta $\rightarrow$ recuperación $\rightarrow$ generación), Agentic-RAG incorpora agentes autónomos de IA en el pipeline que son capaces de planificar, razonar, usar herramientas y adaptarse al contexto de la consulta. Los autores identifican que Agentic-RAG trasciende las limitaciones del RAG estático al permitir razonamiento de múltiples pasos y gestión de tareas complejas.

La tendencia hacia sistemas Agentic-RAG se ve reforzada por avances recientes en modelos de razonamiento de gran escala (Large Reasoning Models), como los modelos basados en aprendizaje por refuerzo (DeepSeek-R1, modelos análogos a OpenAI o1), que poseen capacidades inherentes de razonamiento complejo y de múltiples pasos, integrándose naturalmente en flujos de trabajo agénticos.

LexisNexis ha adoptado este paradigma con su asistente Protégé, descrito explícitamente como un «sistema de IA agéntico» capaz de completar tareas de forma autónoma, revisar su propio trabajo y sugerir mejoras (LexisNexis, 2025). De manera similar, Westlaw con CoCounsel implementa capacidades agénticas a través de Deep Research, que genera planes de investigación de múltiples pasos y explica su lógica.

7.5 Benchmarks para Evaluación Legal

7.5.1 LegalBench, LexGLUE y Otros Benchmarks en Inglés

La evaluación del desempeño de modelos de lenguaje en tareas legales se ha beneficiado del desarrollo de múltiples benchmarks, aunque la gran mayoría se concentran en el idioma inglés y en sistemas jurídicos anglosajones.

LexGLUE (Chalkidis et al., 2021) es un benchmark inspirado en GLUE que comprende siete datasets para evaluar la comprensión del lenguaje legal en inglés, cubriendo

legislación de la Unión Europea y Estados Unidos. Incluye tareas de clasificación multi--etiqueta y multi-clase sobre documentos legales del Parlamento Europeo, la Corte Suprema de Estados Unidos y contratos.

LegalBench (Guha et al., 2023) es un esfuerzo colaborativo de código abierto que agrupa tareas para evaluar el razonamiento legal en LLMs. Contiene tareas diversas que cubren múltiples tipos textuales, estructuras de tareas, dominios legales y niveles de dificultad, centradas principalmente en el derecho estadounidense. LegalBench es notable por su enfoque interdisciplinario, combinando la experiencia de científicos de la computación y abogados.

LegalBench-RAG (Pipitone & Alami, 2024) adapta el benchmark LegalBench para evaluar específicamente sistemas de generación aumentada por recuperación en el dominio legal, permitiendo medir no solo la calidad de la generación sino también la efectividad de la recuperación de documentos relevantes.

LEXTREME (Niklaus et al., 2024) es el primer benchmark multilingüe y multi--tarea para el dominio legal, cubriendo 24 idiomas indoeuropeos y urálicos a través de 11 datasets. Sus resultados muestran que incluso los mejores modelos de referencia alcanzan resultados modestos, y que ChatGPT enfrenta dificultades con muchas tareas, evidenciando la brecha entre los modelos de propósito general y las necesidades del dominio legal especializado.

LawBench (Fei et al., 2024) se enfoca en el sistema legal chino, y **LEXam** (Niklaus & others, 2025) evalúa el razonamiento legal sobre 340 exámenes de derecho de múltiples jurisdicciones.

7.5.2 Benchmarks en Español y Modelos para el Dominio Legal Hispanohablante

El estado del arte en evaluación de NLP legal en español es considerablemente más limitado que en inglés. Si bien LEXTREME incluye algunos idiomas europeos, el español está subrepresentado en los corpus de entrenamiento de los grandes modelos de lenguaje, lo que genera un desafío adicional para el desarrollo de herramientas de NLP para textos legales en español.

MEL (Modelo de Español Legal) (Betancur Sánchez & others, 2025) es un modelo de lenguaje basado en XLM-RoBERTa-large, entrenado específicamente para la comprensión de textos legales en español. Sus evaluaciones muestran una mejora significativa sobre los modelos base en tareas como clasificación legal y reconocimiento de entidades nombradas en español. Este trabajo es especialmente relevante al demostrar que la adaptación al dominio legal en español requiere modelos especializados.

El corpus **Legal-ES** (Samy & others, 2020) constituye un meta-corpus que reúne múltiples fuentes del dominio legal en lengua española, incluyendo textos legislativos, jurisprudenciales (sentencias) y administrativos. Además, presenta resultados preliminares sobre embeddings del español jurídico y un modelo de tópicos entrenado sobre legislación española.

En Argentina, no se han identificado a la fecha benchmarks específicos para evaluar sistemas de consulta sobre normativa argentina, lo que constituye una de las motivaciones del presente trabajo.

7.6 Comparación con el Presente Trabajo

La Tabla 16 presenta una comparación del presente trabajo con los sistemas y plataformas revisados en este capítulo.

Sistema/Trabajo	**Código abierto**	**RAG**	**Agentes**	**Normativa argentina**	**Benchmark propio**
Infoleg / SAIJ	N/A	No	No	Sí	No
Westlaw CoCounsel	No	Sí	Sí	No	No
Lexis+ Protégé	No	Sí	Sí	No	No
vLex Vincent AI	No	Sí	Sí	Parcial	No
Lexius	No	Sí	Sí	Sí	No
Juztina	No	Sí	Sí	Sí	No
Veredicta	No	Sí	No	Sí	No
Lexium	No	Sí	No	Sí	No
AgentsCourt	Sí	No	Sí	No	No
Presente trabajo	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**

Tabla 16: Comparación del presente trabajo con sistemas y trabajos relacionados

7.6.1 Diferencias Metodológicas y Contribuciones Originales

El presente trabajo se diferencia de los trabajos y sistemas revisados en los siguientes aspectos:

1. **Código abierto**: A diferencia de las plataformas comerciales (Lexius, Juztina, Veredicta, Lexium, Westlaw, Lexis+, vLex), el sistema propuesto es de código abierto, con toda la documentación y el código disponible en un repositorio público, permitiendo su replicación y adaptación por cualquier organización.

2. **Enfoque integral Agentic-RAG sobre normativa argentina**: Si bien existen plataformas comerciales que ofrecen funcionalidades de IA sobre normativa argentina, ninguna de las identificadas publica su arquitectura ni permite evaluar independientemente su desempeño. El presente trabajo documenta de manera completa y trasparente la arquitectura Agentic-RAG empleada.

3. **Benchmark específico**: No se han identificado benchmarks públicos para evaluar sistemas de consulta sobre normativa argentina. El desarrollo de un benchmark para el Código de Minería Argentino, elaborado con profesionales del derecho, constituye una contribución original que permite la evaluación reproducible del sistema.

4. **Dominio del derecho civil en español**: La mayoría de los benchmarks y trabajos académicos revisados se concentran en el common law anglosajón. El presente trabajo

contribuye al subrepresentado dominio del derecho civil argentino en idioma español, complementando los esfuerzos de modelos como MEL (Betancur Sánchez & others, 2025) y recursos como Legal-ES (Samy & others, 2020).

5. **Evaluación comparativa de estrategias**: El trabajo evalúa y compara tres estrategias (sin RAG, RAG básico y Agentic-RAG) sobre el mismo corpus y benchmark, permitiendo cuantificar el valor agregado de cada componente de la arquitectura.

7.7 Desarrollos Posteriores al Período Experimental

El ritmo de evolución del campo de la inteligencia artificial implicó que, entre la finalización de los experimentos (segundo semestre de 2025) y la redacción final de este manuscrito, se produjeran avances significativos. Esta sección documenta los desarrollos más relevantes organizados en tres ejes: modelos de lenguaje propietarios, modelos de código abierto y *frameworks* agénticos. La Figura 27 presenta una línea de tiempo con los principales hitos.

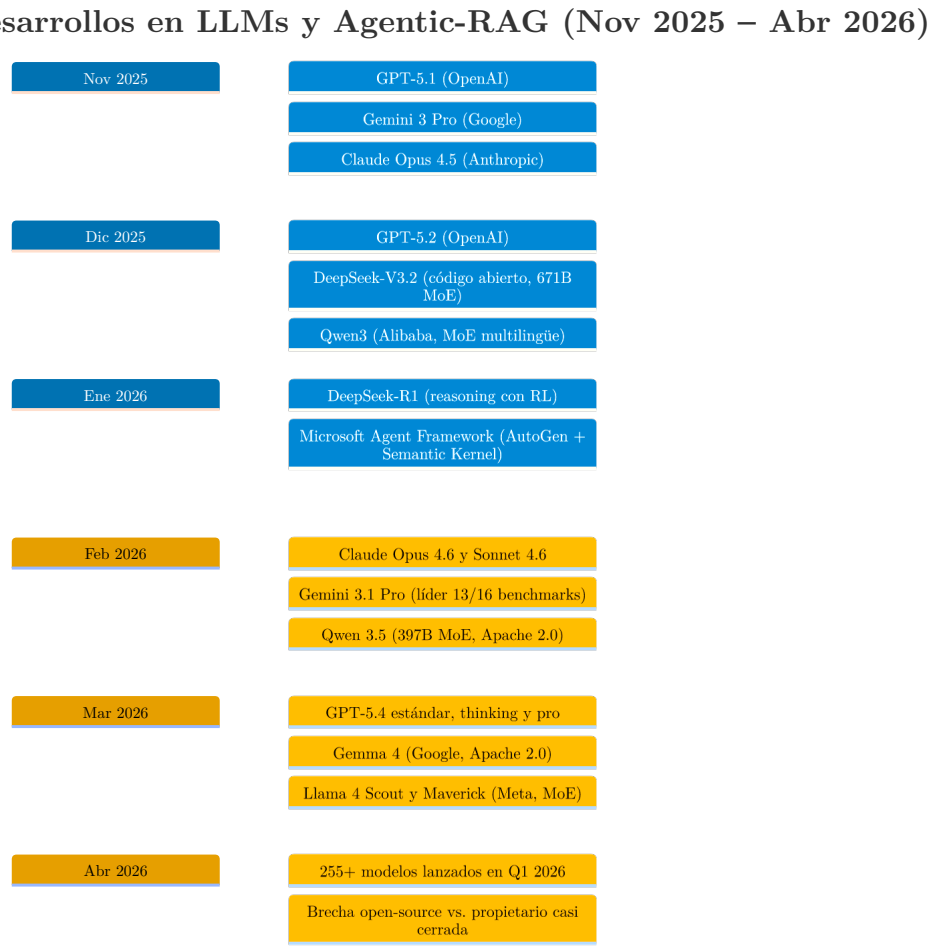


Figura 27: Línea de tiempo de los principales desarrollos en LLMs y Agentic-RAG entre noviembre de 2025 y abril de 2026, período posterior a la finalización de los experimentos de este trabajo.

7.7.1 Modelos Propietarios de Nueva Generación

En un lapso de apenas doce días en noviembre de 2025, los tres principales laboratorios de IA lanzaron nuevos modelos insignia: OpenAI publicó GPT-5.1 (12 de noviembre), que introdujo razonamiento adaptativo donde el modelo decide autónomamente cuánto tiempo de «pensamiento» asignar a cada consulta; Google lanzó Gemini 3 Pro (20 de noviembre), que alcanzó el primer puesto en múltiples *benchmarks* académicos con una ventana de contexto de un millón de *tokens*; y Anthropic presentó Claude Opus 4.5 (24 de noviembre), con mejoras sustanciales en razonamiento avanzado.

La cadencia se aceleró en los meses siguientes. OpenAI iteró rápidamente con GPT-5.2 (diciembre de 2025), GPT-5.4 estándar (5 de marzo de 2026) y las variantes GPT-5.4 mini y nano (17 de marzo de 2026). Anthropic respondió con Claude Opus 4.6 y Sonnet 4.6 (febrero de 2026), donde Sonnet 4.6 alcanzó rendimiento cercano a Opus a un costo significativamente menor. Google presentó Gemini 3.1 Pro (19 de febrero de 2026), que lideró 13 de 16 *benchmarks* principales y empató con GPT-5.4 Pro en el índice de inteligencia de Artificial Analysis a aproximadamente un tercio del costo de API. xAI lanzó Grok 4.20 (febrero–marzo de 2026), notable por introducir una arquitectura multi-agente novedosa.

El dato más revelador del período es la densidad de lanzamientos: solo en el primer trimestre de 2026 se registraron más de 255 lanzamientos de modelos por parte de organizaciones relevantes, con siete modelos de frontera publicados únicamente en febrero de 2026.

7.7.2 Modelos de Código Abierto y Razonamiento

Uno de los desarrollos más disruptivos del período fue la irrupción de modelos de código abierto con capacidades competitivas frente a los modelos propietarios, lo que alteró fundamentalmente la dinámica del mercado de IA.

DeepSeek captó la atención global con dos lanzamientos clave: DeepSeek-V3.2 (diciembre de 2025), un modelo de 671B parámetros con arquitectura *Mixture-of-Experts* (MoE, 37B activos) que alcanzó rendimiento comparable a GPT-5 en razonamiento a una fracción del costo de entrenamiento (aproximadamente $6 millones frente a más de $100 millones estimados para modelos propietarios equivalentes); y DeepSeek-R1 (enero de 2026), un modelo de razonamiento entrenado con aprendizaje por refuerzo a gran escala que demostró capacidades a nivel de GPT-4 en tareas de razonamiento complejo. DeepSeek-R1 introdujo además una familia de modelos destilados (*DeepSeek-R1-Distill*) basados en arquitecturas Qwen y Llama, diseñados para entornos de producción con restricciones de cómputo.

Qwen (Alibaba) avanzó desde Qwen3 (diciembre de 2025) hasta Qwen 3.5 (16 de febrero de 2026), un modelo de 397B parámetros totales con 17B activos por paso de inferencia, licenciado bajo Apache 2.0. Qwen 3.5 se posicionó como líder en tareas de visión, seguimiento de instrucciones y comprensión multimodal, áreas directamente relevantes para aplicaciones de producción.

Meta publicó Llama 4 con las variantes Scout (109B totales/17B activos) y Maverick (400B totales/17B activos), ambas con arquitectura MoE, capacidades multimodales y contexto extendido, manteniendo la línea de modelos abiertos que ha catalizado el ecosistema de código abierto desde 2023. **Google** contribuyó con Gemma 4, una familia de modelos compactos (hasta 26B parámetros) con licencia Apache 2.0, donde el modelo de 9B parámetros alcanzó un 81.7% en GPQA Diamond, superando a modelos de código abierto hasta 13 veces más grandes.

El resultado neto es que, para abril de 2026, la brecha de rendimiento entre modelos de código abierto y propietarios se ha reducido sustancialmente. Análisis independientes sugieren que para aproximadamente el 80% de tareas empresariales comunes (clasificación, extracción, resumen, traducción), los modelos abiertos ofrecen rendimiento comparable a un costo significativamente menor.

7.7.3 *Frameworks* Agénticos y Agentic-RAG

El ecosistema de *frameworks* para construir sistemas agénticos y Agentic-RAG maduró considerablemente durante el período. Microsoft fusionó AutoGen y Semantic Kernel en el unificado Microsoft Agent Framework (enero de 2026), con disponibilidad general prevista para fines del primer trimestre de 2026. Mastra, un *framework* agéntico basado en TypeScript, alcanzó la versión 1.0 (enero de 2026) y superó 1.77 millones de descargas mensuales en NPM. LangGraph consolidó su posición como herramienta de referencia para orquestación de flujos agénticos multi-paso, mientras que LlamaIndex se especializó en pipelines RAG de producción con soporte nativo para flujos agénticos.

En el ámbito de herramientas especializadas para RAG, RAGFlow evolucionó como motor de RAG de código abierto con soporte para GraphRAG y razonamiento agéntico. RAGAS se estableció como el estándar *de facto* para evaluación de pipelines RAG, proporcionando métricas de relevancia, fidelidad y calidad de respuesta. AutoRAG introdujo la automatización de la optimización de pipelines RAG mediante experimentación sistemática.

Según proyecciones de Gartner, se espera que el 40% de las aplicaciones empresariales incluyan agentes de IA específicos para tareas a fines de 2026, frente a menos del 5% en 2025. El mercado global de agentes de IA pasó de $5.4 mil millones en 2024 a $7.6 mil millones en 2025, con proyecciones de alcanzar $50 mil millones para 2030.

7.7.4 Implicaciones para el Presente Trabajo

Estos desarrollos tienen implicaciones directas para la interpretación de los resultados de esta tesis. En primer lugar, el hallazgo de la sobrecarga agéntica observada en modelos de menor capacidad (gpt-5-nano) podría no reproducirse con los modelos de razonamiento de nueva generación (DeepSeek-R1, Gemini 3.1 Pro), que integran cadenas de pensamiento explícitas capaces de gestionar instrucciones complejas de manera más efectiva. En segundo lugar, la disponibilidad de modelos de código abierto competitivos con las APIs propietarias abre la posibilidad de eliminar la dependencia de APIs comerciales identificada como limitación del sistema, habilitando despliegues locales relevantes para entornos donde la confidencialidad de las consultas legales sea un requisito. Por último,

la maduración de los *frameworks* agénticos simplificaría una futura reimplementación del sistema multi-agente, potencialmente mejorando la eficiencia y modularidad de la arquitectura.

La inclusión de estos desarrollos en el análisis comparativo requeriría experimentación adicional fuera del alcance del presente trabajo; no obstante, la metodología propuesta —diseño factorial, *benchmark* específico y evaluación LLM-as-a-Judge— es directamente aplicable a futuras generaciones de modelos sin modificaciones sustanciales.

7.8 Conclusiones del Capítulo

La revisión de la literatura y de las plataformas existentes evidencia un crecimiento acelerado en la integración de IA al dominio legal, tanto en el ámbito comercial como académico. Las grandes plataformas internacionales (Westlaw, LexisNexis, vLex) han evolucionado desde buscadores por palabras clave hacia sistemas agénticos con RAG, mientras que en Latinoamérica plataformas como Lexius, Juztina, Veredicta y Lexium están adoptando tecnologías similares adaptadas al contexto regional.

Sin embargo, se identifican vacíos significativos: (1) la ausencia de sistemas de código abierto que permitan la evaluación independiente y la replicación de resultados en el ámbito legal argentino; (2) la falta de benchmarks específicos para normativa argentina; y (3) la subrepresentación del español y del derecho civil en la investigación académica sobre NLP legal. El presente trabajo se posiciona en la intersección de estos vacíos, proponiendo un sistema Agentic-RAG de código abierto para el Código de Minería Argentino, acompañado de un benchmark específico que permite su evaluación reproducible.

8 Conclusiones y Trabajos Futuros

8.1 Introducción

El presente trabajo se propuso diseñar, implementar y evaluar un sistema de Generación Aumentada por Recuperación Agéntica (Agentic-RAG) de código abierto para la consulta de normativa argentina, utilizando el Código de Minería Argentino como prueba de concepto. A lo largo de los capítulos precedentes se presentaron los fundamentos teóricos de modelos de lenguaje, sistemas RAG, arquitecturas agénticas y Agentic-RAG; se describió la metodología y el diseño del sistema multi-agente; se detalló la implementación del prototipo completo —desde la extracción de datos hasta el pipeline de evaluación—; y se analizaron los resultados experimentales mediante un diseño factorial de 3 estrategias de recuperación × 3 modelos de lenguaje, evaluados con métricas textuales, semánticas y evaluación LLM-as-a-Judge.

Este capítulo sintetiza los hallazgos principales, revisa el cumplimiento de los objetivos planteados, discute las fortalezas y limitaciones del trabajo y propone líneas de investigación futuras.

8.2 Conclusiones

8.2.1 Cumplimiento de Objetivos

El trabajo planteó cuatro objetivos específicos, cuyo grado de cumplimiento se detalla a continuación:

1. **Desarrollar una base de datos vectorial del Código de Minería Argentino diseñada específicamente para trabajar con sistemas RAG.** Este objetivo se cumplió completamente. Se implementó una base de datos vectorial en Milvus con embeddings de 768 dimensiones generados por el modelo `hiiamsid/sentence_similarity_spanish_es`, empleando una estrategia de segmentación basada en la estructura natural del documento legal (un artículo por fragmento). El índice IVF_FLAT con similitud coseno permite búsquedas semánticas eficientes sobre el corpus del Código de Minería.

2. **Desarrollar un sistema de agentes de inteligencia artificial para que trabajen de manera conjunta con el sistema RAG.** Este objetivo se cumplió completamente. Se implementó una arquitectura multi-agente mediante Pydantic AI compuesta por dos agentes especializados activos en la configuración experimental: el Agente Especialista Legal (generación de respuestas) y el Agente de Recuperación (descomposición de consultas y búsqueda vectorial). Adicionalmente, se diseñó e implementó un tercer agente —el Agente de Calidad de Recuperación— encargado de evaluar la pertinencia de los fragmentos recuperados, pero fue desactivado en la configuración experimental final debido a que su complejidad adicional no se tradujo en mejoras marginales significativas. El sistema registra la trazabilidad completa de las interacciones entre agentes.

3. **Desarrollar un benchmark específico para el Código de Minería Argentino para evaluar el desempeño del sistema.** Este objetivo se cumplió. Se generó un benchmark de 25 preguntas tipo caso de estudio legal (76 subpreguntas explícitas más 3 preguntas evaluadas como unidad única, totalizando 79 unidades de evaluación), distribuidas entre los títulos del Código de Minería, con respuestas esperadas generadas por GPT-4.1 y revisadas por profesionales del derecho argentino. El benchmark fue empleado de manera efectiva como instrumento de evaluación en las nueve configuraciones experimentales.

4. **Cargar el código y una descripción de la metodología a un repositorio abierto para que cualquier organización, empresa o individuo pueda replicar y adaptar el sistema a otras normativas.** Este objetivo se cumplió. El código fue organizado en módulos independientes (notebooks) con documentación técnica en cada directorio, siguiendo el flujo del pipeline experimental, y se publicó en un repositorio público de GitHub.

8.2.2 Contribuciones del Trabajo

Las principales contribuciones del presente trabajo son:

1. **Sistema Agentic-RAG de código abierto para normativa argentina.** Se desarrolló un sistema completo que integra recuperación semántica, base de datos vectorial y agentes inteligentes, constituyendo, hasta donde se tiene conocimiento, el primer sistema de este tipo aplicado al Código de Minería Argentino con código disponible públicamente.

2. **Evidencia empírica sobre la interacción estrategia × modelo.** El hallazgo más relevante del trabajo es la interacción significativa entre la estrategia de recuperación y el modelo de lenguaje ($H = 56.62$, $p < 10^{-11}$), que demuestra que la efectividad de una estrategia depende del modelo con el que se combine. Este resultado tiene implicaciones prácticas directas: no existe una recomendación universal, y la selección de modelo y estrategia debe hacerse de manera conjunta.

3. **Efecto igualador del RAG básico.** Se demostró que el RAG básico elimina las diferencias significativas de desempeño entre modelos ($p = 0.095$), permitiendo que modelos más pequeños y económicos alcancen un desempeño comparable al de modelos superiores cuando se les provee de contexto relevante.

4. **Identificación de la sobrecarga agéntica en modelos pequeños.** Se documentó que el sistema Agentic-RAG degrada el desempeño de modelos de capacidad reducida. El análisis de efectos simples con corrección de Holm-Bonferroni confirma que gpt-5-nano sufre una degradación significativa con Agentic-RAG frente a RAG básico (media de 0.40 vs. 0.59, $p_{\text{corr}} = 0.017$), aunque la diferencia frente a sin RAG no alcanza significación estadística tras la corrección ($p_{\text{corr}} = 0.064$). Esto sugiere que la complejidad de las instrucciones y contextos generados por el sistema agéntico excede la capacidad de procesamiento de estos modelos.

5. **Benchmark para el Código de Minería Argentino.** Se creó un benchmark específico con 25 preguntas (79 unidades de evaluación), elaborado con asistencia de

LLM y validado por profesionales del derecho, que puede emplearse como referencia para futuras evaluaciones de sistemas de consulta legal sobre esta normativa.

6. **Validación de LLM-as-a-Judge para evaluación legal.** Se demostró que la evaluación mediante tres jueces LLM (DeepSeek, Grok, Kimi K2) alcanza un acuerdo inter-juez bueno ($\alpha = 0.81$) para la métrica de «respuesta correcta», validando su uso como herramienta de evaluación en este dominio.

7. **Metodología replicable y documentada.** La documentación completa de la metodología, las decisiones de diseño y los resultados permite la replicación del sistema para otros cuerpos normativos y jurisdicciones.

Cabe destacar que la contribución principal del trabajo no reside en los valores de desempeño absolutos de los modelos evaluados —que reflejan el estado de los sistemas disponibles durante el período experimental— sino en la metodología de evaluación, el benchmark creado y el sistema replicable, que pueden aplicarse sobre cualquier generación futura de modelos.

8.2.3 Impacto de la Propuesta

El presente trabajo demuestra la viabilidad técnica de aplicar sistemas Agentic-RAG de código abierto a la consulta de normativa legal argentina. Los resultados confirman que la incorporación de mecanismos de recuperación de información mejora significativamente la calidad de las respuestas de los modelos de lenguaje en el dominio legal, con mejoras que van desde una mediana global de 0.58 (sin RAG) hasta 0.83 (Agentic-RAG) en la evaluación de «respuesta correcta». Cabe precisar que este beneficio es dependiente del modelo y no uniforme. El análisis de efectos simples con corrección de Holm-Bonferroni revela que Agentic-RAG beneficia significativamente a gpt-5-mini (0.92 frente a 0.33 sin RAG, $p_{\text{corr}} < 0.0001$) pero degrada a gpt-5-nano frente a RAG básico (0.08 vs. 0.58, $p_{\text{corr}} = 0.017$). Para gpt-5, las diferencias entre estrategias no alcanzan significación estadística tras la corrección por comparaciones múltiples ($p_{\text{corr}} > 0.05$ en todos los pares). El RAG básico, aunque con una mejora más modesta (mediana global de 0.71), ofrece el beneficio adicional de igualar el desempeño entre modelos de distinta capacidad.

La disponibilidad del código y la metodología como recurso abierto habilita su adaptación por instituciones públicas, organizaciones de la sociedad civil y desarrolladores independientes que busquen mejorar el acceso ciudadano a la información legal. En un contexto donde las herramientas avanzadas de consulta legal son predominantemente comerciales y de acceso restringido, esta contribución tiene el potencial de reducir las barreras de acceso a la normativa para usuarios no especializados.

8.3 Discusión de Resultados

8.3.1 Fortalezas del Sistema

El sistema desarrollado presenta las siguientes fortalezas:

1. **Mejora significativa con recuperación.** Tanto el RAG básico como el Agentic-RAG superan significativamente al baseline sin RAG ($p < 0.001$ para Agentic-RAG

vs. sin RAG; $p = 0.0066$ para RAG básico vs. sin RAG), confirmando el valor de anclar las respuestas en evidencia documental.

2. **Configuración costo-eficiente viable.** La combinación Agentic-RAG con gpt-5--mini alcanza una media de 0.78 en corrección, comparable al 0.80 de gpt-5, ofreciendo una alternativa práctica para escenarios con restricciones presupuestarias.

3. **Evaluación multidimensional robusta.** La combinación de métricas textuales, semánticas y evaluación por jueces LLM proporciona una visión multidimensional de la calidad, mitigando los sesgos de cada familia de métricas por separado.

4. **Trazabilidad completa.** El sistema de logging registra todas las interacciones entre agentes, herramientas invocadas, fragmentos recuperados y decisiones intermedias, lo que facilita la auditoría y la depuración del sistema.

5. **Análisis estadístico riguroso.** El uso de pruebas no paramétricas justificadas por la verificación de supuestos (Shapiro-Wilk, Kruskal-Wallis, Dunn con Bonferroni, Scheirer-Ray-Hare), complementadas con análisis de efectos simples (Mann-Whitney U con corrección de Holm-Bonferroni y el análisis de acuerdo inter-juez (Krippendorff's alpha), proporcionan una base estadística sólida para las conclusiones.

8.3.2 Debilidades y Limitaciones

El trabajo presenta las siguientes limitaciones que deben considerarse al interpretar los resultados:

1. **Alcance del corpus.** El sistema se limita al Código de Minería Argentino como prueba de concepto, sin incluir otras normativas nacionales, provinciales o municipales. La generalización de los resultados a otros cuerpos normativos requiere validación adicional.

2. **Ausencia de diferencia marginal entre Agentic-RAG y RAG básico.** A nivel global, no se encontró diferencia significativa entre ambas estrategias ($p = 0.383$), aunque el análisis de interacción muestra que esta conclusión enmascara diferencias condicionales al modelo. Este resultado sugiere que, para ciertos modelos, la complejidad adicional del sistema agéntico no se traduce en mejora marginal respecto a un RAG más simple.

3. **Degradación con modelos pequeños.** El sistema Agentic-RAG perjudica el desempeño de gpt-5-nano, lo que constituye una limitación para despliegues con restricciones de costo o en entornos con acceso limitado a modelos de mayor capacidad. El análisis de efectos simples confirma que esta degradación es significativa frente a RAG básico ($p_{\text{corr}} = 0.017$, Holm-Bonferroni) pero marginal frente a sin RAG ($p_{\text{corr}} = 0.064$).

4. **Confiabilidad insuficiente de la métrica de precisión legal.** La rúbrica `score_precision_legal` no alcanzó niveles aceptables de acuerdo inter-juez en todos los subgrupos ($\alpha < 0.667$ en base_rag y no_rag), lo que impidió su uso como

variable de comparación y limitó la evaluación de la precisión en la citación de artículos.

5. **Benchmark de escala limitada.** Aunque las 79 unidades de evaluación proporcionan una base razonable para la evaluación, un benchmark de mayor escala, con mayor diversidad de tipos de preguntas y grados de dificultad, fortalecería la validez externa de los resultados.

6. **Dependencia de APIs comerciales.** El empleo de modelos GPT-5 a través de APIs de pago introduce una dependencia externa y costos recurrentes que podrían limitar la accesibilidad del sistema en el largo plazo.

7. **Ausencia de evaluación con usuarios reales.** No se realizaron pruebas de usabilidad con usuarios finales (abogados, funcionarios, ciudadanos), por lo que no se cuenta con evidencia directa sobre la utilidad percibida y la experiencia de usuario del sistema.

8. **Alcance temporal de los modelos evaluados.** Los modelos de la familia GPT-5 evaluados corresponden al estado de las APIs disponibles durante el período experimental (segundo semestre de 2025). Modelos posteriores —en particular modelos de razonamiento (*reasoning models*) lanzados entre fines de 2025 y 2026 — podrían mostrar patrones de comportamiento distintos, especialmente respecto a la sobrecarga agéntica observada en modelos de menor capacidad (gpt-5-nano). La validez de los resultados se circunscribe al conjunto de modelos evaluados.

9. **Ausencia de validación manual de las evaluaciones LLM-as-a-Judge.** Si bien la sección sobre LLM-as-a-judge del marco teórico (capítulo Sección 4) plantea la validación manual de una muestra de evaluaciones como estrategia para mitigar sesgos del juez LLM, esta validación no pudo realizarse en la práctica. La cantidad de contenido generado por las nueve configuraciones experimentales (tres estrategias de recuperación $\times$ tres modelos de lenguaje) y la falta de disponibilidad de profesionales del derecho con el tiempo necesario para revisar y puntuar manualmente las respuestas impidieron llevar a cabo esta verificación. Como consecuencia, la concordancia entre las evaluaciones automáticas del juez LLM y el criterio de expertos legales no ha sido cuantificada directamente, lo que constituye una limitación importante para la interpretación de los resultados de la evaluación automática.

10. **Ausencia de prompting de cadena de pensamiento (CoT) en los jueces LLM.** Aunque el marco teórico menciona el uso de cadenas de pensamiento como estrategia para reducir sesgos superficiales en la evaluación, los prompts empleados en la implementación práctica no incluyen instrucciones explícitas de razonamiento paso a paso. El análisis del código de evaluación (archivo `01_llm_as_a_judge.ipynb`) revela que los prompts se limitan a presentar la rúbrica con criterios y puntajes, solicitando una justificación textual del puntaje asignado mediante el parámetro `include_reason=True`, pero sin requerir una cadena de razonamiento estructurada antes de emitir la calificación. Esta omisión podría incrementar la susceptibilidad de los jueces a sesgos como el de posición o verbosidad, particularmente en los

modelos DeepSeek-V3 y Kimi K2 0905 que no poseen capacidad de razonamiento nativa. Solo Grok 4.1 Fast cuenta con un modo de razonamiento nativo que podría haberse activado explícitamente.

8.4 Trabajos Futuros

A partir de las limitaciones identificadas y los resultados obtenidos, se proponen las siguientes líneas de investigación futuras:

8.4.1 Extensión del Corpus Legal

La extensión natural del trabajo consiste en ampliar el corpus más allá del Código de Minería para incluir la totalidad de la normativa nacional disponible en Infoleg. Esto implicaría abordar desafíos adicionales como la heterogeneidad de formatos, la mayor profundidad de las estructuras jerárquicas y el volumen significativamente mayor de documentos. En una etapa posterior, podría incorporarse normativa provincial y municipal, así como jurisprudencia y dictámenes.

8.4.2 Mejoras en la Arquitectura

Se identifican varias oportunidades de mejora en la arquitectura del sistema:

- **Políticas adaptativas de complejidad agéntica.** Un agente clasificador que determine la complejidad de la consulta podría decidir dinámicamente si emplear RAG básico o Agentic-RAG, evitando la sobrecarga agéntica para consultas simples y modelos de menor capacidad.
- **Recuperación híbrida.** La combinación de búsqueda vectorial con búsqueda léxica (BM25) y búsqueda sobre grafos de conocimiento podría mejorar la cobertura de la recuperación, especialmente para consultas que requieren coincidencias exactas de términos legales o razonamiento sobre relaciones entre normas.
- **Modelos de embeddings especializados.** El entrenamiento o ajuste fino de modelos de embeddings sobre texto jurídico argentino podría mejorar la calidad de la recuperación semántica, dado que los modelos de propósito general pueden no capturar adecuadamente los matices del lenguaje legal.
- **Modelos de lenguaje de código abierto.** La evaluación de modelos de lenguaje de código abierto (por ejemplo, de las familias Llama, Mistral o Qwen) permitiría reducir la dependencia de APIs comerciales y facilitar el despliegue local, relevante para entornos donde la confidencialidad de las consultas sea un requisito.

8.4.3 Ampliación del Benchmark

El benchmark podría ampliarse en varias direcciones:

- Incrementar el número de preguntas y la diversidad de tipos (factoides, multi-salto, comparativas, hipotéticas).
- Incluir preguntas de mayor dificultad que requieran razonamiento sobre múltiples artículos o cuerpos normativos.
- Desarrollar rúbricas más específicas para la evaluación de precisión legal, incorporando anclas de dominio que mejoren el acuerdo inter-juez.

- Incorporar la evaluación de la capacidad de abstención del sistema (es decir, su habilidad para reconocer cuando no tiene información suficiente para responder).

8.4.4 Integración con Sistemas Existentes

La integración del sistema con plataformas como Infoleg o SAIJ permitiría acceder a información actualizada de manera automática y ofrecer una interfaz de consulta más completa. Asimismo, la integración con grafos de conocimiento que modelen las relaciones entre normas (derogaciones, modificaciones, referencias cruzadas) habilitaría un razonamiento más profundo sobre la estructura del ordenamiento jurídico.

8.4.5 Evaluación con Usuarios Reales

Una línea de trabajo complementaria consiste en la realización de estudios de usabilidad con usuarios finales, incluyendo profesionales del derecho, funcionarios públicos y ciudadanos sin formación jurídica. Estos estudios permitirían evaluar la utilidad percibida del sistema, identificar patrones de uso y necesidades no cubiertas, y ajustar la interfaz y las respuestas del sistema a las expectativas de los distintos perfiles de usuario.

8.4.6 Optimización del Entrenamiento con Aprendizaje por Refuerzo

Los avances recientes en optimización de políticas Agentic-RAG mediante aprendizaje por refuerzo con supervisión de proceso (como RAG-Gym, ReasonRAG y TreePS-RAG) abren la posibilidad de entrenar agentes que aprendan a seleccionar estrategias de recuperación óptimas para cada tipo de consulta, superando las políticas manuales basadas en prompts empleadas en este trabajo.

8.4.7 Re-evaluación sobre Modelos de Nueva Generación

La velocidad de iteración en el desarrollo de LLMs hace deseable repetir el diseño experimental sobre modelos lanzados después del período de estudio, especialmente modelos de razonamiento que integran cadenas de pensamiento explícitas (*chain-of--thought*). Estos modelos podrían alterar la dinámica de interacción estrategia × modelo documentada en este trabajo, en particular reduciendo o eliminando la degradación observada en el modo Agentic-RAG con modelos de menor capacidad. La metodología de evaluación propuesta —diseño factorial, benchmark específico y validación mediante LLM-as-a-Judge— es directamente aplicable a futuras generaciones de modelos sin modificaciones sustanciales.

8.5 Reflexiones Finales

El presente trabajo demuestra que los sistemas de Generación Aumentada por Recuperación Agéntica constituyen una herramienta viable para mejorar el acceso a la normativa legal argentina. La combinación de modelos de lenguaje con mecanismos de recuperación semántica y agentes especializados permite generar respuestas más precisas y contextualizadas que las obtenidas por modelos sin acceso a información externa.

El hallazgo de la interacción significativa entre la estrategia de recuperación y el modelo de lenguaje subraya la importancia de un enfoque integral en el diseño de estos sistemas: la selección de una estrategia de recuperación no puede realizarse de manera independiente del modelo que la utilizará. El análisis de efectos simples con corrección

de Holm-Bonferroni confirma que Agentic-RAG beneficia significativamente a gpt-5--mini pero degrada a gpt-5-nano frente a RAG básico, mientras que para gpt-5 no se detectan diferencias significativas entre estrategias. Este resultado es transferible a otros dominios y aplicaciones de RAG, y constituye un aporte al entendimiento de cómo interactúan los componentes de estos sistemas.

La publicación del código, la metodología y el benchmark como recursos abiertos busca facilitar que otros investigadores, instituciones y desarrolladores puedan replicar, adaptar y mejorar el sistema para otras normativas y jurisdicciones. En un país donde el acceso a herramientas avanzadas de consulta legal permanece en gran medida restringido a plataformas comerciales, la disponibilidad de una alternativa abierta y documentada representa un paso concreto hacia la democratización del acceso a la información legal.

Referencias

Achanta, A. (2025, septiembre). *Agentic RAG: Embedding Autonomous Agents into Retrieval-Augmented Generation*. https://www.computer.org/publications/tech-news/trends/agentic-rag

Anthropic. (2023,). *Introducing Claude*. https://www.anthropic.com/news/introducing-claude

Anthropic. (2025a,). *Introducing Claude 4*. https://www.anthropic.com/news/claude-4

Anthropic. (2025b, noviembre). *Effective Harnesses for Long-Running Agents*. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

Antimundo. (2026,). *Archivo:Red neuronal artificial.svg*. https://commons.wikimedia.org/wiki/File:Red_neuronal_artificial.svg

Ariai, F., Mackenzie, J., & Demartini, G. (2025). Natural Language Processing for the Legal Domain: A Survey of Tasks, Datasets, Models, and Challenges. *ACM Computing Surveys*, *58*(6), 1-37. https://doi.org/10.1145/3777009

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511*. https://arxiv.org/abs/2310.11511

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., & others. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*.

Bai, Y., Ying, J., Cao, Y., Lv, X., He, Y., Wang, X., Yu, J., Zeng, K., Xiao, Y., Lyu, H., & others. (2023). Benchmarking Foundation Models with Language-Model-as-an-Examiner. *Advances in Neural Information Processing Systems (NeurIPS)*, *36*.

Bandara, E., Gore, R., Foytik, P., Shetty, S., Mukkamala, R., Rahman, A., Liang, X., Bouk, S. H., & others. (2025, diciembre). *A Practical Guide for Designing, Developing, and Deploying Production-Grade Agentic AI Workflows*. https://arxiv.org/abs/2512.08769

Baron, J. R., Lewis, D. D., & Oard, D. W. (2006,). TREC 2006 Legal Track Overview. *The Fifteenth Text REtrieval Conference Proceedings (TREC 2006)*. https://trec.nist.gov/pubs/trec15/papers/LEGAL06.OVERVIEW.pdf

Basheer, I. A., & Hajmeer, M. N. (2000). Artificial neural networks: fundamentals, computing, design, and application. *Journal of Microbiological Methods*, *43*(1), 3-31.

Bavaresco, A., Bernardi, R., Bertolazzi, L., Elliott, D., Fernández, R., Gatt, A., Ghaleb, E., Giulianelli, M., Hanna, M., Koller, A., & others. (2024,). *LLMs Instead of Human Judges? A Large Scale Empirical Study Across 20 NLP Evaluation Tasks*.

Belcic, I., & Stryker, C. (2025,). *What Is Agentic RAG?*. https://www.ibm.com/think/topics/agentic-rag

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, *3*, 1137-1155. https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, *5*(2), 157-166.

Betancur Sánchez, D., & others. (2025). MEL: Legal Spanish Language Model. *arXiv preprint arXiv:2501.16011*. https://arxiv.org/abs/2501.16011

Bhatia, G., Mubarak, H., Jarrar, M., Mikros, G., Zaraket, F., Alhirthani, M., Al-Khatib, M., Cochrane, L., & others. (2026, enero). *From RAG to Agentic RAG for Faithful Islamic Question Answering*. https://arxiv.org/abs/2601.07528

Bommasani, R., Davis, K., & Cardon, C. (2023). Measuring and Mitigating Gender Bias in Legal Contextualized Language Models. *ACM Transactions on Computing and Sustainable Societies*, *1*(1), 1-33. https://doi.org/10.1145/3628602

Bonferroni, C. E. (1936). *Teoria statistica delle classi e calcolo delle probabilità*. Pubblicazioni del R. Istituto Superiore di Scienze Economiche e Commerciali di Firenze.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., & others. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, *33*, 1877-1901. https://arxiv.org/abs/2005.14165

Cao, Y., Hong, S., Li, X., Ying, J., & Ma, Y. (2025). Toward Generalizable Evaluation in the LLM Era: A Survey Beyond Benchmarks. *arXiv preprint arXiv:2504.18838*. https://arxiv.org/abs/2504.18838v1

Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). Legal-BERT: The Muppets straight out of Law School. *arXiv preprint arXiv:2010.02559*. https://arxiv.org/abs/2010.02559

Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., & Androutsopoulos, I. (2021). LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, *1*, 4310-4330. https://arxiv.org/abs/2110.00976v4

Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., & Liu, Z. (2023,). *ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate*.

Chen, G. H., Chen, S., Liu, Z., Jiang, F., & Wang, B. (2024). Humans or LLMs as the Judge? A Study on Judgement Biases. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 8301-8327.

Chen, S. F., & Goodman, J. (1999). An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, *13*(4), 359-393. https://doi.org/10.1006/csla.1999.0128

Chen, Y., Zhang, E., Hu, T., Wang, S., Yang, Z., Zhong, M., Wei, X., Gao, Y., Wu, Y., & others. (2026, enero). *JADE: Bridging the Strategic-Operational Gap in Dynamic Agentic RAG*. https://arxiv.org/abs/2601.21916

Chinchor, N. (1992). MUC-4 Evaluation Metrics. *Proceedings of the 4th Conference on Message Understanding (MUC-4)*, 22-29. https://doi.org/10.3115/1072064.1072067

Cho, K., Merriënboer, B. van, Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *Proceedings of EMNLP*.

Colvin, S., & others. (2024,). *Pydantic AI: Agent Framework, the Pydantic way*. Pydantic. https://github.com/pydantic/pydantic-ai

Conley, A. C. (2024). Legal Education's Role in Combating Automation Bias and Complacency with Generative AI in the Practice of Law. *KiLaw Journal*. https://scholarworks.umt.edu/faculty_lawreviews/1232

Cruzes, S. (2026). AI Infrastructure Sovereignty. *arXiv preprint arXiv:2602.10900*. https://arxiv.org/abs/2602.10900

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, *2*(4), 303-314. https://doi.org/10.1007/BF02551274

Dahl, M., Magesh, V., Suzgun, M., & Ho, D. E. (2024). Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models. *Journal of Legal Analysis*, *16*(1), 64-93. https://doi.org/10.1093/jla/laae003

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*, *35*.

DeepSeek-AI. (2024,). *DeepSeek-V3 Technical Report*. https://arxiv.org/abs/2412.19437

DeepSeek-AI. (2025,). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. https://arxiv.org/abs/2501.12948

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314*. https://arxiv.org/abs/2305.14314

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*. https://arxiv.org/abs/1810.04805

Du, M., Xu, B., Zhu, C., Wang, S., Wang, P., Wang, X., & Mao, Z. (2026, febrero). *A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces*. https://arxiv.org/abs/2602.03442

Dunn, O. J. (1964). Multiple Comparisons Using Rank Sums. *Technometrics*, *6*(3), 241-252. https://doi.org/10.1080/00401706.1964.10490181

Ehtesham, A., Singh, A., Gupta, G., & Kumar, S. (2025). A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *arXiv preprint arXiv:2505.02279*. https://arxiv.org/abs/2505.02279v2

Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, *14*(2), 179-211.

Emsi. (2024, 11). *Mamba: Revolutionizing Sequence Modeling with Selective State Spaces*. https://www.emsi.me/emsi/article/mamba-revolutionizing-sequence-modeling--with-selective-state-spaces/2024-01-11/173a22

Es, S., James, J., Anke, L. E., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, 150-158.

fdeloche. (2017b,). *File:Gated Recurrent Unit.svg*. https://commons.wikimedia.org/wiki/File:Gated_Recurrent_Unit.svg

fdeloche. (2017a,). *File:Long Short-Term Memory.svg*. https://commons.wikimedia.org/wiki/File:Long_Short-Term_Memory.svg

Fei, Z., Shen, X., Zhu, D., & others. (2024,). LawBench: Benchmarking Legal Knowledge of Large Language Models. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2309.16289

Ferrazzi, P., Cvjeticanin, M., Piraccini, A., & Giannuzzi, D. (2026, enero). *Is Agentic RAG Worth It? An Experimental Comparison of RAG Approaches*. https://arxiv.org/abs/2601.07711

Ffets. (2026,). *Ficheiro:Recurrent neural network unfold.svg*. https://gl.wikipedia.org/wiki/Ficheiro:Recurrent_neural_network_unfold.svg

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*. https://arxiv.org/abs/2312.10997v5

Garvin, P. L. (1967). The Georgetown-IBM Experiment of 1954: An Evaluation in Retrospect. En *Papers in Linguistics in Honor of Léon Dostert* (pp. 46-56). De Gruyter. https://doi.org/10.1515/9783111675886-006

Gosmar, D., & Dahl, D. (2025). Hallucination Mitigation using Agentic AI Natural Language-Based Frameworks. *arXiv preprint arXiv:2501.13946*. https://arxiv.org/abs/2501.13946v1

Govind Srinivasan, A., George, R. J., Joe, J. K., Kant, H., Harshith M R, Sundar, S., Suresh, S., Vimalkanth, R., & others. (2025,). Enhancing Financial RAG with

Agentic AI and Multi-HyDE: A Novel Approach to Knowledge Retrieval and Hallucination Reduction. *Proceedings of the FinNLP Workshop at EMNLP 2025*. https://arxiv.org/abs/2509.16369

Grayson, S. (2017,). *Archivo:T-SNE visualisation of word embeddings generated using 19th century literature.png*. https://es.wikipedia.org/wiki/Archivo:T-SNE_visualisation_of_word_embeddings_generated_using_19th_century_literature.png

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*.

Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Wang, Y., Gao, W., Ni, L., & Guo, J. (2024,). *A Survey on LLM-as-a-Judge*.

Guha, N., Nyarko, J., Ho, D., Ré, C., & Chilton, A. (2023). LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, *36*, 19. https://arxiv.org/abs/2308.11462v1

Hariharan, M., Arvapalli, S., Barma, S., & Sheela, E. (2025, octubre). *Agentic RAG for Software Testing with Hybrid Vector-Graph and Multi-Agent Orchestration*. https://arxiv.org/abs/2510.10824

Hashimoto, M. (2026, febrero). *My AI Adoption Journey*. https://mitchellh.com/writing/my-ai-adoption-journey

Hayes, A. F., & Krippendorff, K. (2007). Answering the Call for a Standard Reliability Measure for Coding Data. *Communication Methods and Measures*, *1*(1), 77-89. https://doi.org/10.1080/19312450709336664

He, Z., Cao, P., Wang, C., & others. (2024). AgentsCourt: Building Judicial Decision-Making Agents with Court Debate Simulation and Legal Knowledge Augmentation. *Findings of the Association for Computational Linguistics: EMNLP 2024*, 9399-9416.

Hebb, D. O. (1949). *The Organization of Behavior: A Neuropsychological Theory*. John Wiley & Sons. https://doi.org/10.4324/9781410612403

Hindi, M., Alwarafy, A., & others. (2025). Enhancing the Precision and Interpretability of Retrieval-Augmented Generation (RAG) in Legal Technology: A Survey. *IEEE Access*. https://doi.org/10.1109/ACCESS.2025.3550145

Hochreiter, S. (1998). The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, *6*(2), 107-116.

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, *9*(8), 1735-1780.

Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, *6*(2), 65-70. https://doi.org/10.2307/4615733

Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, *79*(8), 2554-2558.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, *2*(5), 359-366. https://doi.org/10.1016/0893-6080(89)90020-8

Horty, J. F. (1960,). *Application of Information Retrieval Techniques to Legal Research.* https://tlp.law.pitt.edu/ojs/tlp/article/download/212/216/494

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*. https://arxiv.org/abs/2106.09685

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, *9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Hutchins, W. J. (2004). The Georgetown-IBM Experiment Demonstrated in January 1954. En *Machine Translation: From Real Users to Research* (pp. 102-114). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-30194-3_12

IBM Institute for Business Value. (2026). *The Calculus of AI Sovereignty* [Technical report]. https://www.ibm.com/thought-leadership/institute-business-value/en-us/report/ai-sovereignty

Isbarov, J., & Huseynova, K. (2024, agosto). Enhanced document retrieval with topic embeddings. *18th IEEE International Conference on Application of Information and Communication Technologies (AICT 2024)*. https://arxiv.org/abs/2408.10435v1

Islam, S., Lauscher, A., & Glavaš, G. (2025). How Much Do LLMs Hallucinate across Languages? On Multilingual Estimation of LLM Hallucination in the Wild. *arXiv preprint arXiv:2502.12769*. https://arxiv.org/abs/2502.12769v2

Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et du Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, *37*, 547-579.

Jacobs, R. A., Jordan, M. I., Nowlan, S. J., & Hinton, G. E. (1991). Adaptive Mixtures of Local Experts. *Neural Computation*, *3*(1), 79-87.

Jain, S., Keshava, V., Sathyendra, S. M., Fernandes, P., Liu, P., Neubig, G., & Zhou, C. (2023,). *Multi-dimensional Evaluation of Text Summarization with In-Context Learning.*

Jeong, C. (2024). A Study on the Implementation Method of an Agent-Based Advanced RAG System Using Graph. *Knowledge Management Research*, *25*(3). https://doi.org/10.15813/kmr.2024.25.3.005

Jing, Z., Su, Y., Han, Y., Yuan, B., & Xu, H. (2024). When Large Language Models Meet Vector Databases: A Survey. *arXiv preprint arXiv:2402.01763*, 7-13. https://arxiv.org/abs/2402.01763v4

Jordan, M. I. (1986). Serial order: A parallel distributed processing approach. *ICS Report 8604*.

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computationa Linguistics, and Speech Recognition, with Language Models* (3rd ed.). https://web.stanford.edu/~jurafsky/slp3/

Juztina. (2025,). *Juztina - Asistente legal con inteligencia artificial*.

Khanda, R. (2024, diciembre). *Agentic AI-Driven Technical Troubleshooting for Enterprise Systems: A Novel Weighted Retrieval-Augmented Generation Paradigm*. https://arxiv.org/abs/2412.12006

Kim, S., Shin, J., Cho, Y., Jang, J., Longpre, S., Lee, H., Yun, S., Shin, S., Kim, S., Thorne, J., & others. (2024a,). Prometheus: Inducing Fine-grained Evaluation Capability in Language Models. *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*.

Kim, S., Suk, J., Longpre, S., Lin, B. Y., Shin, J., Welleck, S., Neubig, G., Lee, M., Lee, K., & Seo, M. (2024b). Prometheus 2: An Open Source Language Model Specialized in Evaluating Other Language Models. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 4334-4353.

Kimi Team. (2025,). *Kimi K2: Open Agentic Intelligence*. https://arxiv.org/abs/2507.20534

Kirkovska, A., & Vargas, D. (2025, diciembre). *The 2026 Guide to AI Agent Workflows*. https://www.vellum.ai/blog/agentic-workflows-emerging-architectures--and-design-patterns

Klem, S., & Al Moubayed, N. (2025). LLMs for LLMs: A Structured Prompting Methodology for Long Legal Documents. *arXiv preprint arXiv:2509.02241*. https://arxiv.org/abs/2509.02241

Kolesnikov, V. (2026, febrero). *Building Agents with Retrieval-Augmented Generation*. https://codelabs.developers.google.com/codelabs/production-ready-ai--with-gc/7-advanced-agent-capabilities/building-agents-with-retrieval-augmented--generation

Koo, R., Lee, M., Raheja, V., Park, J. I., Kim, Z. M., & Kang, D. (2023,). *Benchmarking Cognitive Biases in Large Language Models as Evaluators*.

Kotoge, R., Nishimura, M., & Ma, J. (2025, agosto). *Can Compact Language Models Search Like Agents? Distillation-Guided Policy Optimization for Preserving Agentic RAG Capabilities*. https://arxiv.org/abs/2508.20324

Krippendorff, K. (2018). *Content Analysis: An Introduction to Its Methodology* (4.ª ed.). SAGE Publications.

Kruskal, W. H., & Wallis, W. A. (1952). Use of Ranks in One-Criterion Variance Analysis. *Journal of the American Statistical Association*, *47*(260), 583-621. https://doi.org/10.1080/01621459.1952.10483441

Lambert, N., Pyatkin, V., Morrison, J., Miranda, L., Lin, B. Y., Chandu, K., Dziri, N., Kumar, S., Zick, T., Choi, Y., & others. (2024,). *RewardBench: Evaluating Reward Models for Language Modeling.*

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436-444. https://doi.org/10.1038/nature14539

Levenshtein, V. I. (1966). Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics Doklady*, *10*(8), 707-710.

LexisNexis. (2025,). *Lexis+ with Protégé - AI Legal Research and Drafting.*

Lexium. (2025,). *Lexium - IA de Gestión Jurídica Especializada en Argentina.*

Lexius. (2025,). *Lexius - Plataforma de consulta legal con IA.*

Li, D., Jiang, B., Huang, L., Beigi, A., Zhao, C., Tan, Z., Bhattacharjee, A., Jiang, Y., Chen, C., Wu, T., Shu, K., Cheng, L., & Liu, H. (2024a,). *From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge.*

Li, D., Yan, J., Zhang, T., Wang, C., He, X., Huang, L., Xue, H., & Huang, J. (2024b,). *DALK: Dynamic Active Learning for Knowledge Retrieval in LLMs.*

Li, H. et al. (2024,). *LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods.*

Li, J. et al. (2023,). *Generative Judge for Evaluating Alignment.*

Li, R. et al. (2023,). *PRD: Peer Rank and Discussion Improve Large Language Model Based Evaluations.*

Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 74-81.

Lin, Y.-T., & Chen, Y.-N. (2023,). *LLM-Eval: Unified Multi-Dimensional Automatic Evaluation for Open-Domain Conversations with Large Language Models.*

Liu, C.-W., Lowe, R., Serban, I. V., Noseworthy, M., Charlin, L., & Pineau, J. (2016). How NOT to Evaluate Your Dialogue System: An Empirical Study of Unsupervised Evaluation Metrics for Dialogue Response Generation. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2122-2132.

Liu, S., Li, X., Liu, Z., Yan, Y., Yang, C., Zeng, Z., Liu, Z., Sun, M., & Yu, G. (2025,). *Judge as A Judge: Improving the Evaluation of Retrieval-Augmented Generation through the Judge-Consistency of Large Language Models.*

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023a). G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2511-2522.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023b,). *Large Language Models are not Fair Evaluators*.

Loffredo, J. R., & Yun, S. (2025, marzo). *Agent-Enhanced Large Language Models for Researching Political Institutions*. https://arxiv.org/abs/2503.13524

Markov, A. A. (1913). An example of statistical investigation of the text Eugene Onegin concerning the connection of samples in chains. *Izvestiya Imperatorskoi Akademii Nauk*.

Markov, A. A. (2006). An example of statistical investigation of the text Eugene Onegin concerning the connection of samples in chains. *Science in Context*, *19*(4), 591-600.

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, *5*(4), 115-133. https://doi.org/10.1007/BF02478259

Microsoft. (2026, enero). *Retrieval-Augmented Generation (RAG) in Azure AI Search*. https://learn.microsoft.com/en-us/azure/search/retrieval-augmented--generation-overview

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. https://arxiv.org/abs/1301.3781

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 12076-12100.

Ministerio de Justicia de la Nación. (2025,). *Infoleg - Información Legislativa y Documental*.

Ministerio de Justicia y Derechos Humanos. (2025,). *SAIJ - Sistema Argentino de Información Jurídica*.

Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. https://doi.org/10.7551/mitpress/4943.001.0001

Mishra, S., Niroula, S., Yadav, U., Thakur, D., Gyawali, S., & Gaire, S. (2026, marzo). *SoK: Agentic Retrieval-Augmented Generation (RAG): Taxonomy, Architectures, Evaluation, and Research Directions*. https://arxiv.org/abs/2603.07379

Mnasri, M. (2019). Recent advances in conversational NLP: A survey. *arXiv preprint arXiv:1912.06038*.

Niklaus, J., & others. (2025,). *LEXam: Benchmarking Legal Reasoning on 340 Law Exams*. https://arxiv.org/abs/2505.12864

Niklaus, J., Matoshi, V., Rani, P., Galassi, A., & Stürmer, M. (2024). LEXTREME: A Multi-Lingual and Multi-Task Benchmark for the Legal Domain. *Findings of the Association for Computational Linguistics: EMNLP 2023*, 3016-3054. https://doi.org/10.18653/v1/2023.findings-emnlp.200

OpenAI. (2022, noviembre 30). *Introducing ChatGPT*. https://openai.com/index/chatgpt/

OpenAI. (2023,). *GPT-4 Technical Report*. https://arxiv.org/abs/2303.08774

OpenAI. (2024,). *Learning to Reason with LLMs*. https://openai.com/index/learning-to-reason-with-llms/

OpenAI. (2025a,). *GPT-4.5 System Card*. https://openai.com/index/gpt-4-5-system-card/

OpenAI. (2025b,). *Introducing Codex*. https://openai.com/index/introducing-codex/

OpenAI. (2025c,). *Introducing GPT-5*. https://openai.com/index/introducing-gpt-5/

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., & others. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, *35*.

Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311-318.

Park, J., Jwa, S., Ren, M., Kim, D., & Choi, S. (2024,). *OffsetBias: Leveraging Debiased Data for Tuning Evaluators*.

Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *Proceedings of the 30th International Conference on Machine Learning*.

Patronus AI. (2025,). *Agentic RAG: Tutorial and Examples*. https://www.patronus.ai/ai-agent-development/agentic-rag

Peng, B., & Ma, Y. (2019). A Survey of Information Retrieval Based on Chatbots. *Journal of Physics: Conference Series*, *1187*(5), 52069.

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532-1543. https://aclanthology.org/D14-1162/

Pipitone, N., & Alami, G. (2024). LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. *arXiv preprint arXiv:2408.10343*. https://arxiv.org/abs/2408.10343v1

Polars Development Team. (2024,). *Polars: Blazingly Fast DataFrames in Rust and Python*. https://pola.rs/

Press, O., Smith, N. A., & Lewis, M. (2022,). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=R8sQPpGCv0

Python Software Foundation. (2023,). *Python Language Reference, Version 3.12*. https://www.python.org/

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). *Improving language understanding by generative pre-training* [Technical report]. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., & others. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*.

Raeini, M. G. (2025). The Evolution of Language Models: From N-Grams to LLMs, and Beyond. *Natural Language Processing Journal*, *12*, 100168. https://doi.org/10.1016/j.nlp.2025.100168

Raina, V., Liusie, A., & Gales, M. (2024,). *Is LLM-as-a-Judge Robust? Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment*.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982-3992. https://doi.org/10.18653/v1/D19-1410

Reimers, N., & Gurevych, I. (2020). Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 4512-4525. https://doi.org/10.18653/v1/2020.emnlp-main.365

Rogers, C. R. (1951). *Client-Centered Therapy: Its Current Practice, Implications, and Theory*. Houghton Mifflin.

Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, *65*(6), 386-408.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1985). Learning Internal Representations by Error Propagation. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, *1*, 318-362.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986a). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533-536. https://doi.org/10.1038/323533a0

Rumelhart, D. E., McClelland, J. L., & Group, P. R. (1986b). *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1: Foundations*. MIT Press.

Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed., p. 17). Pearson.

Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2024). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 338-354.

Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2024,). *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. https://arxiv.org/abs/2402.07927

Salve, A., Attar, S., Deshmukh, M., Shivpuje, S., & Mitra Utsab, A. (2024, diciembre). *A Collaborative Multi-Agent Approach to Retrieval-Augmented Generation Across Diverse Data*. https://arxiv.org/abs/2412.05838

Samy, D., & others. (2020,). Legal-ES: A Set of Large Scale Resources for Spanish Legal Text Processing. *Proceedings of Workshop on Language Technologies in Government and Public Administration (LT4Gov 2020), co-located with LREC 2020*. https://aclanthology.org/2020.lt4gov-1.6/

Scheirer, C. J., Ray, W. S., & Hare, N. (1976). The Analysis of Ranked Data Derived from Completely Randomized Factorial Designs. *Biometrics*, *32*(2), 429-434. https://doi.org/10.2307/2529511

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*.

Schulhoff, S., Ilie, M., Balepur, N., Kahadze, K., Liu, A., Si, C., Li, Y., Gupta, A., Han, H., & others. (2024,). *The Prompt Report: A Systematic Survey of Prompting Techniques*. https://arxiv.org/abs/2406.06608

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference*, 92-96. https://doi.org/10.25080/majora-92bf1922-011

Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, *27*(3), 379-423.

Shapiro, S. S., & Wilk, M. B. (1965). An Analysis of Variance Test for Normality (Complete Samples). *Biometrika*, *52*(3/4), 591-611. https://doi.org/10.2307/2333709

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously Large Neural Networks: Sparsely-Gated Mixture-of-Experts Layers. *arXiv preprint arXiv:1701.06538*.

Shekhar, S., Dubey, T., Mukherjee, K., Saxena, A., Tyagi, A., & Kotla, N. (2024). Towards Optimizing the Costs of LLM Usage. *arXiv preprint arXiv:2402.01742*. https://arxiv.org/abs/2402.01742v1

Shi, J., Yuan, Z., Liu, Y., Huang, Y., Zhou, P., Sun, L., & Gong, N. Z. (2024,). *Optimization-based Prompt Injection Attack to LLM-as-a-Judge*.

Shorten, E., & Monigatti, L. (2024, noviembre). *What Is Agentic RAG? From LLM RAG to AI Agents*. https://weaviate.io/blog/what-is-agentic-rag

Shum, H.-Y., He, X., & Li, D. (2018). From Eliza to Xiaoice: Challenges and opportunities with social chatbots. *Frontiers of Information Technology & Electronic Engineering*, *19*(1), 10-22.

Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. *Findings of the Association for Computational Linguistics: EMNLP 2021*, 3784-3803. https://arxiv.org/abs/2104.07567v1

Siddhartha, H. I. (2023,). *hiiamsid/sentence_similarity_spanish_es: Sentence-Transformers model for Spanish semantic similarity*. https://huggingface.co/hiiamsid/sentence_similarity_spanish_es

Siino, M., Falco, M., Croce, D., & Rosso, P. (2025). Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches. *IEEE Access*, *13*, 18253-18276.

Silveira, R., Ponte, C., Almeida, V., Pinheiro, V., & Furtado, V. (2023). LegalBert-pt: A Pretrained Language Model for the Brazilian Portuguese Legal Domain. *Intelligent Systems: 12th Brazilian Conference, BRACIS 2023, Proceedings, Part III*, *14197*. https://doi.org/10.1007/978-3-031-45392-2_18

Singh, A., Ehtesham, A., Kumar, S., & Khoei, T. T. (2025). Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. *arXiv preprint arXiv:2501.09136*. https://arxiv.org/abs/2501.09136v3

Spearman, C. (1904). The Proof and Measurement of Association between Two Things. *The American Journal of Psychology*, *15*(1), 72-101. https://doi.org/10.2307/1412159

Su, J., Ahmed, M., Lu, Y., Pan, B., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, *568*, 127063. https://doi.org/10.1016/j.neucom.2023.127063

Sutskever, I., Vinyals, O., & Le, Q. V. (2014,). *Sequence to Sequence Learning with Neural Networks*. arXiv. https://doi.org/10.48550/ARXIV.1409.3215

Tan, S., Zhuang, S., Montgomery, K., Tang, W. Y., Cuadron, A., Wang, C., Popa, R. A., & Stoica, I. (2024,). *JudgeBench: A Benchmark for Evaluating LLM-based Judges*.

Thompson, B., Mathur, N., Deutsch, D., & Khayrallah, H. (2024). Improving Statistical Significance in Human Evaluation of Automatic Metrics via Soft Pairwise Accuracy. *Proceedings of the Ninth Conference on Machine Translation (WMT)*, 1222-1234.

Thomson Reuters. (2025,). *Westlaw Precision with CoCounsel - AI-Assisted Legal Research*.

Tian, F., Fang, J., Ganguly, D., Meng, Z., & Macdonald, C. (2025, julio). *Am I on the Right Track? What Can Predicted Query Performance Tell Us about the Search Behaviour of Agentic RAG*. https://arxiv.org/abs/2507.10411

Tribunal Fiscal de la Nación. (2025,). *Búsqueda de jurisprudencia asistida por IA*.

Tsitsulin, A., Munkhoeva, M., & Perozzi, B. (2023). Unsupervised Embedding Quality Evaluation. *Proceedings of Machine Learning Research*, *221*, 169-188. https://arxiv.org/abs/2305.16562v2

Tunstall, L., Werra, L. von, & Wolf, T. (2022). *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. O'Reilly Media, Inc.

Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, *59*(236), 433-460. https://doi.org/10.1093/mind/LIX.236.433

various. (2025,). Towards Trustworthy Legal AI: A Multi-Agent Approach to Integrating Legislative Knowledge. *Lecture Notes in Computer Science*. https://doi.org/10.1007/978-3-032-09318-9_1

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, *30*.

Veredicta. (2025,). *Veredicta - Software legal con IA para abogados argentinos*.

Virtanen, P., & others. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261-272. https://doi.org/10.1038/s41592-019-0686-2

vLex. (2025,). *Vincent AI - Legal AI Engineered for Lawyers*.

Vu, T., Krishna, K., Alzubi, S., Tar, C., Faruqui, M., & Sung, Y.-H. (2024,). *Foundational Autoraters: Taming Large Language Models for Better Automatic Evaluation*.

Wan, H., & others. (2025). From single-agent to multi-agent: a comprehensive review of LLM-based legal agents. *AI Agent*. https://doi.org/10.1016/j.aiagent.2025.06

Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., & others. (2021). Milvus: A Purpose-Built Vector Data Management System. *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2614-2627. https://doi.org/10.1145/3448016.3457550

Wang, P. et al. (2024). Large Language Models are not Fair Evaluators. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 9440-9450.

Wang, T. et al. (2024,). *Self-Taught Evaluators*.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., & Fedus, W. (2022a). Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2206.07682

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022b). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, *35*, 24824-24837. https://arxiv.org/abs/2201.11903

Weizenbaum, J. (1966). ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine. *Communications of the ACM*, *9*(1), 36-45.

Winograd, T. (1972). Understanding natural language. *Cognitive Psychology*, *3*(1), 1-191.

World Law Group. (2025,). *Argentina: National Tax Court Launches Its Innovative AI-Assisted Search Tool*.

xAI. (2025,). *Grok 4.1 Fast: Pushing the Frontier of Cost-Efficient Intelligence*. https://x.ai/news/grok-4-1-fast

Xiong, G., Jin, Q., Wang, X., Fang, Y., Liu, H., Yang, Y., Chen, F., Song, Z., Wang, D., & others. (2025, febrero). *RAG-Gym: Systematic Optimization of Language Agents for Retrieval-Augmented Generation*. https://arxiv.org/abs/2502.13957

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023b). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*, *36*, 11809-11822.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023a,). ReAct: Synergizing Reasoning and Acting in Language Models. *Proceedings of the International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629

Yousefi Maragheh, R., Vadla, P., Gupta, P., Zhao, K., Inan, A., Yao, K., Xu, J., Kanumala, P., & others. (2025, junio). *ARAG: Agentic Retrieval Augmented Generation for Personalized Recommendation*. https://arxiv.org/abs/2506.21931

Zeggai, A., Traikia, I., Lakehal, A., & Boulesnane, A. (2025, julio). *AI-VaxGuide: An Agentic RAG-Based LLM for Vaccination Decisions*. https://arxiv.org/abs/2507.03493

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020,). BERTScore: Evaluating Text Generation with BERT. *International Conference on Learning Representations*.

Zhang, T., Li, K., Li, J., Li, Y., Luo, H., Wu, X., Glass, J., & Meng, H. (2026, enero). *TreePS-RAG: Tree-based Process Supervision for Reinforcement Learning in Agentic RAG*. https://arxiv.org/abs/2601.06922

Zhang, W., Li, X., Dong, K., Wang, Y., Jia, P., Li, X., Zhang, Y., Xu, D., Du, Z., & others. (2025, mayo). *Process vs.\ Outcome Reward: Which Is Better for Agentic RAG Reinforcement Learning*. https://arxiv.org/abs/2505.14069

Zhao, W. X., & others. (2026). A Survey of Large Language Models. *Springer Nature Computer Science / Journal of Computer Science and Technology*. https://doi.org/10.1007/s11704-026-60308-3

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM--as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, *36*, 46595-46623.

Zheng, Q., Sun, Y., Wu, C., Zhao, W., Qiu, P., Yu, Y., Sun, K., Zhang, J., Wang, Y., & others. (2025, agosto). *End-to-End Agentic RAG System Training for Traceable Diagnostic Reasoning*. https://arxiv.org/abs/2508.15746

Zhong, Z., Liu, H., Cui, X., Zhang, X., & Qin, Z. (2024). Mix-of-Granularity: Optimize the Chunking Granularity for Retrieval-Augmented Generation. *Proceedings of the International Conference on Computational Linguistics (COLING)*, 5756-5774. https://arxiv.org/abs/2406.00456v2

Zhu, L., Wang, X., & Wang, X. (2023,). *JudgeLM: Fine-tuned Large Language Models are Scalable Judges*.

A Estructura del Repositorio de Código

A.1 Organización de Carpetas

El repositorio del proyecto se organiza en dos grandes áreas: un directorio de exploración y módulos auxiliares (`archivo/`) y un directorio principal de experimentación (`examples/notebooks/`). A continuación se presenta el árbol de directorios con los componentes principales.

```
tesis/
├── data/                          # Datos del benchmark y código de minería
│   ├── evaluation_questions_*.json
│   └── codigo_de_mineria_*.json
├── evaluation_dataset/            # Conjunto de datos de evaluación
├── system_description.md          # Descripción arquitectónica del sistema
│
├── archivo/                       # Módulos auxiliares y exploración
│   ├── 01_bo_web_scrapping/       # Scraping del Boletín Oficial
│   ├── 01_descarga_normas_infoleg/# Descarga de normas desde Infoleg
│   ├── 02_html_infoleg_parser/    # Parser de HTML a texto estructurado
│   ├── 02_text_extraction/        # Extracción de texto de PDFs
│   ├── 03_relational_database_creation/
│   ├── 04_vector_database_creation/ # Docker Compose para pgvector
│   ├── 05_graph_db_creation/
│   ├── 06_llm_interface/
│   ├── 07_evaluation/
│   └── bo_llm_fine_tuning/        # Experimentos de fine-tuning
│
├── examples/notebooks/            # Pipeline principal de experimentación
│   ├── main.py                    # Punto de entrada
│   ├── pyproject.toml             # Dependencias del proyecto
│   ├── performed_tasks_summary.md # Resumen de tareas realizadas
│   ├── notebooks/                 # Exploración y descarga de datos
│   ├── part_a/                    # Generación de preguntas del benchmark
│   ├── part_b/                    # (Reservado)
│   ├── part_c/                    # Creación de base de datos vectorial
│   ├── part_d/                    # Pruebas de recuperación baseline
│   ├── part_e/                    # Respuestas sin RAG + métricas
│   ├── part_f/                    # Respuestas con RAG básico + métricas
│   ├── part_g/                    # Respuestas con Agentic-RAG + métricas
│   ├── part_h/                    # Integración de datos
│   ├── part_h_1/                  # Evaluación LLM-as-a-Judge
│   ├── part_i/                    # Preparación de datos para análisis
│   └── part_j/                    # Análisis estadístico
│
└── 00_escritura/                  # Manuscrito de la tesis (Typst)
```

A.2 Descripción de Componentes Principales

A.2.1 Módulos de Exploración y Extracción (`archivo/`)

- **`01_bo_web_scrapping/`**: scraping del Boletín Oficial de la República Argentina mediante notebooks de Jupyter. Incluye almacenamiento de datos crudos en `bos_data/`.
- **`01_descarga_normas_infoleg/`**: descarga asíncrona de textos legales desde el sitio Infoleg del Ministerio de Justicia. Implementado como módulo Python con `src/`, tests y logs de ejecución.
- **`02_html_infoleg_parser/`**: parser que transforma HTML descargado de Infoleg en texto estructurado, preservando metadatos de artículos (número, título, capítulo, sección). Incluye Dockerfile para reproducibilidad, HTMLs de ejemplo y salidas de referencia.
- **`02_text_extraction/`**: extracción de texto a partir de documentos PDF escaneados, empleando modelos de visión (o4-mini de OpenAI) para OCR estructurado.
- **`04_vector_database_creation/`**: configuración de Docker Compose para el despliegue de PostgreSQL con la extensión pgvector, utilizada como alternativa local a Milvus para almacenamiento de embeddings.
- **`bo_llm_fine_tuning/`**: experimentos preliminares de fine-tuning de modelos de lenguaje con datos del Boletín Oficial.

A.2.2 Pipeline de Experimentación (`examples/notebooks/`)

Directorio	Función
`notebooks/`	Exploración inicial: descarga del Código de Minería desde Infoleg (Docling y Requests), parsing de HTML/PDF, pruebas de Pydantic AI.
`part_a/`	Generación del benchmark: segmentación del Código de Minería por títulos y generación de preguntas tipo caso de estudio legal mediante GPT-4.1, con respuestas esperadas y artículos de referencia.
`part_c/`	Creación de la base de datos vectorial Milvus con embeddings de 768 dimensiones generados por el modelo `hiiamsid/sentence_similarity_spanish_es`. Población con artículos del Código de Minería.
`part_d/`	Pruebas de recuperación baseline: evaluación de la precisión de búsqueda vectorial para las preguntas del benchmark.
`part_e/`	Generación de respuestas sin RAG con tres modelos (gpt-5-nano, gpt-5--mini, gpt-5). Cálculo de más de 11 métricas de similitud textual y semántica.
`part_f/`	Generación de respuestas con RAG básico: búsqueda vectorial top-10 inyectada como contexto en el prompt. Mismos modelos y métricas.
`part_g/`	Generación de respuestas con Agentic-RAG: arquitectura de dos agentes activos (Especialista Legal, Recuperación) implementada con Pydantic AI. Un tercer agente (Calidad) fue diseñado e implementado pero desactivado en la configuración experimental final.
`part_h/`	Integración de resultados de las tres estrategias en un archivo JSON unificado con todas las métricas.
`part_h_1/`	Evaluación LLM-as-a-Judge mediante `pydantic_evals` con tres jueces (DeepSeek, Grok, Kimi K2) y dos rúbricas de evaluación.
`part_i/`	Transformación de datos JSON anidados a CSVs tabulares. Visualizaciones con Polars y plotnine (ggplot2).
`part_j/`	Análisis estadístico en R/Python: exploración descriptiva, tests de normalidad, Kruskal-Wallis con post-hoc de Dunn, modelos mixtos y análisis de costos.

Tabla 17: Descripción de los directorios del pipeline de experimentación.

A.2.3 Principales Dependencias del Proyecto

Librería	Versión	Función
`pydantic-ai`	`1.1.0`	Framework de agentes con herramientas
`pymilvus`	`≥2.6.3`	Cliente de la base de datos vectorial Milvus
`sentence-transformers`	`≥5.1.1`	Generación de embeddings
`openai`	`≥1.97.1`	Cliente del proveedor de LLM
`polars`	`≥1.35.2`	Transformación y análisis de datos
`bert-score`	`≥0.3.13`	Métrica semántica BERTScore
`rouge-score`	`≥0.1.2`	Métricas ROUGE
`levenshtein`	`≥0.27.1`	Distancia de edición
`scikit-learn`	`≥1.7.2`	Utilidades de aprendizaje automático
`plotnine`	`≥0.15.1`	Visualización estilo ggplot2
`sqlmodel`	`≥0.0.24`	ORM para bases de datos SQL
`krippendorff`	`≥0.8.2`	Métricas de acuerdo inter-evaluador

Tabla 18: Principales dependencias del proyecto (Python ≥ 3.13).

B Ejemplos de Consultas y Respuestas

Este anexo presenta ejemplos representativos de consultas y respuestas del sistema para las tres estrategias evaluadas. Se seleccionó la primera subpregunta del benchmark (dominio del yacimiento de litio) para ilustrar las diferencias entre las configuraciones.

B.1 Consulta de Ejemplo

Escenario: El ingeniero Juan Pérez ha descubierto un importante yacimiento de litio en una finca privada ubicada en territorio de la Provincia de Salta. La propietaria de la finca, la Sra. Marta Gómez, sostiene que, por ser dueña del suelo, es también dueña exclusiva del litio allí existente y que nadie puede explotarlo sin su consentimiento. Juan Pérez, por su parte, desea solicitar una concesión de explotación al Estado, argumentando que el litio es propiedad del Estado.

Subpregunta 1: Analice a quién corresponde el dominio del yacimiento de litio descubierto y determine a qué categoría de minas pertenece dicho yacimiento, fundamentando su respuesta en los artículos pertinentes del Código de Minería.

Respuesta esperada: El yacimiento de litio descubierto corresponde a la primera categoría de minas, según el artículo 3° inciso a) del Código de Minería Argentino. Esta categoría está compuesta por sustancias metalíferas, incluyendo expresamente el litio. Las minas de la primera categoría «pertenecen exclusivamente al Estado, y sólo pueden explotarse en virtud de concesión legal otorgada por autoridad competente» (art. 2° inc. 1). Por lo tanto, el dominio originario del yacimiento es estatal, aunque la mina se encuentre en una propiedad privada.

Artículos de referencia: Art. 2 (inciso 1), Art. 3 (inciso a).

B.2 Consultas sin RAG

A continuación se presenta la respuesta generada por el modelo gpt-5-nano sin acceso a contexto adicional.

Modelo: gpt-5-nano (sin RAG)
Tokens de entrada: 315 | **Tokens de salida:** 7818 | **Costo:** USD 0,003

Respuesta (extracto): «Dominio del yacimiento: correspondería a la Nación. La categoría es dominio de minas y yacimientos de minerales en el subsuelo (derecho minero)…»

La respuesta es extensa (ratio de longitud 4,62 respecto a la esperada) y carece de citaciones precisas a artículos específicos del Código de Minería. El modelo recurre a conocimiento general sobre derecho minero sin poder fundamentar en artículos concretos.

Figura 28: Respuesta sin RAG (gpt-5-nano) para la subpregunta 1.1.

Métrica	Valor
Exact Match	0,000
Token F1	0,198
Jaccard	0,110
Levenshtein	0,173
BLEU	0,018
ROUGE-1	0,208
ROUGE-2	0,068
SBERT Similarity (español)	0,797
BERTScore	0,710
Ratio de longitud	4,62

Tabla 19: Métricas de la respuesta sin RAG (gpt-5-nano) para la subpregunta 1.1.

Se observa que las métricas léxicas (Token F1, Jaccard, BLEU) son bajas debido a la extensión excesiva y la ausencia de terminología legal específica. Las métricas semánticas (SBERT, BERTScore) capturan una similitud temática moderada, indicando que el modelo aborda el tema correcto sin la precisión requerida.

B.3 Consultas con RAG Básico

En la configuración de RAG básico, el sistema realiza una búsqueda vectorial que retorna los 10 fragmentos más similares del Código de Minería. Estos fragmentos se inyectan en el prompt del sistema como contexto antes de la generación.

Modelo: gpt-5-nano (RAG básico, top-10)

Al disponer de los artículos relevantes como contexto, la respuesta cita correctamente el artículo 3° inciso a) y el artículo 2° inciso 1, identificando el litio como mina de primera categoría perteneciente al Estado. La extensión de la respuesta se reduce significativamente y la argumentación se estructura en torno a los artículos recuperados.

Figura 29: Respuesta con RAG básico (gpt-5-nano) para la subpregunta 1.1.

La mejora principal del RAG básico se manifiesta en las métricas léxicas, que reflejan una mayor coincidencia terminológica con la respuesta esperada, y en las métricas de evaluación por jueces, donde la precisión legal mejora por la capacidad de citar artículos específicos.

B.4 Consultas con Agentic-RAG

En la configuración Agentic-RAG, el flujo de procesamiento activo involucra dos agentes:

1. **Agente Especialista Legal:** recibe la pregunta y determina que necesita información del Código de Minería. Invoca la herramienta `retrieve_relevant_documents`.

2. **Agente de Recuperación:** descompone la consulta en subconsultas semánticas (por ejemplo, «categorías de minas litio», «dominio estatal minas primera categoría»). Ejecuta múltiples búsquedas vectoriales (hasta 6) y consolida los resultados.

3. **Agente Especialista Legal:** con los fragmentos recuperados, genera la respuesta final citando los artículos pertinentes.

Nota: un tercer agente —el Agente de Calidad— fue diseñado e implementado para evaluar la pertinencia de los fragmentos recuperados y proveer retroalimentación, pero fue desactivado en la configuración experimental final.

Modelo: gpt-5-nano (Agentic-RAG)

El sistema agéntico recupera los artículos 2° y 3° del Código de Minería con alta precisión, gracias a la descomposición de la consulta por el Agente de Recuperación. La respuesta identifica correctamente la clasificación del litio como mina de primera categoría, establece el dominio exclusivo del Estado y fundamenta en los artículos pertinentes con mayor especificidad que el RAG básico.

Figura 30: Respuesta con Agentic-RAG (gpt-5-nano) para la subpregunta 1.1.

La ventaja del enfoque agéntico se evidencia en la capacidad de descomponer consultas complejas en búsquedas más específicas, lo que mejora la precisión de la recuperación y, consecuentemente, la calidad de la respuesta generada.

C Benchmark Completo

Este anexo presenta el benchmark utilizado para la evaluación del sistema. El conjunto de datos consta de 25 preguntas principales tipo caso de estudio legal, que se descomponen en 76 subpreguntas explícitas más 3 preguntas evaluadas como unidad única, totalizando 79 unidades de evaluación. Las preguntas fueron generadas a partir de los títulos del Código de Minería Argentino mediante GPT-4.1, y las respuestas esperadas fueron revisadas y validadas por profesionales del derecho argentino con experiencia en legislación minera.

C.1 Metadatos del Benchmark

Campo	**Valor**
Título	Evaluación derecho minero argentino
Norma evaluada	Código de Minería de la República Argentina
Jerarquía normativa	Código
Fecha de sanción	1994-01-01
Última actualización	2024-12-09
Estado	En vigencia
Autoridad de sanción	Congreso de la Nación Argentina
Total de preguntas principales	25
Total de unidades de evaluación	79
Tipo de pregunta	Caso de estudio legal con múltiples subpreguntas
Generación	GPT-4.1 a partir de títulos del Código de Minería
Validación	Revisión por profesionales del derecho argentino

Tabla 20: Metadatos del benchmark de evaluación.

C.2 Estructura del Benchmark

Las 25 preguntas se distribuyen entre los títulos del Código de Minería Argentino, cubriendo temas como: dominio originario de las minas y clasificación (Título I), conflictos de interés de funcionarios judiciales (Título II), permisos de exploración y límites legales (Título III), derechos del descubridor y registro de minas (Título IV), dimensiones de pertenencias para sustancias de primera categoría (Título V), obligaciones ambientales y servidumbres mineras (Título VI), entre otros.

Cada pregunta se formula como un caso de estudio legal con un escenario narrativo, seguido de subpreguntas específicas que requieren la identificación y aplicación de artículos concretos del Código de Minería. Las respuestas esperadas incluyen la fundamentación legal con citación de los artículos pertinentes. El benchmark completo está disponible en el repositorio de código del proyecto, en el directorio `data/`.

C.3 Ejemplos de Preguntas

A continuación se presentan dos preguntas representativas del benchmark a modo de ejemplo.

C.3.1 Pregunta 1: Dominio de Yacimiento de Litio en Propiedad Privada

Escenario: El ingeniero Juan Pérez descubre un yacimiento de litio en una finca privada en Salta. La propietaria (Marta Gómez) reclama propiedad exclusiva por ser dueña del suelo. Juan Pérez desea solicitar concesión al Estado.

Subpreguntas:

1. Analice a quién corresponde el dominio del yacimiento de litio y determine a qué categoría de minas pertenece.
2. ¿Puede el Estado otorgar la concesión de explotación a Juan Pérez sin el consentimiento de la propietaria del terreno?
3. Explique si la Sra. Gómez puede impedir legítimamente la explotación minera en su propiedad.

Artículos de referencia: 2 (inc. 1), 3 (inc. a), 7, 8, 10, 11, 13, 17.

C.3.2 Pregunta 2: Conflictos de Interés de Funcionario Judicial con Actividad Minera

Escenario: El juez Raúl Juárez posee el 50% de participación en una mina adquirida antes de su designación. Posteriormente, su esposa hereda otra mina en el mismo distrito. Un ingeniero estatal intenta reclamar derechos sobre ambas minas.

Subpreguntas:

1. ¿Puede el juez Juárez mantener la participación minera adquirida antes de su nombramiento?
2. ¿Puede la esposa del juez adquirir legalmente una mina por herencia pese a la prohibición de conflicto de interés?
3. ¿Puede el ingeniero estatal reclamar derechos sobre las minas del juez invocando las restricciones legales?

Artículos de referencia: 22 (inc. 2, 3), 23, 24.

C.4 Respuestas Esperadas (Ejemplos)

Las respuestas esperadas del benchmark se elaboraron a partir del texto vigente del Código de Minería Argentino. Cada respuesta incluye la fundamentación legal con citación de artículos específicos. A continuación se presentan las respuestas esperadas para las dos preguntas de ejemplo.

C.4.1 Respuestas a la Pregunta 1

Subpregunta 1.1: El yacimiento de litio corresponde a la primera categoría de minas, según el artículo 3° inciso a) del Código de Minería. Esta categoría comprende sustancias metalíferas, incluyendo expresamente el litio. Las minas de primera categoría «pertenecen exclusivamente al Estado, y sólo pueden explotarse en virtud de concesión legal otorgada por autoridad competente» (art. 2° inc. 1). El dominio originario del yacimiento es estatal, independientemente de la propiedad del suelo.

Subpregunta 1.2: El Estado puede otorgar la concesión sin consentimiento de la propietaria del terreno. El artículo 7 establece que las minas son bienes privados de la

Nación o de las provincias según el territorio. Los artículos 10 y 11 determinan que sin perjuicio del dominio del Estado, las minas se conceden a los particulares por tiempo ilimitado. La propiedad minera constituye un derecho real separado de la propiedad del suelo (art. 8).

Subpregunta 1.3: La Sra. Gómez no puede impedir la explotación. La actividad minera es declarada de utilidad pública (art. 13), lo que prevalece sobre la propiedad superficiaria. La propietaria solo puede oponerse por razones de seguridad o ambientales (art. 17), pero no puede invocar la propiedad del suelo para impedirla.

C.4.2 Respuestas a la Pregunta 2

Subpregunta 2.1: El juez Juárez puede mantener la participación minera adquirida antes de su designación. El artículo 23 establece que las restricciones del artículo 22 no aplican a las minas adquiridas con anterioridad al nombramiento del funcionario.

Subpregunta 2.2: La esposa puede adquirir legalmente la mina por herencia. El artículo 22 inciso 3 prohíbe la adquisición de minas por ciertos funcionarios y sus familiares, pero el artículo 23 exceptúa las adquisiciones por sucesión hereditaria, que constituyen un modo de adquisición involuntario y protegido legalmente.

Subpregunta 2.3: El ingeniero estatal no puede reclamar derechos sobre las minas. El artículo 22 inciso 2 le prohíbe, como empleado del Estado vinculado a la actividad minera, adquirir minas o participar en sociedades mineras. El artículo 24 establece la nulidad de las adquisiciones realizadas en contravención del artículo 22, lo que afectaría cualquier intento del ingeniero por apropiarse de las minas.

Acrónimos y Siglas

Agentic-RAG Agentic Retrieval-Augmented Generation (Generación Aumentada por Recuperación Agéntica)
API Application Programming Interface (Interfaz de Programación de Aplicaciones)
BM25 Best Matching 25 (algoritmo de ranking)
BLEU Bilingual Evaluation Understudy (métrica de evaluación)
BERT Bidirectional Encoder Representations from Transformers
CER Character Error Rate (Tasa de Error de Caracteres)
CoT Chain of Thought (Cadena de Pensamiento)
CRF Conditional Random Fields (Campos Aleatorios Condicionales)
CSV Comma-Separated Values (Valores Separados por Comas)
GPU Graphics Processing Unit (Unidad de Procesamiento Gráfico)
GraphRAG Graph Retrieval-Augmented Generation
HNSW Hierarchical Navigable Small World (algoritmo de búsqueda)
HTML HyperText Markup Language (Lenguaje de Marcas de Hipertexto)
IA Inteligencia Artificial
IQR Interquartile Range (Rango Intercuartílico)
IVF_FLAT Inverted File Flat (índice vectorial)
JSON JavaScript Object Notation
LLM Large Language Model (Gran Modelo de Lenguaje)
LoRA Low-Rank Adaptation
MoE Mixture of Experts (Mezcla de Expertos)
MRR Mean Reciprocal Rank (Rango Recíproco Medio)
NDCG Normalized Discounted Cumulative Gain
NLI Natural Language Inference (Inferencia de Lenguaje Natural)
NLP Natural Language Processing (Procesamiento de Lenguaje Natural)
PCA Principal Component Analysis (Análisis de Componentes Principales)
PLN Procesamiento de Lenguaje Natural
POMDP Partially Observable Markov Decision Process
QLoRA Quantized Low-Rank Adaptation
RAG Retrieval-Augmented Generation (Generación Aumentada por Recuperación)
ROUGE Recall-Oriented Understudy for Gisting Evaluation (métrica de evaluación)
SAIJ Sistema Argentino de Información Jurídica
SBERT Sentence-BERT
SLM Small Language Model (Modelo de Lenguaje Pequeño)
SoK Systematization of Knowledge (Sistematización de Conocimiento)
SRH Scheirer-Ray-Hare (test estadístico no paramétrico)